

CSE225 - Developing Android Application

CA1 PROJECT REPORT

On

Fund Start

(A financing platform for startups, offering various funding options and resources to help new businesses grow and succeed.)

School of Computer Science & Engineering

LOVELY PROFESSIONAL UNIVERSITY



LOVELY
PROFESSIONAL
UNIVERSITY

Name - Samhitha Madala
Registration Number - 12214200
Roll No: 39

Name of the Project with Caption: Fund Start

Technical Requirements :

Developed an Android application using Kotlin and Android Studio. Implemented a custom toolbar and organized the main activity layout using ConstraintLayout inside a ScrollView. Added interactive dashboard cards for navigation to different activities such as Apply Funding, Explore Resources, Loan Calculator, and Pitch Idea. Set up click listeners and basic notification setup in MainActivity.

Description / Project Overview:

This project is a **Finance Startup Application** built natively for Android devices using **Kotlin** and **Android Studio**.

The app is designed to help startups easily apply for funding, explore useful resources, calculate loans, and pitch their startup ideas through a clean and intuitive interface.

The main screen uses a **ConstraintLayout** inside a **ScrollView** to keep the design responsive across different screen sizes. A **custom toolbar** was created to maintain a consistent and professional look throughout the app.

Each section of the dashboard consists of **interactive cards** that are clickable and navigate users to different activities such as Apply Funding, Explore Resources, Loan Calculator, and Pitch Idea. Basic **notification setup** was also implemented to alert users about important updates.

The overall structure of the application focuses on simplicity, performance, and ease of navigation for users.

This app is designed keeping in mind the needs of startup founders to quickly

access funding opportunities and tools necessary for their growth.

Features:

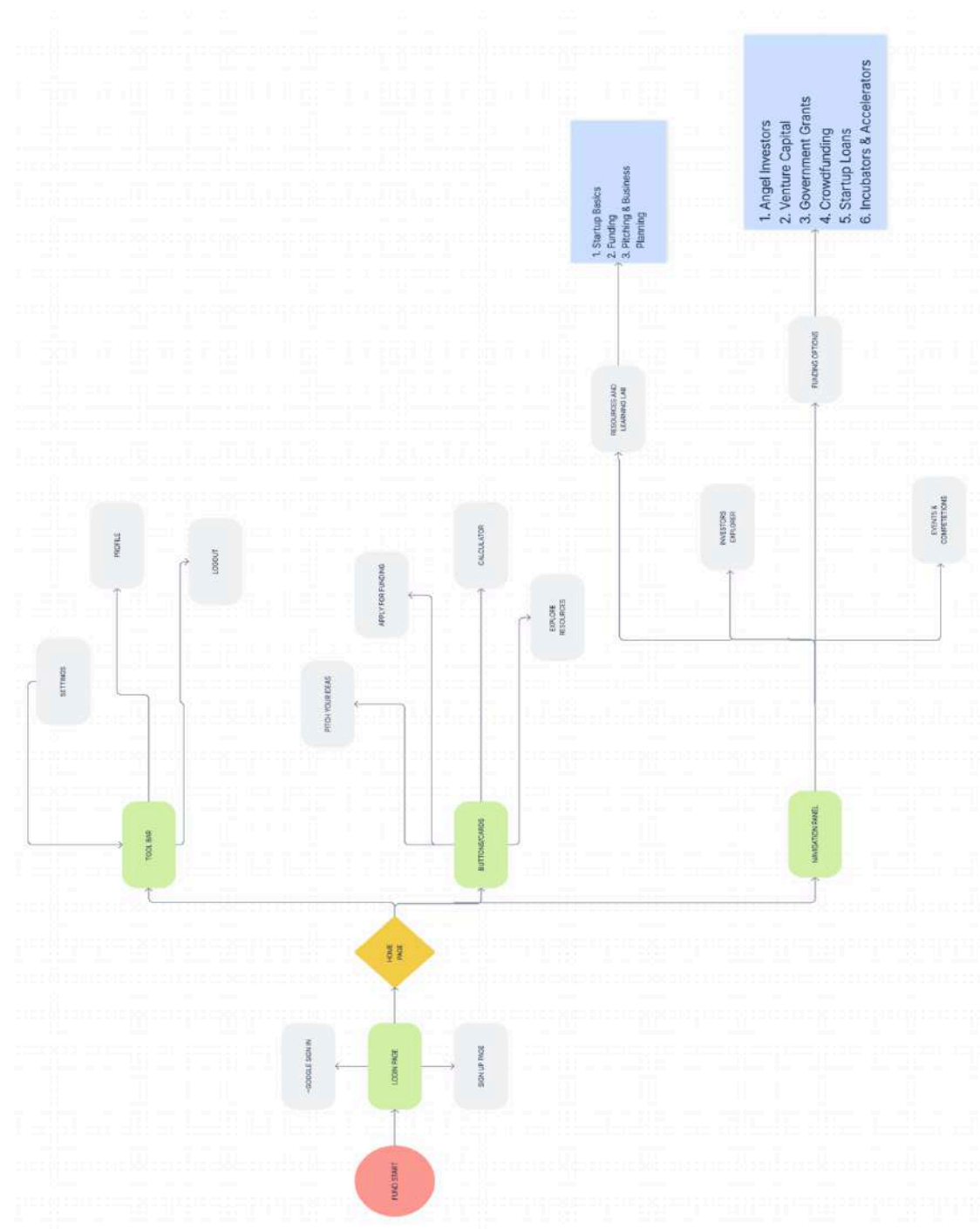
- » Responsive and clean UI using ConstraintLayout.
- » Smooth scrolling with ScrollView integration for the dashboard.
- » Custom toolbar with consistent navigation and title bar.
- » Interactive dashboard cards with click listeners for easy navigation to different features.
- » Separate activities for Apply Funding, Explore Resources, Loan Calculator, and Pitch Idea.
- » Basic notification functionality to keep users informed.
- » Designed to be compatible with various Android screen sizes and versions.

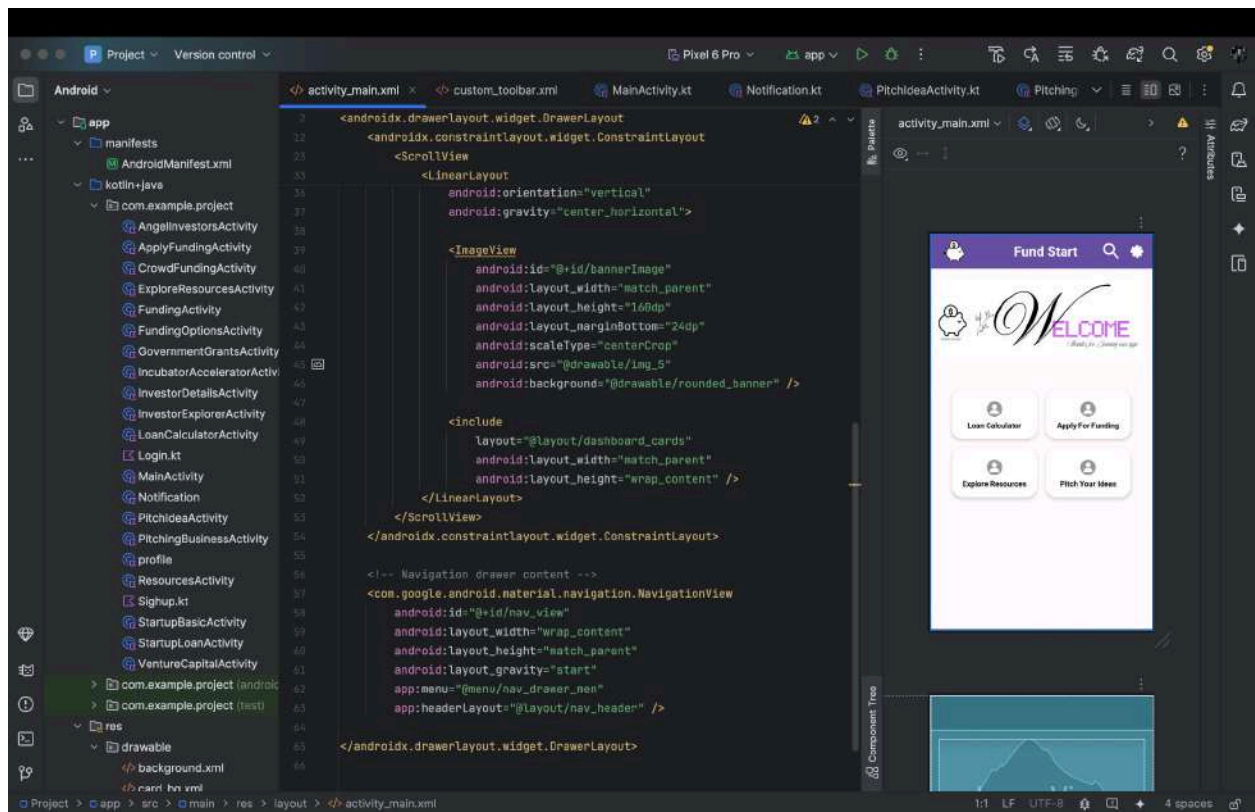
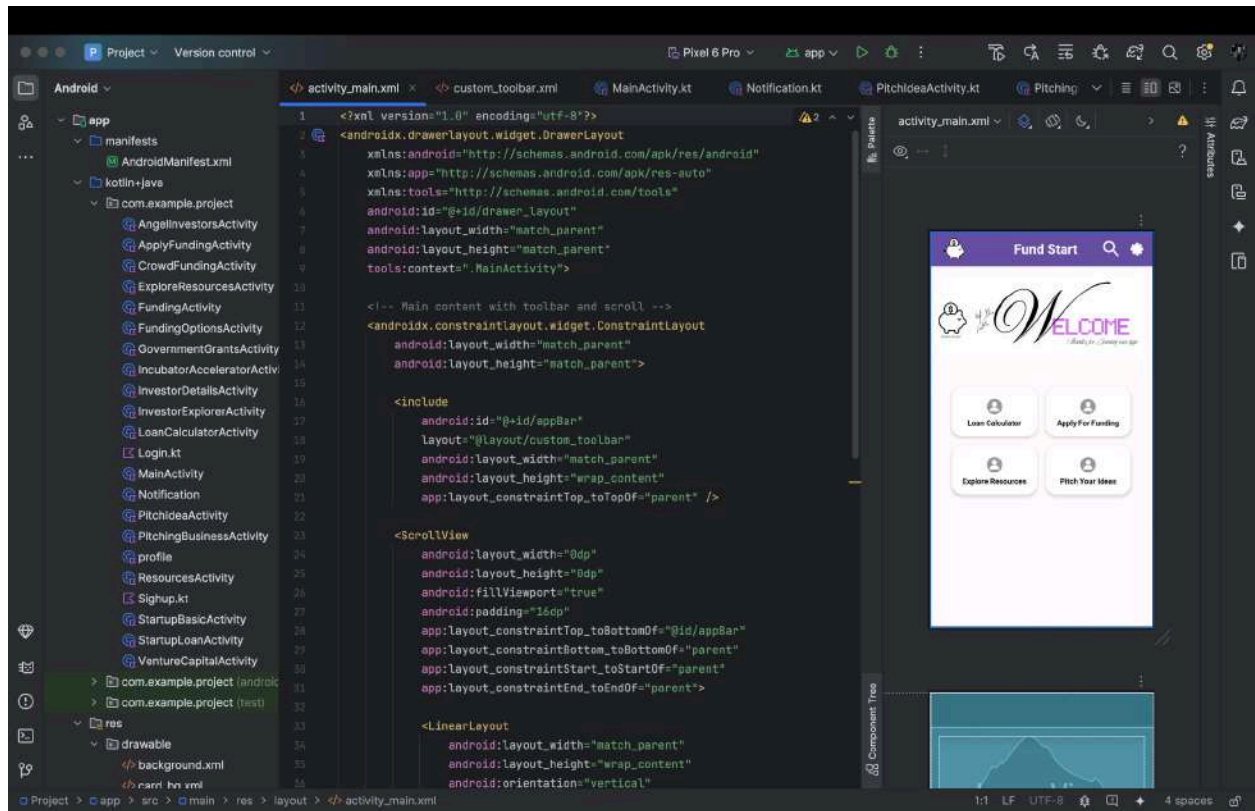
We had the following technologies / classes in this app:

- » Kotlin - Modern programming language used for building Android apps with clear and concise syntax.
- » ConstraintLayout - Used for creating responsive and flexible UI designs.
- » ScrollView - Used to allow vertical scrolling of dashboard contents.
- » Intent and Click Listeners - Used for navigation between activities.
- » NotificationManager - Used for setting up basic in-app notifications

FLOWCHART

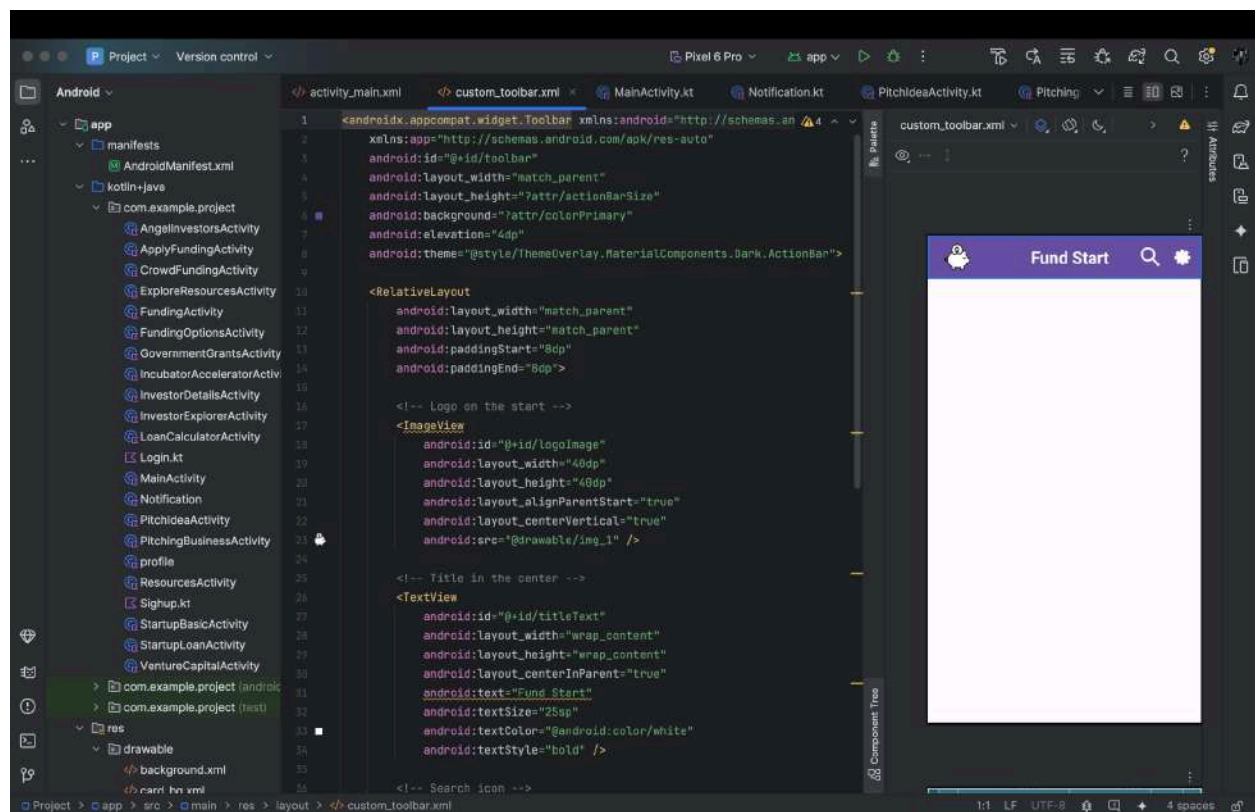
-> Activity_main.xml

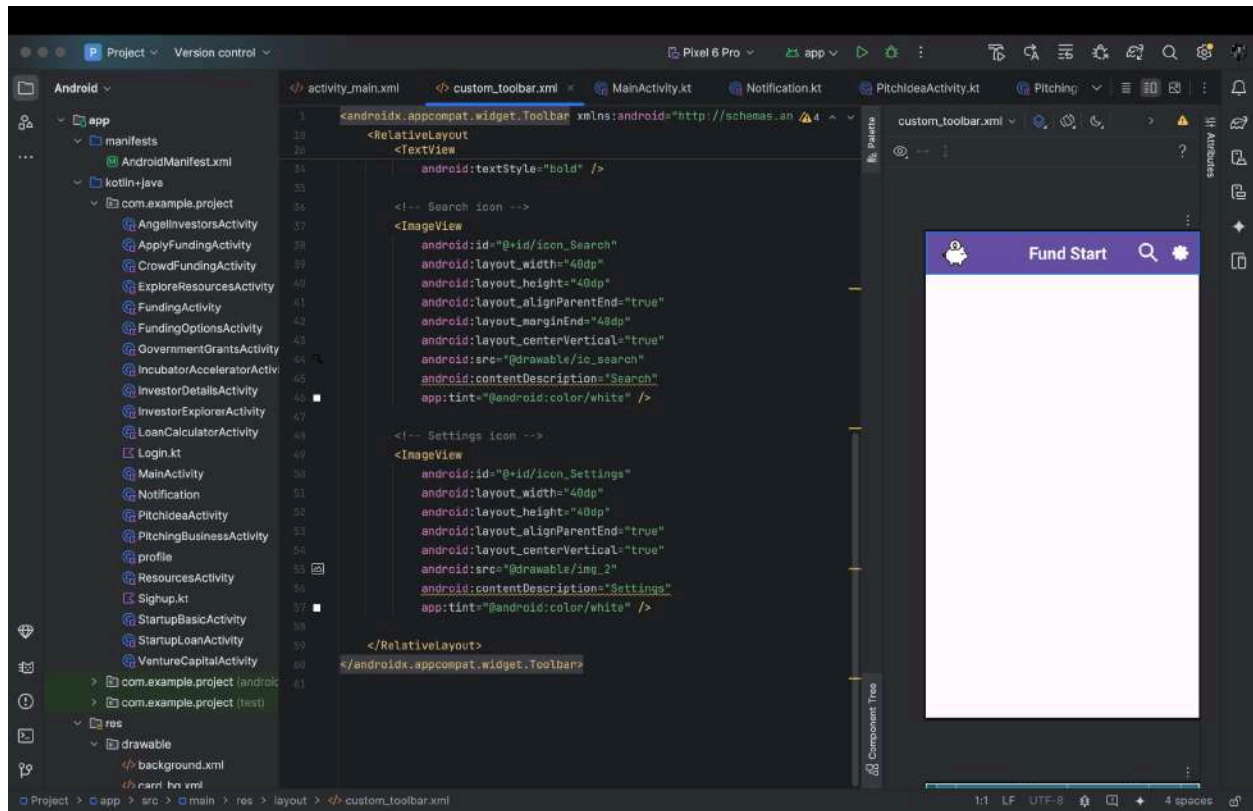




1. DrawerLayout is used as the root to allow a side navigation drawer.
2. ConstraintLayout is used inside DrawerLayout to design the main screen structure.
3. Custom Toolbar is added using `<include>` to have a consistent top bar.
4. ScrollView is added below the toolbar to make the main content scrollable.
5. ImageView is placed inside ScrollView to show a banner image with rounded corners.
6. Dashboard Cards are included using another `<include>` to display clickable feature cards.
7. NavigationView is added for the side drawer menu with a header and menu items.

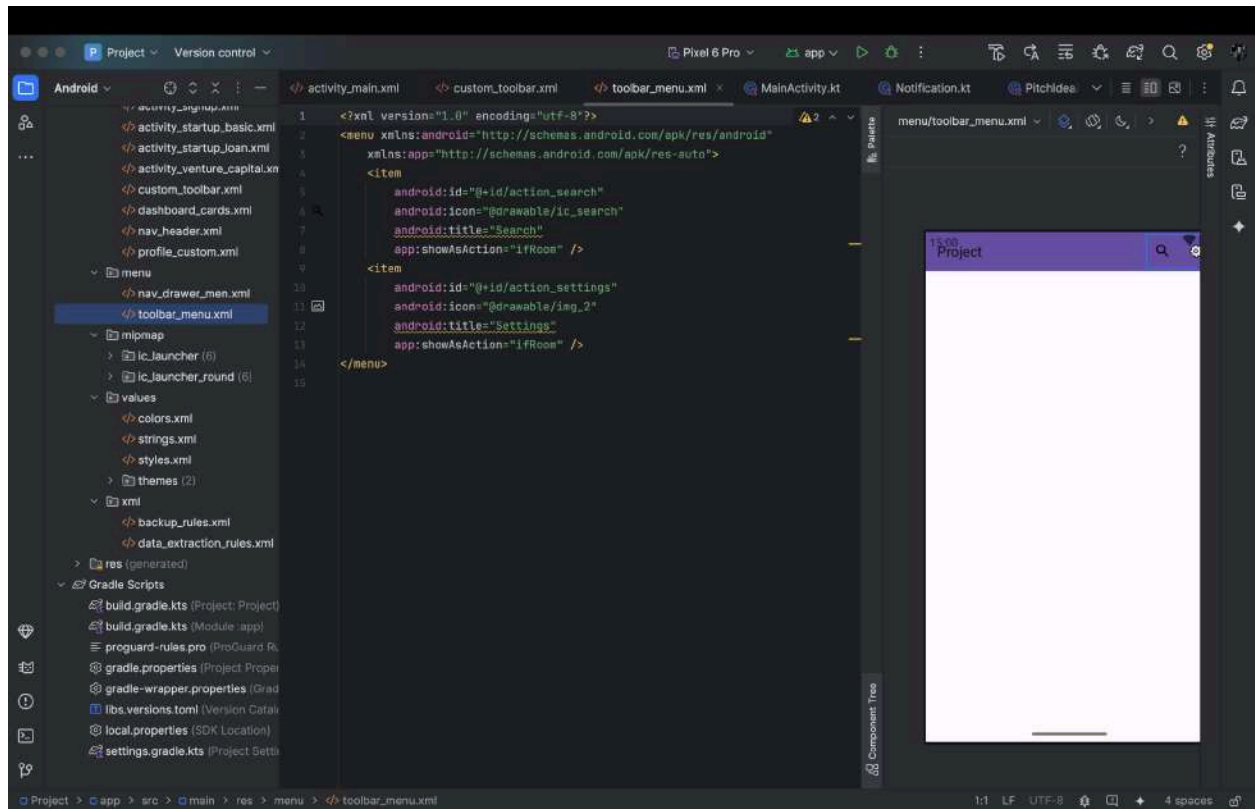
-> custom_toolbar.xml





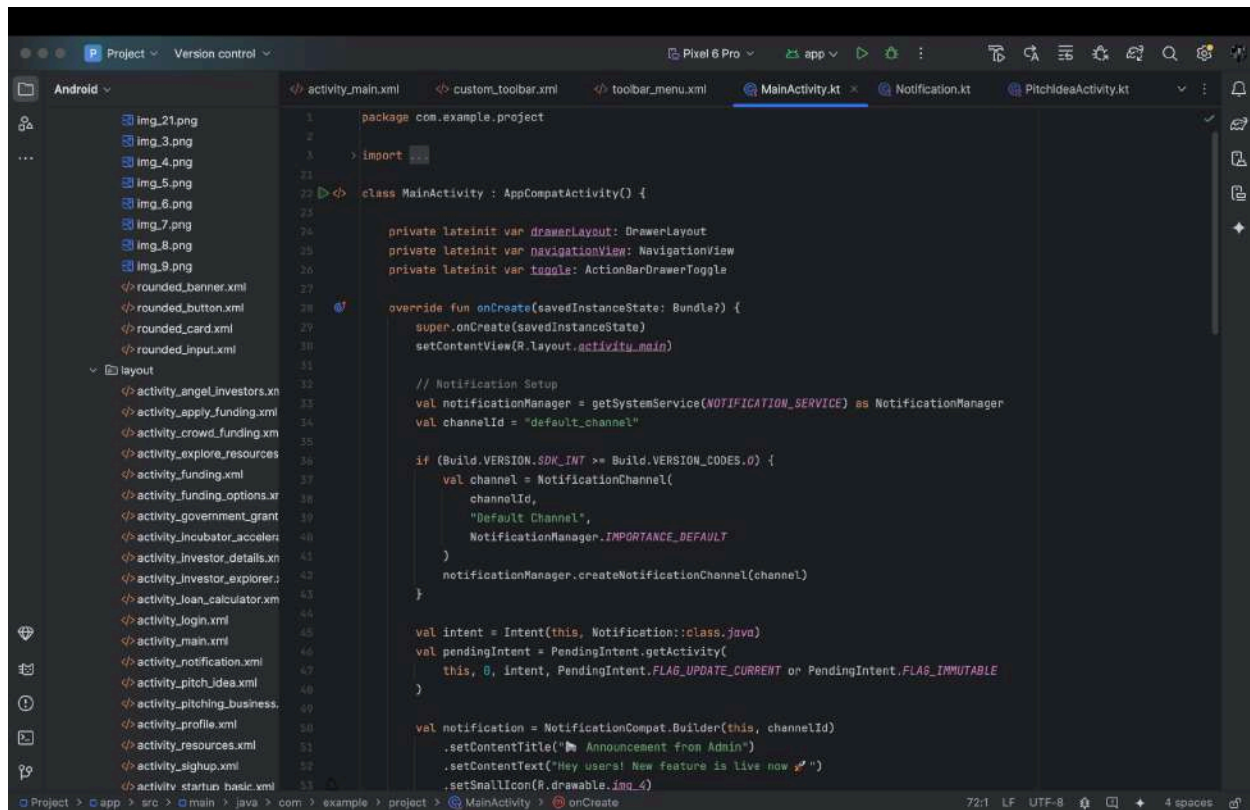
1. Toolbar is used as the container for the app bar with background color and elevation.
2. RelativeLayout is placed inside the Toolbar to arrange logo, title, and icons easily.
3. Logo ImageView is added at the start (left side) to show the app logo.
4. Title TextView is centered to display the app name "Fund Start" in bold white text.
5. Search Icon ImageView is added near the right side to represent search functionality.
6. Settings Icon ImageView is added at the rightmost corner for settings/options access.

3. toolbar_menu.xml



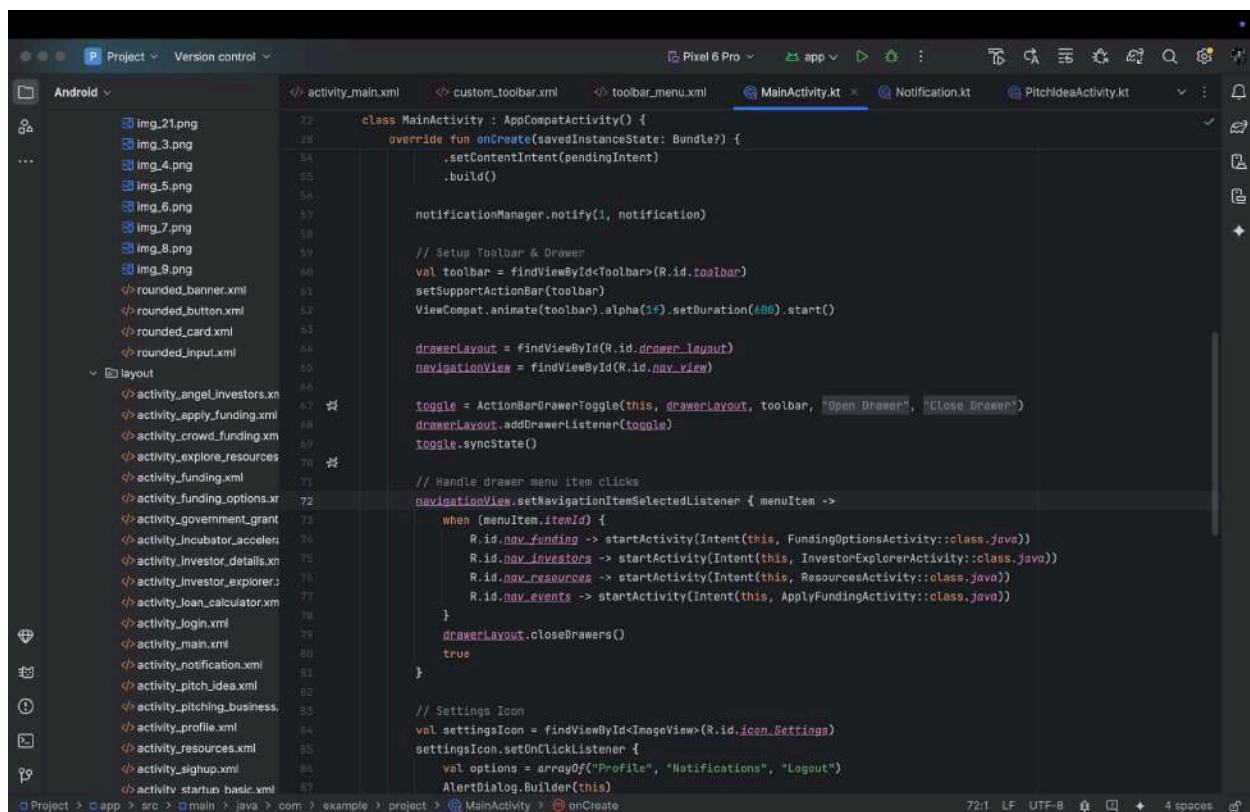
1. Menu Resource is created to define action items (Search and Settings) for the toolbar.
2. Search Item is added with an icon and title, shown directly in the toolbar if there is space.
3. Settings Item is added with an icon and title, also shown in the toolbar if there is space.
4. Relation to Toolbar: These menu items (Search and Settings) are linked with the [custom_toolbar.xml](#) icons and can be triggered when clicked.
5. Relation to Main Layout: When the toolbar is included in the [activity_main.xml](#), this menu adds functional actions (like search and settings) on the app bar.

-> MainActivity.kt



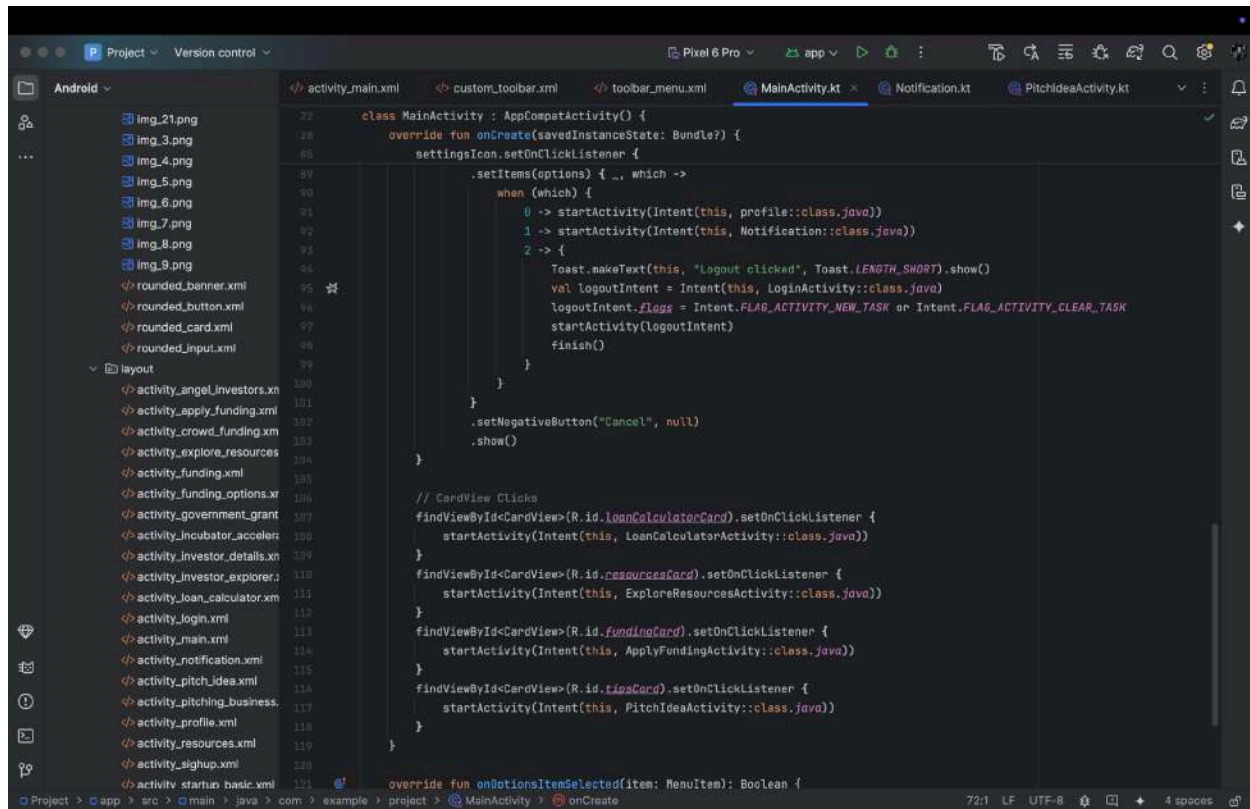
This screenshot shows the initial setup of MainActivity.kt. The code defines the MainActivity class, initializes notification services, and sets up a notification channel. It also includes a pending intent for an announcement.

```
1 package com.example.project
2
3 import androidx.appcompat.app.AppCompatActivity
4
5 class MainActivity : AppCompatActivity() {
6
7     private lateinit var drawerLayout: DrawerLayout
8     private lateinit var navigationView: NavigationView
9     private lateinit var toggle: ActionBarDrawerToggle
10
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         setContentView(R.layout.activity_main)
14
15         // Notification Setup
16         val notificationManager = getSystemService(NOTIFICATION_SERVICE) as NotificationManager
17         val channelId = "default_channel"
18
19         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
20             val channel = NotificationChannel(
21                 channelId,
22                 "Default Channel",
23                 NotificationManager.IMPORTANCE_DEFAULT
24             )
25             notificationManager.createNotificationChannel(channel)
26         }
27
28         val intent = Intent(this, Notification::class.java)
29         val pendingIntent = PendingIntent.getActivity(
30             this, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
31         )
32
33         val notification = NotificationCompat.Builder(this, channelId)
34             .setContentTitle("📢 Announcement from Admin")
35             .setContentText("Hey users! New feature is live now 🎉")
36             .setSmallIcon(R.drawable.img_4)
37     }
38 }
```



This screenshot shows the continuation of MainActivity.kt. It includes the logic for displaying the notification, setting up the toolbar and drawer, and handling navigation events.

```
39
40     notificationManager.notify(1, notification)
41
42     // Setup Toolbar & Drawer
43     val toolbar = findViewById<Toolbar>(R.id.toolbar)
44     setSupportActionBar(toolbar)
45     ViewCompat.animate(toolbar).alpha(1f).setDuration(400).start()
46
47     drawerLayout = findViewById(R.id.drawer_layout)
48     navigationView = findViewById(R.id.nav_view)
49
50     toggle = ActionBarDrawerToggle(this, drawerLayout, toolbar, "Open Drawer", "Close Drawer")
51     drawerLayout.addDrawerListener(toggle)
52     toggle.syncState()
53
54     // Handle drawer menu item clicks
55     navigationView.setNavigationItemSelectedListener { menuItem ->
56         when (menuItem.itemId) {
57             R.id.nav_funding -> startActivity(Intent(this, FundingOptionsActivity::class.java))
58             R.id.nav_investors -> startActivity(Intent(this, InvestorExplorerActivity::class.java))
59             R.id.nav_resources -> startActivity(Intent(this, ResourcesActivity::class.java))
60             R.id.nav_events -> startActivity(Intent(this, ApplyFundingActivity::class.java))
61         }
62         drawerLayout.closeDrawers()
63         true
64     }
65
66     // Settings Icon
67     val settingsIcon = findViewById<ImageView>(R.id.icon_settings)
68     settingsIcon.setOnClickListener {
69         val options = arrayOf("Profile", "Notifications", "Logout")
70         AlertDialog.Builder(this)
71             .setTitle("Settings")
72             .setItems(options)
73             .setPositiveButton("OK") { dialog, _ -> }
74             .show()
75     }
76 }
```

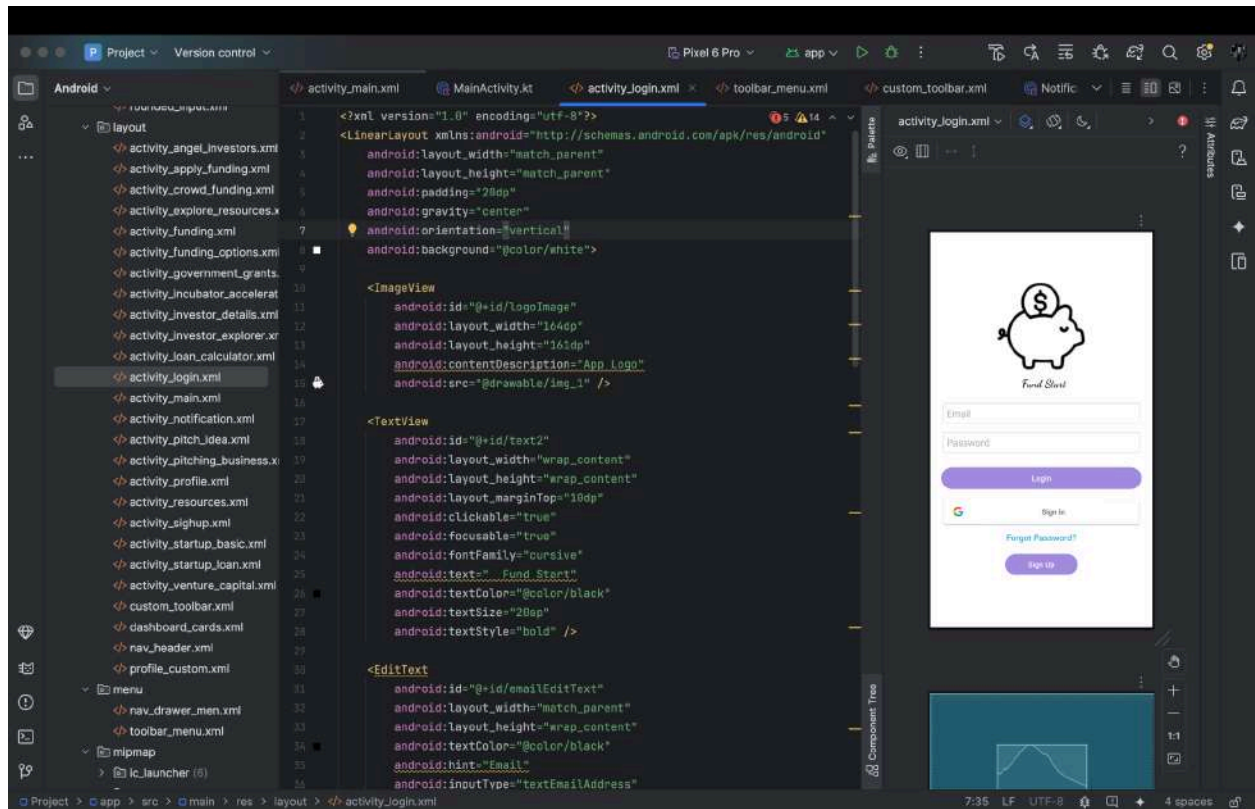


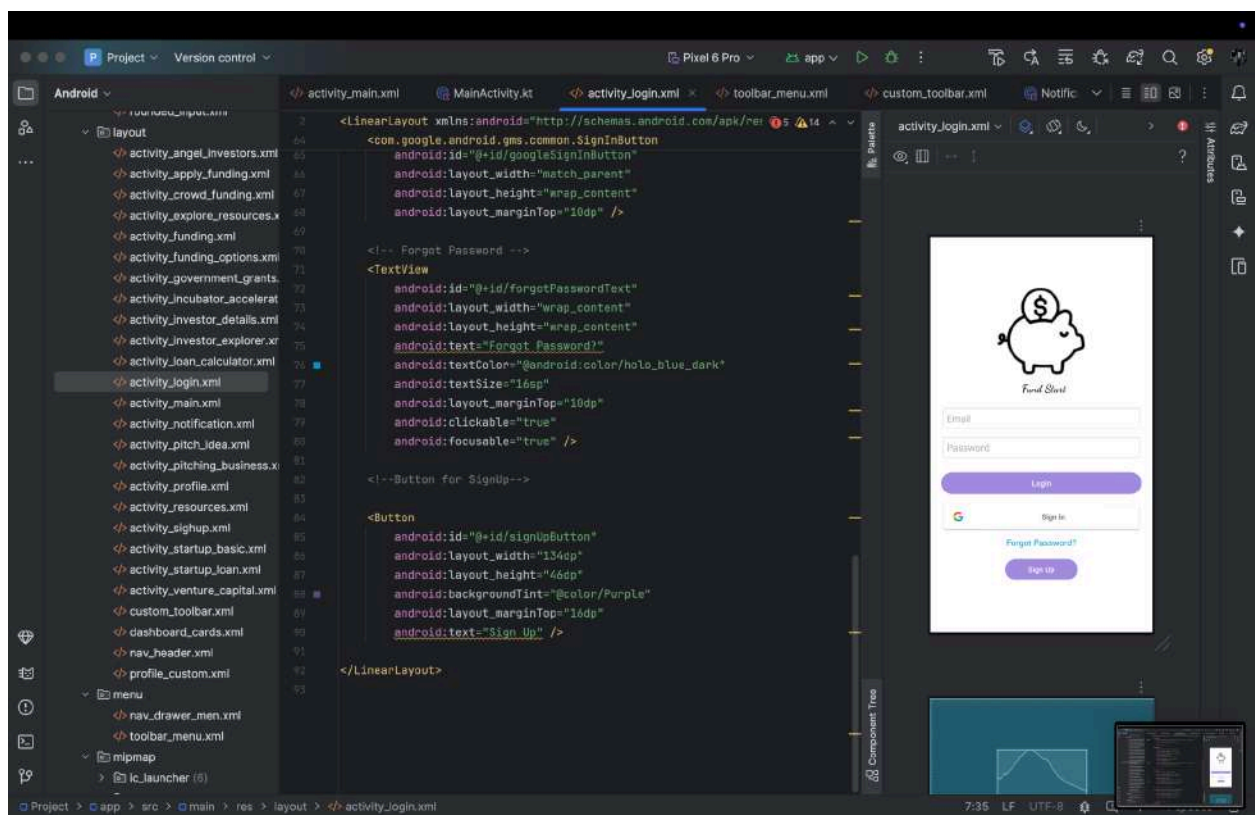
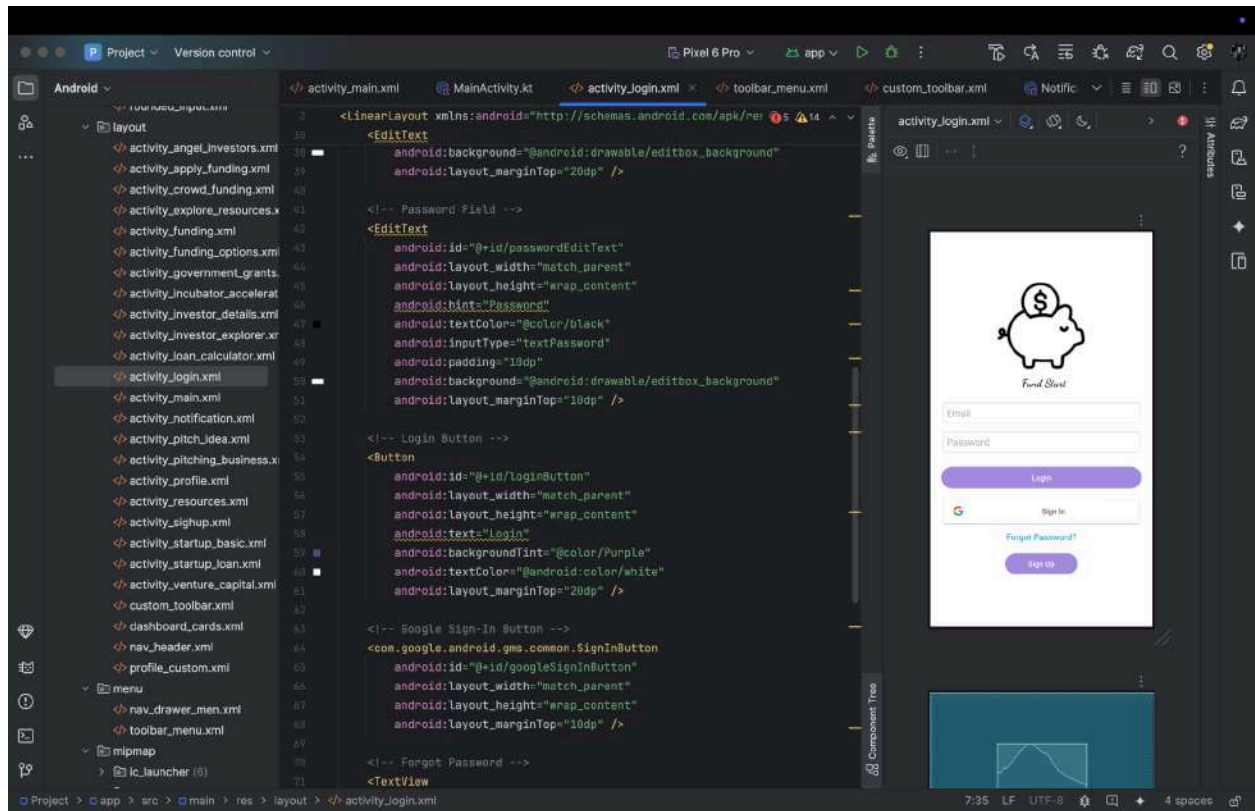
1. Set Layout: Loaded `activity_main.xml` using `setContentview`.
2. Notification Setup: Created a notification channel and sent a welcome notification.
3. Toolbar Setup: Found and set `custom_toolbar.xml` as the action bar with animation.
4. Drawer Setup: Connected `DrawerLayout` with `ActionBarDrawerToggle` for hamburger menu.
5. Navigation Menu: Handled navigation drawer item clicks to open different activities.
6. Settings Icon: On clicking Settings, showed an AlertDialog with Profile, Notifications, and Logout options.
7. CardView Clicks: Handled dashboard card clicks to navigate to different app features like Loan Calculator, Resources, Funding, and Pitch Idea sections.

8. Menu Item Handling: Enabled opening and closing of the navigation drawer from toolbar menu.

-> **activity_login.xml**

This is a activity file for login page :





1. `LinearLayout`: Main vertical container with padding and centered content.
2. Logo Image: Shows app logo at the top.
3. App Title: Displays "Fund Start" name below logo.
4. Email Field: `EditText` to enter a user's email.
5. Password Field: `EditText` to enter user's password.
6. Login Button: Button to log into the app.
7. Google Sign-In: Google button for signing in with Google account.
8. Forgot Password: Text link to recover password.
9. Sign Up Button: Button to register a new user.

-> Login.kt

1. Firebase Authentication: Initializes Firebase Auth (`auth = FirebaseAuth.getInstance()`).
2. Google Sign-In Setup: Configures Google Sign-In with `GoogleSignInOptions` and `GoogleSignInClient`.
3. Views: Retrieves views like `EditText` (email/password), `Button` (Login/SignUp), `TextView` (Forgot Password), and `SignInButton` (Google).
4. Google Sign-In: Launches Google Sign-In on button click and handles results in `onActivityResult`.
5. Login with Email: Authenticates using email and password with Firebase. On success, navigate to `MainActivity`.
6. Forgot Password: Sends password reset email using Firebase if email is entered.
7. Sign-Up: Redirects to `SignUpActivity` for new user registration.

```
Project | Version control | Pixel 6 Pro | app | 146:1 LF UTF-8 | 4 spaces

Android | app | manifests | AndroidManifest.xml | kotlin-jvm | com.example.project | Login.kt | MainActivity | Notification | PitchIdeaActivity | PitchingBusinessActivity | profile | ResourcesActivity | Signup.kt | StartupBasicActivity | StartupLoanActivity | VentureCapitalActivity | com.example.project (androidTest) | com.example.project (test) | res | drawable | background.xml | ...

activity_funding.xml | InvestorDetailsActivity.kt | InvestorExplorerActivity.kt | LoanCalculatorActivity.kt | Login.kt

1 package com.example.project
2
3 import ...
4
5 class LoginActivity : AppCompatActivity() {
6
7     private lateinit var auth: FirebaseAuth
8     private lateinit var googleSignInClient: GoogleSignInClient
9
10    @SuppressLint("MissingInflateId")
11    override fun onCreate(savedInstanceState: Bundle?) {
12        super.onCreate(savedInstanceState)
13        setContentView(R.layout.activity_login)
14
15        auth = FirebaseAuth.getInstance() // INITIALIZING FIREBASE
16
17        val gso = GoogleSignInOptions.Builder(GoogleSignInOptions.DEFAULT_SIGN_IN)
18            .requestIdToken("955687675445-d81akac9rs99480h8bcpik3haahknag.apps.googleusercontent.com")
19            .requestEmail()
20            .build()
21
22        googleSignInClient = GoogleSignIn.getClient(this, gso)
23
24        // Find Views
25        val emailEditText = findViewById<EditText>(R.id.emailEditText)
26        val passwordEditText = findViewById<EditText>(R.id.passwordEditText)
27        val loginButton = findViewById<Button>(R.id.loginButton)
28        val googleSignInButton = findViewById<Button>(R.id.googleSignInButton)
29        val forgotPasswordText = findViewById<TextView>(R.id.forgotPasswordText)
30
31        // Google Sign-In Button Click Listener
32        googleSignInButton.setOnClickListener {
33            val signInIntent = googleSignInClient.signInIntent
34            startActivityForResult(signInIntent, RC_SIGN_IN)
35        }
36
37        // Sign-Up Button Click Listener
38        findViewById<Button>(R.id.signupButton).setOnClickListener {
39            startActivity(Intent(this, SignupActivity::class.java))
40        }
41
42        // Login Button
43        loginButton.setOnClickListener {
44            val email = emailEditText.text.toString().trim()
45            val password = passwordEditText.text.toString().trim()
46
47            if (email.isEmpty() || password.isEmpty()) {
48                Toast.makeText(this, "Please enter email and password", Toast.LENGTH_SHORT).show()
49                return@setOnClickListener
50            }
51
52            auth.signInWithEmailAndPassword(email, password)
53                .addOnCompleteListener { task ->
54                    if (task.isSuccessful) {
55                        Toast.makeText(this, "Login Successful", Toast.LENGTH_SHORT).show()
56                        startActivity(
57                            Intent(
58                                this,
59                                MainActivity::class.java
60                            )
61                        ) // Navigate to Home Screen
62                        finish()
63                    } else {
64                        Toast.makeText(
65                            this,
66                            "Login Failed: ${task.exception?.message}",
67                            Toast.LENGTH_SHORT
68                        ).show()
69                    }
70                }
71        }
72    }
73 }
```

```
Project | Version control | Pixel 6 Pro | app | 146:1 LF UTF-8 | 4 spaces

Android | app | manifests | AndroidManifest.xml | kotlin-jvm | com.example.project | Login.kt | MainActivity | Notification | PitchIdeaActivity | PitchingBusinessActivity | profile | ResourcesActivity | Signup.kt | StartupBasicActivity | StartupLoanActivity | VentureCapitalActivity | com.example.project (androidTest) | com.example.project (test) | res | drawable | background.xml | ...

activity_funding.xml | InvestorDetailsActivity.kt | InvestorExplorerActivity.kt | LoanCalculatorActivity.kt | Login.kt

16 class LoginActivity : AppCompatActivity() {
17     override fun onCreate(savedInstanceState: Bundle?) {
18         startActivityForResult(signInIntent, RC_SIGN_IN)
19     }
20
21     // Sign-Up Button Click Listener
22     findViewById<Button>(R.id.signupButton).setOnClickListener {
23         startActivity(Intent(this, SignupActivity::class.java))
24     }
25
26     // Login Button
27     loginButton.setOnClickListener {
28         val email = emailEditText.text.toString().trim()
29         val password = passwordEditText.text.toString().trim()
30
31         if (email.isEmpty() || password.isEmpty()) {
32             Toast.makeText(this, "Please enter email and password", Toast.LENGTH_SHORT).show()
33             return@setOnClickListener
34         }
35
36         auth.signInWithEmailAndPassword(email, password)
37             .addOnCompleteListener { task ->
38                 if (task.isSuccessful) {
39                     Toast.makeText(this, "Login Successful", Toast.LENGTH_SHORT).show()
40                     startActivity(
41                         Intent(
42                             this,
43                             MainActivity::class.java
44                         )
45                     ) // Navigate to Home Screen
46                     finish()
47                 } else {
48                     Toast.makeText(
49                         this,
50                         "Login Failed: ${task.exception?.message}",
51                         Toast.LENGTH_SHORT
52                     ).show()
53                 }
54             }
55     }
56 }
```

```
Project | Version control | Pixel 6 Pro | app | Login.kt

Android | app | manifests | AndroidManifest.xml | kotlin+java | com.example.project | Login.kt | MainActivity | Notification | PitchIdeaActivity | PitchingBusinessActivity | profile | ResourcesActivity | Signup.kt | StartupBasicActivity | StartupLoanActivity | VentureCapitalActivity | com.example.project (androidTest) | com.example.project (test) | res | drawable | background.xml | raw | h1.vml

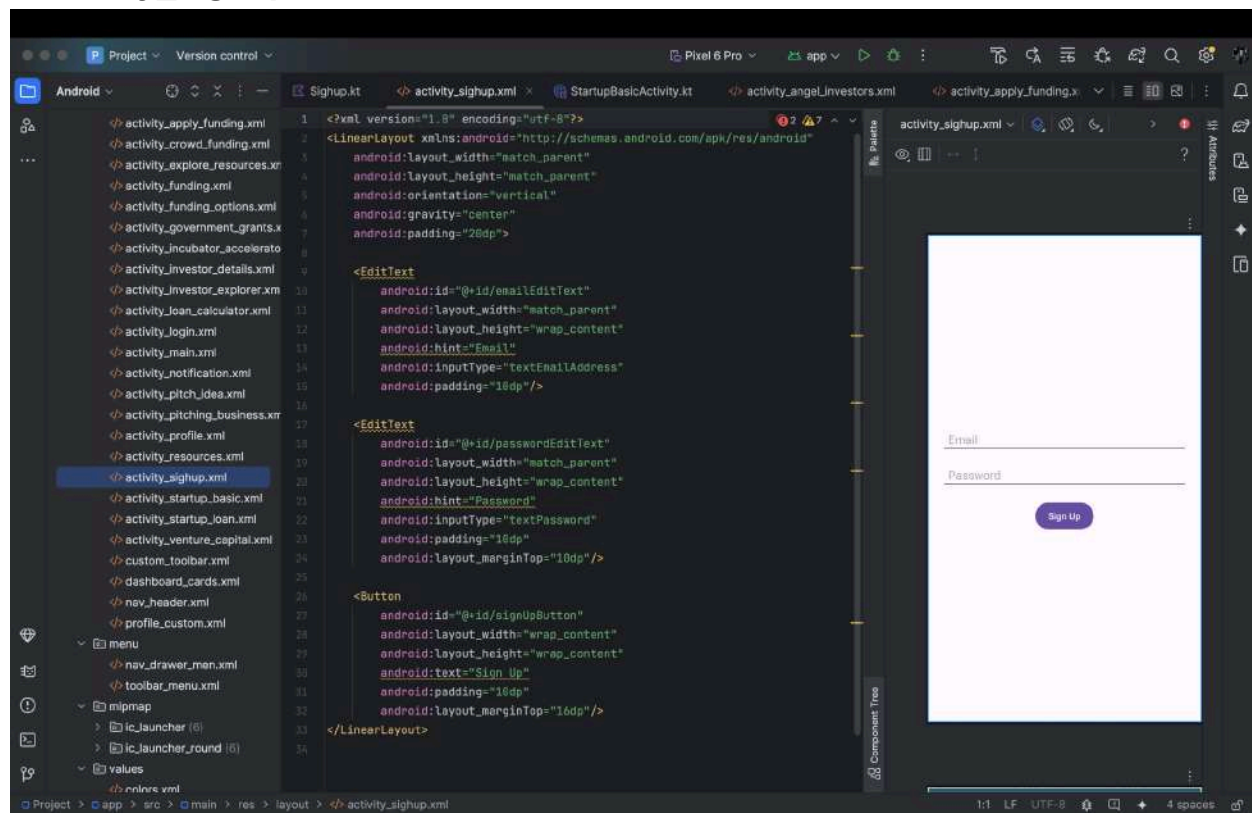
16 class LoginActivity : AppCompatActivity() {
17     override fun onCreate(savedInstanceState: Bundle?) {
18         loginButton.setOnClickListener {
19             .addOnCompleteListener { task ->
20                 Toast.LENGTH_SHORT
21                 }.show()
22             }
23         }
24     }
25     // Forget Password Click
26     forgotPasswordText.setOnClickListener {
27         val email = emailEditText.text.toString().trim()
28         if (email.isEmpty()) {
29             Toast.makeText(this, "Enter your email first", Toast.LENGTH_SHORT).show()
30             return@setOnClickListener
31         }
32         auth.sendPasswordResetEmail(email)
33         .addOnCompleteListener { task ->
34             if (task.isSuccessful) {
35                 Toast.makeText(this, "Reset link sent to your email", Toast.LENGTH_SHORT)
36                 .show()
37             } else {
38                 Toast.makeText(
39                     this,
40                     "Failed: ${task.exception?.message}",
41                     Toast.LENGTH_SHORT
42                 ).show()
43             }
44         }
45     }
46 }
47
48 override fun onActivityResult(requestCode: Int, resultCode: Int, data: Intent?) {
49     super.onActivityResult(requestCode, resultCode, data)
50 }
```

```
Project | Version control | Pixel 6 Pro | app | Login.kt

Android | app | manifests | AndroidManifest.xml | kotlin+java | com.example.project | Login.kt | MainActivity | Notification | PitchIdeaActivity | PitchingBusinessActivity | profile | ResourcesActivity | Signup.kt | StartupBasicActivity | StartupLoanActivity | VentureCapitalActivity | com.example.project (androidTest) | com.example.project (test) | res | drawable | background.xml | raw | h1.vml

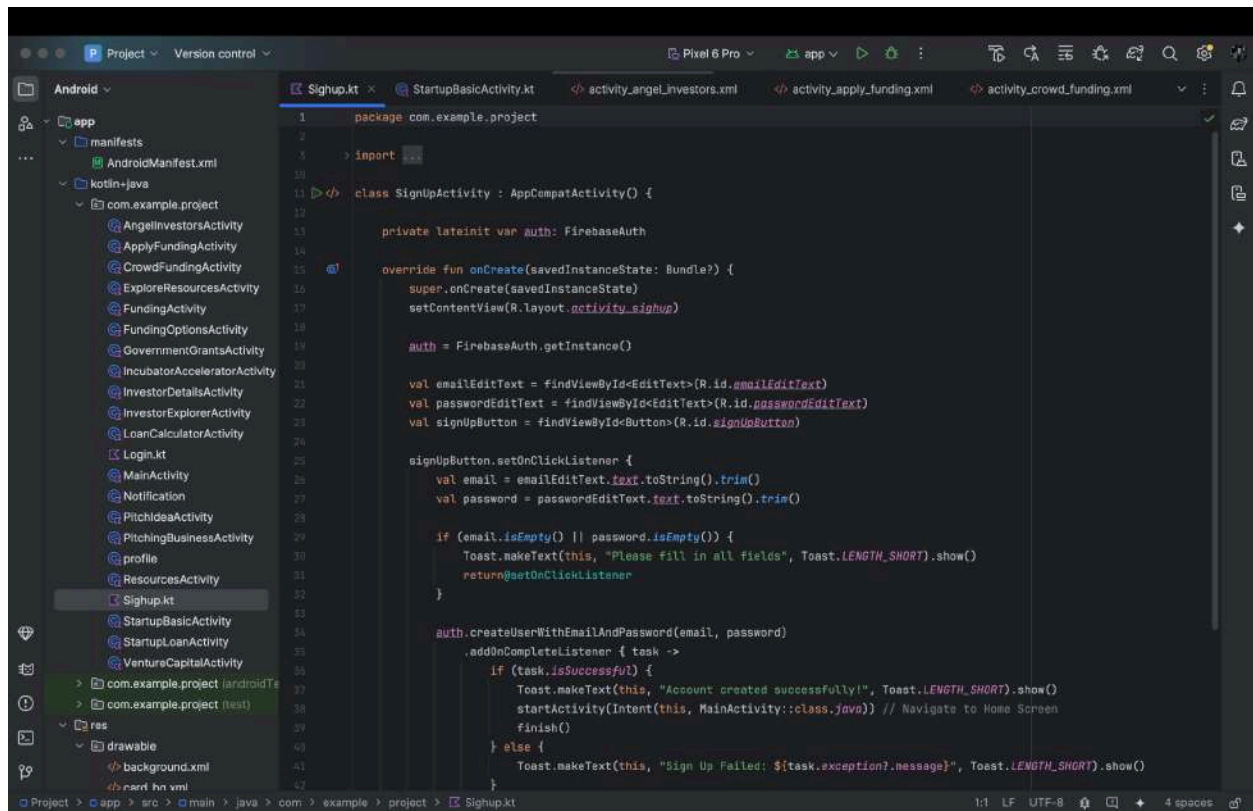
112 override fun onActivityResult(requestCode: Int, resultCode: Int, data: Intent?) {
113     super.onActivityResult(requestCode, resultCode, data)
114 }
115
116 if (requestCode == RC_SIGN_IN) {
117     val task = GoogleSignIn.getSignedInAccountFromIntent(data)
118     try {
119         val account = task.getResult(ApiException::class.java)
120         firebaseAuthWithGoogle(account.idToken!!)
121     } catch (e: ApiException) {
122         Toast.makeText(this, "Google Sign-In Failed", Toast.LENGTH_SHORT).show()
123     }
124 }
125
126 // Firebase Authentication with Google
127 private fun firebaseAuthWithGoogle(idToken: String) {
128     val credential = GoogleAuthProvider.getCredential(idToken, null)
129     auth.signInWithCredential(credential)
130     .addOnCompleteListener(this) { task ->
131         if (task.isSuccessful) {
132             Toast.makeText(this, "Google Sign-In Successful", Toast.LENGTH_SHORT).show()
133             startActivity(Intent(this, MainActivity::class.java)) // Navigate to Home Screen
134             finish()
135         } else {
136             Toast.makeText(this, "Google Sign-In Failed", Toast.LENGTH_SHORT).show()
137         }
138     }
139 }
140
141 companion object {
142     private const val RC_SIGN_IN = 9001
143 }
144
145 }
```


->activity_sighup.xml



1. Parent Layout: `LinearLayout` with vertical orientation, centered content, and padding of 20dp for spacing.
2. Email Input: An `EditText` for email input with a hint ("Email") and set to accept email addresses.
3. Password Input: An `EditText` for password input with a hint ("Password"), and it masks the input text.
4. Sign-Up Button: A `Button` to trigger the sign-up action, with a text of "Sign Up" and padding for spacing.

->Sighup.kt

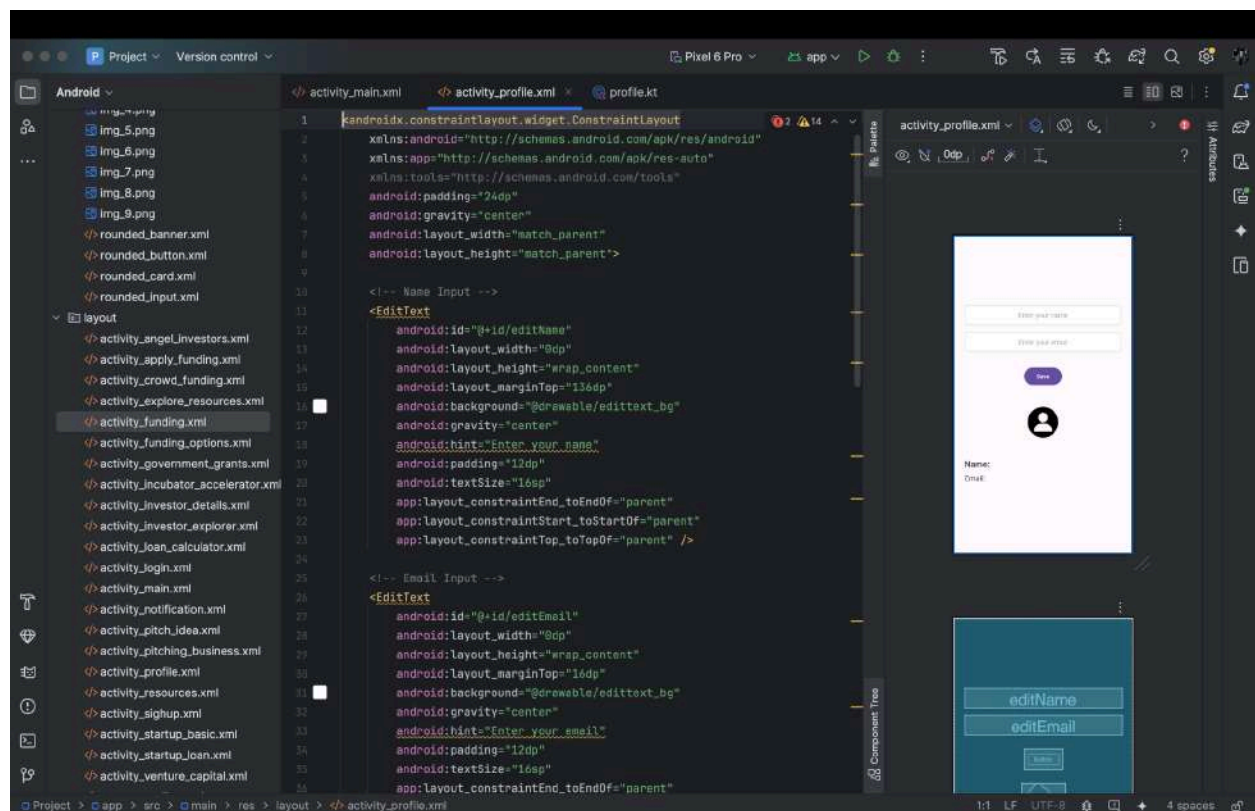


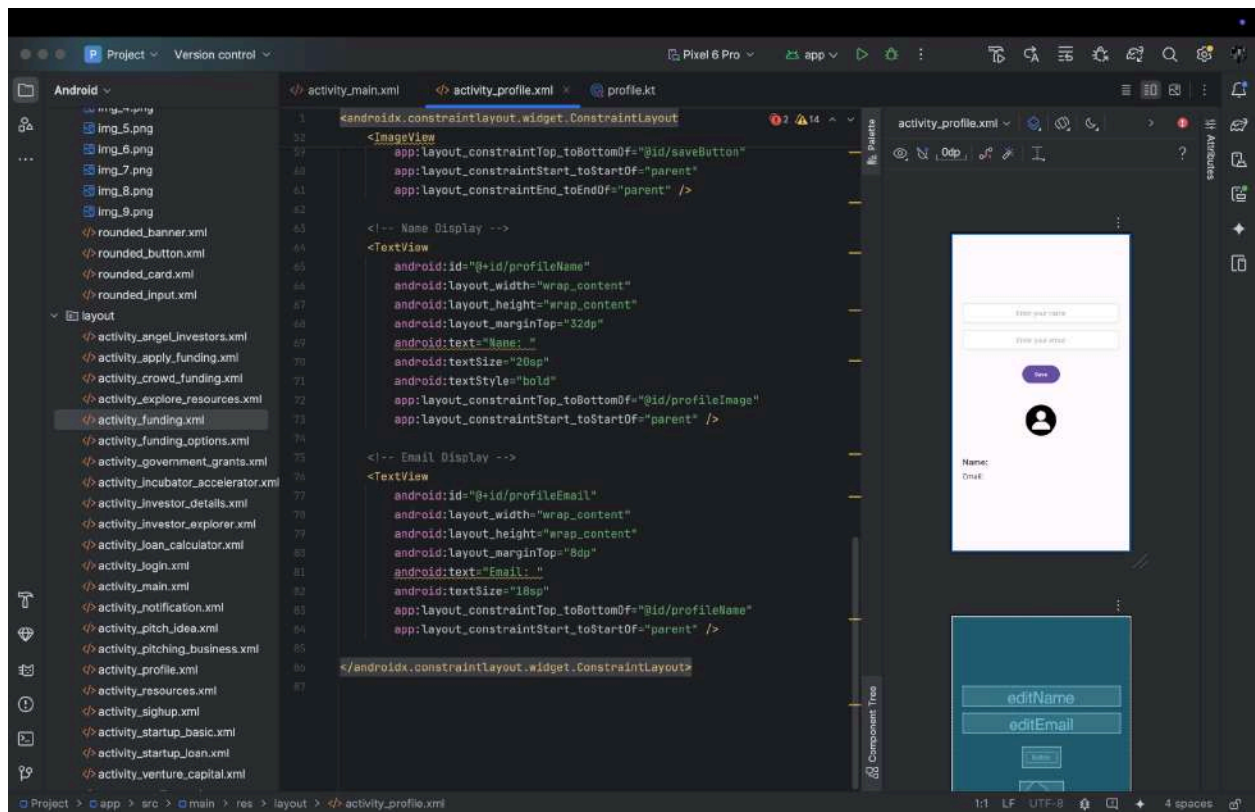
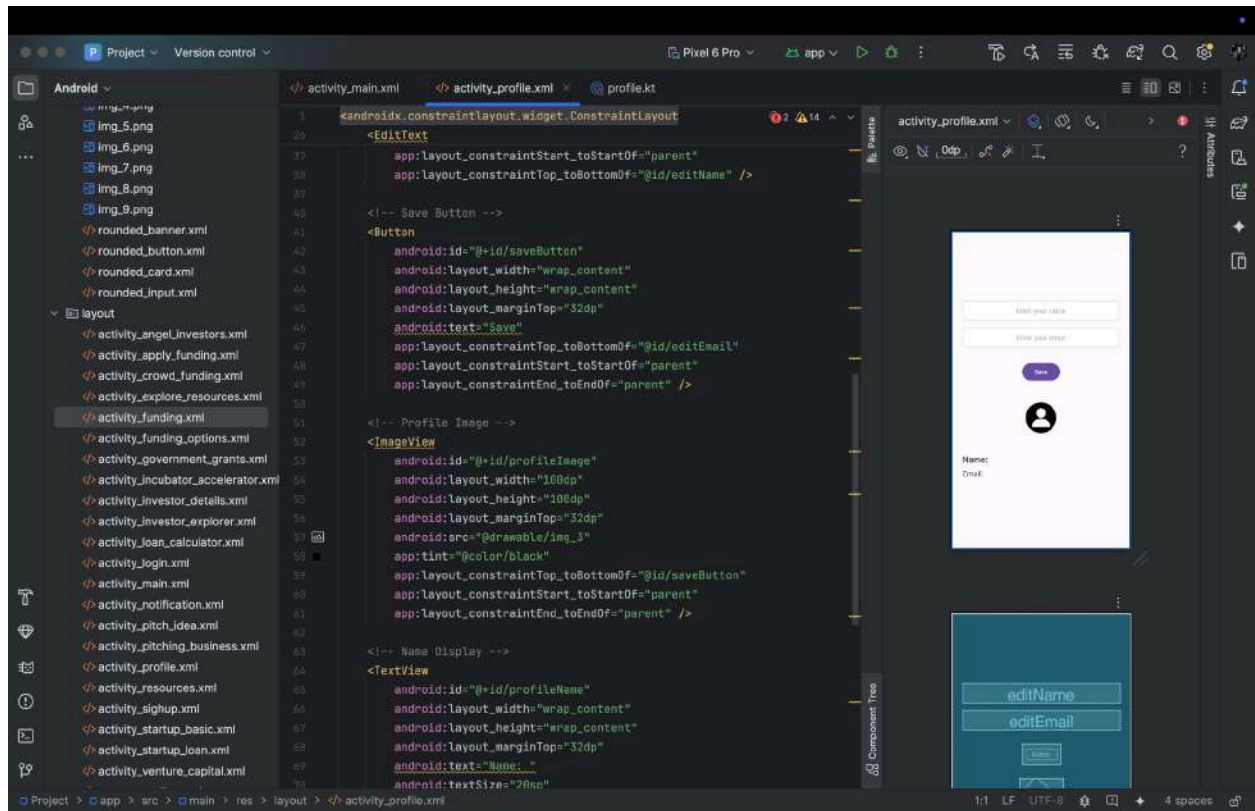
1. Firebase Authentication: Initializes Firebase Auth (`auth = FirebaseAuth.getInstance()`).
2. Views: Retrieves `EditText` (for email/password) and the `Button` (Sign Up).
3. Sign-Up Process: On button click, it checks if email and password are filled. Then, it creates a new user with Firebase (`auth.createUserWithEmailAndPassword`).
4. On Success: If sign-up is successful, it navigates to `MainActivity` and displays a success toast.
5. On Failure: If sign-up fails, it shows a toast with an error message.

-> Now Toolbar Features :

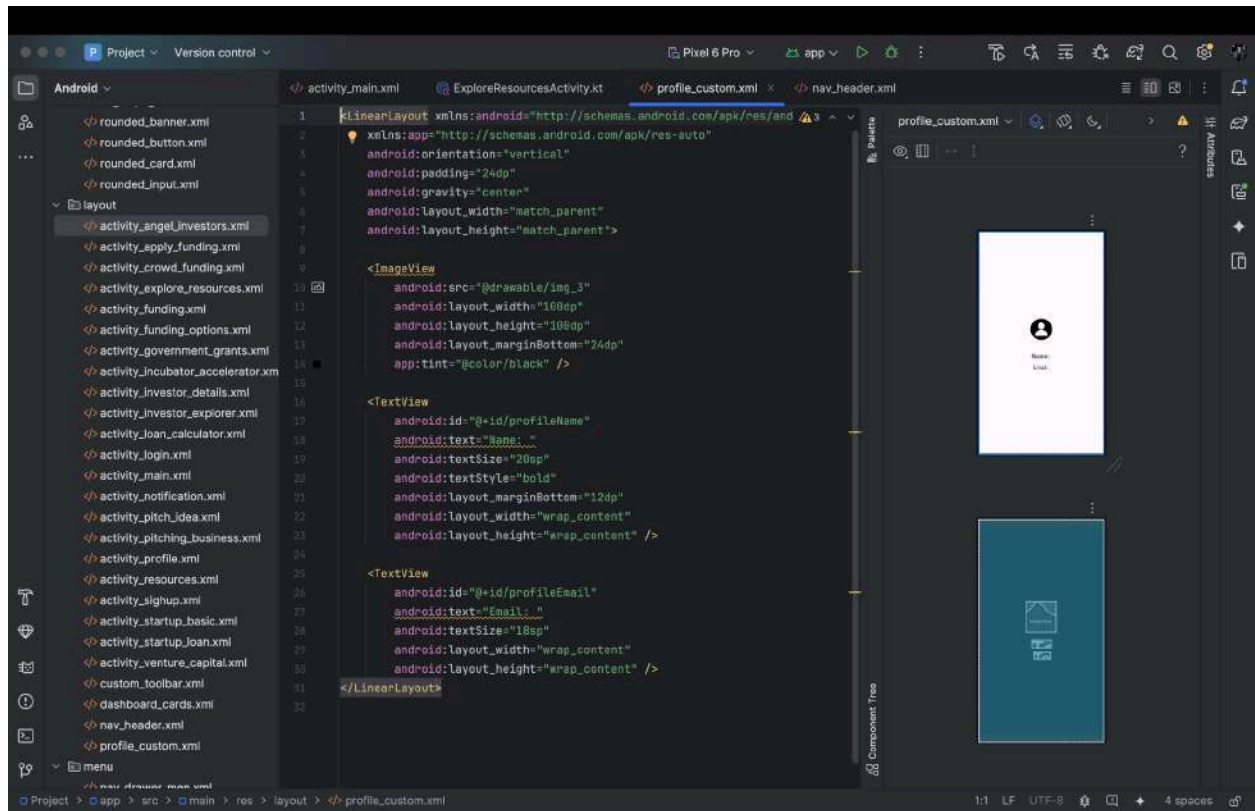
->activity_profile

1. Parent Layout: **ConstraintLayout** — gives flexible positioning.
2. Name Input (**EditText**): Lets the user enter their name.
3. Email Input (**EditText**): Lets the user enter their email.
4. Save Button (**Button**): User clicks it to save the entered data.
5. Profile Image (**ImageView**): Displays a profile image (**img_3** from drawable).
6. Name Display (**TextView**): Shows "Name: " (you can later update this with the entered name).
7. Email Display (**TextView**): Shows "Email: " (you can update it too)

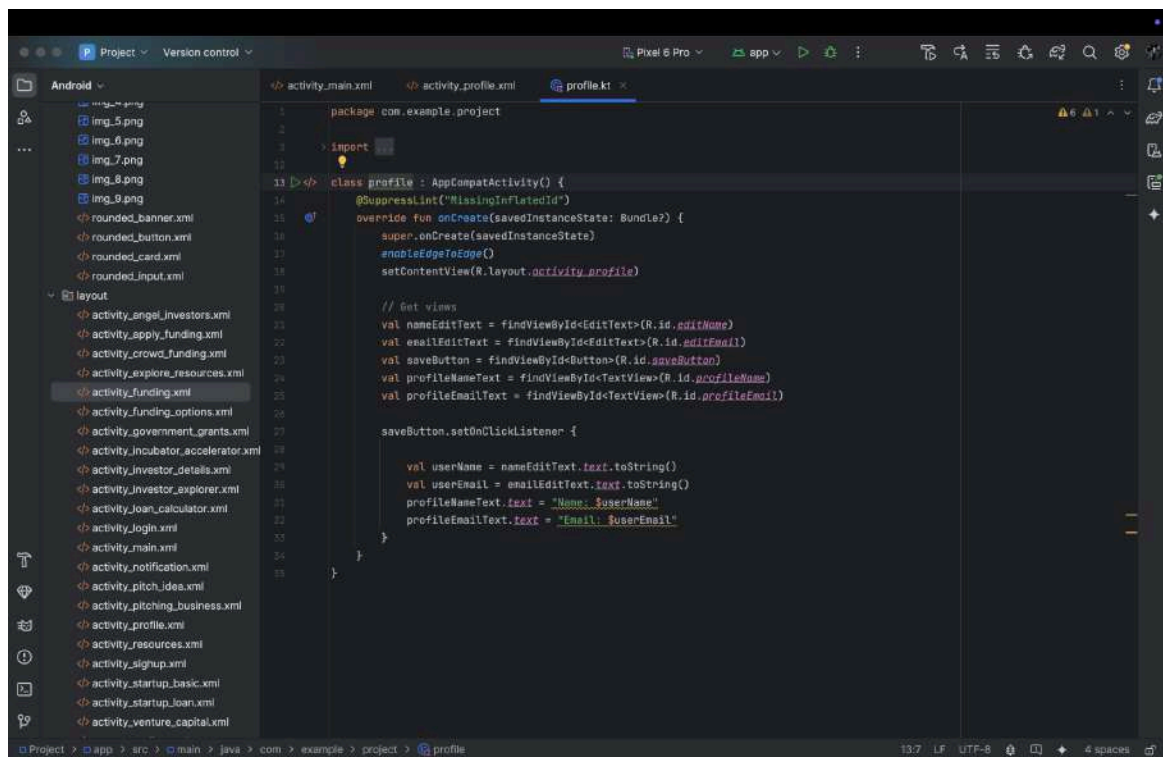




->profile_custom.xml



-> profile.kt



- It gets the input fields (name, email), Save button, and TextViews (for displaying name and email).
- When the user clicks the Save button:

It reads the text from the `EditText` fields.

It updates the `TextView` below with the entered Name and Email.

-> activity_notification.xml

ScrollView → So that if many notifications come, the user can scroll.

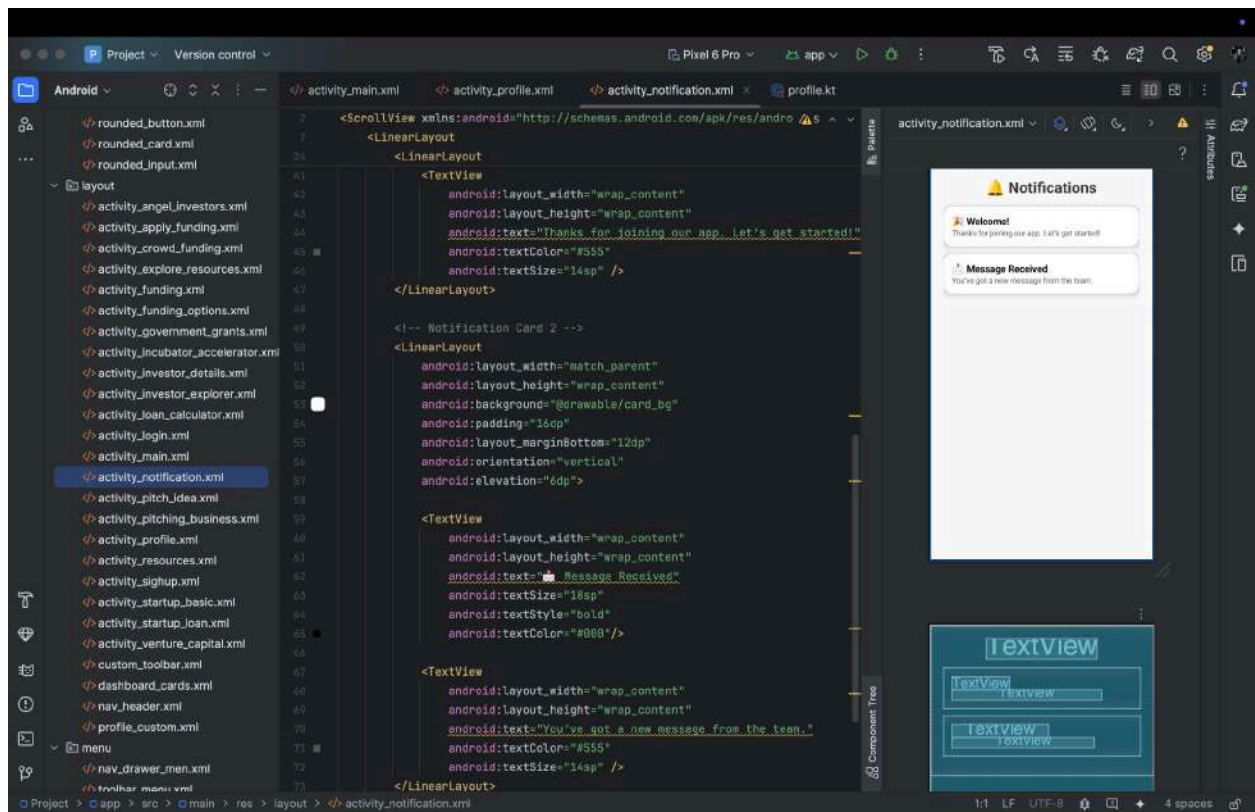
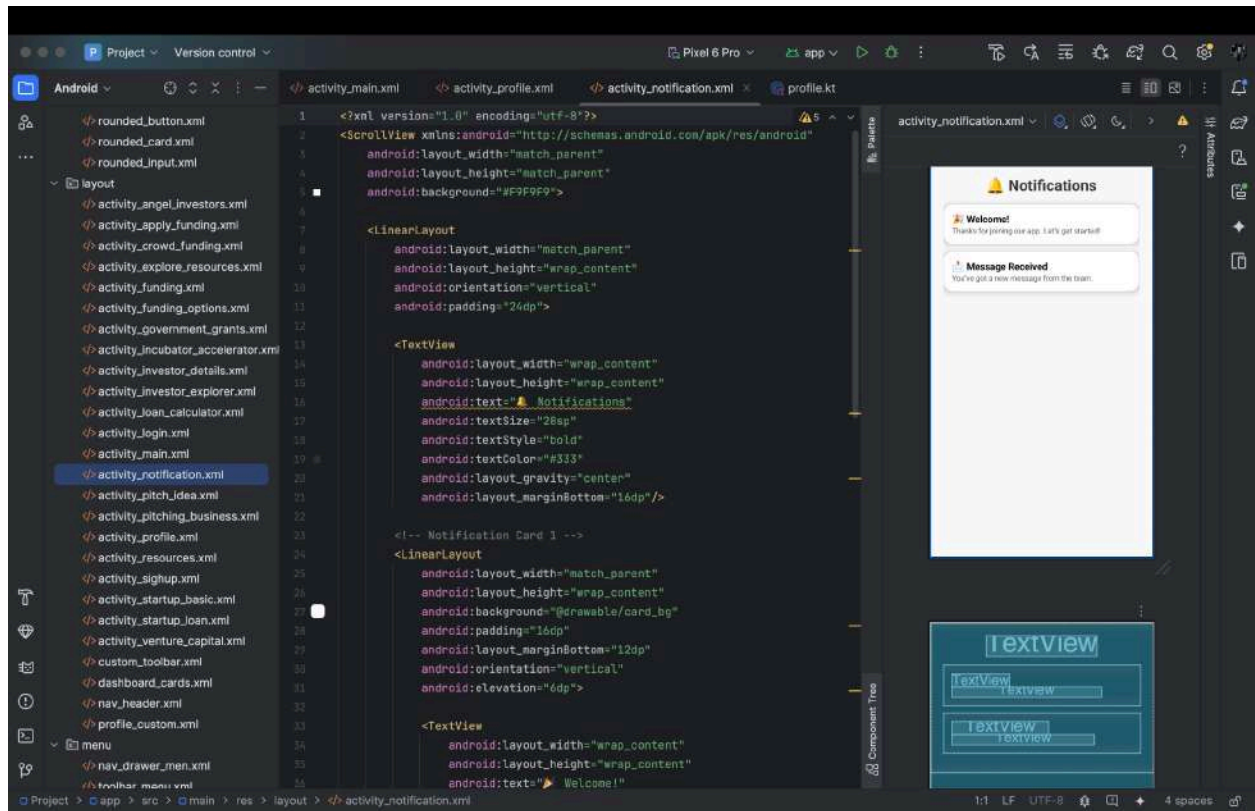
Inside it, you have a LinearLayout (vertical) to stack notifications one below another.

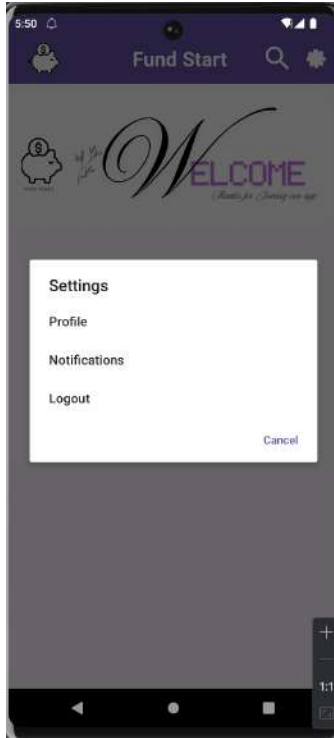
Title TextView → "Notifications" big heading at top (28sp, bold).

Notification Cards → Each notification is a mini card (with background from `@drawable/card_bg`, padding, margin, and elevation for a raised look).

Each card has:

- A bold title (Welcome!, Message Received)
- A small description below.
-





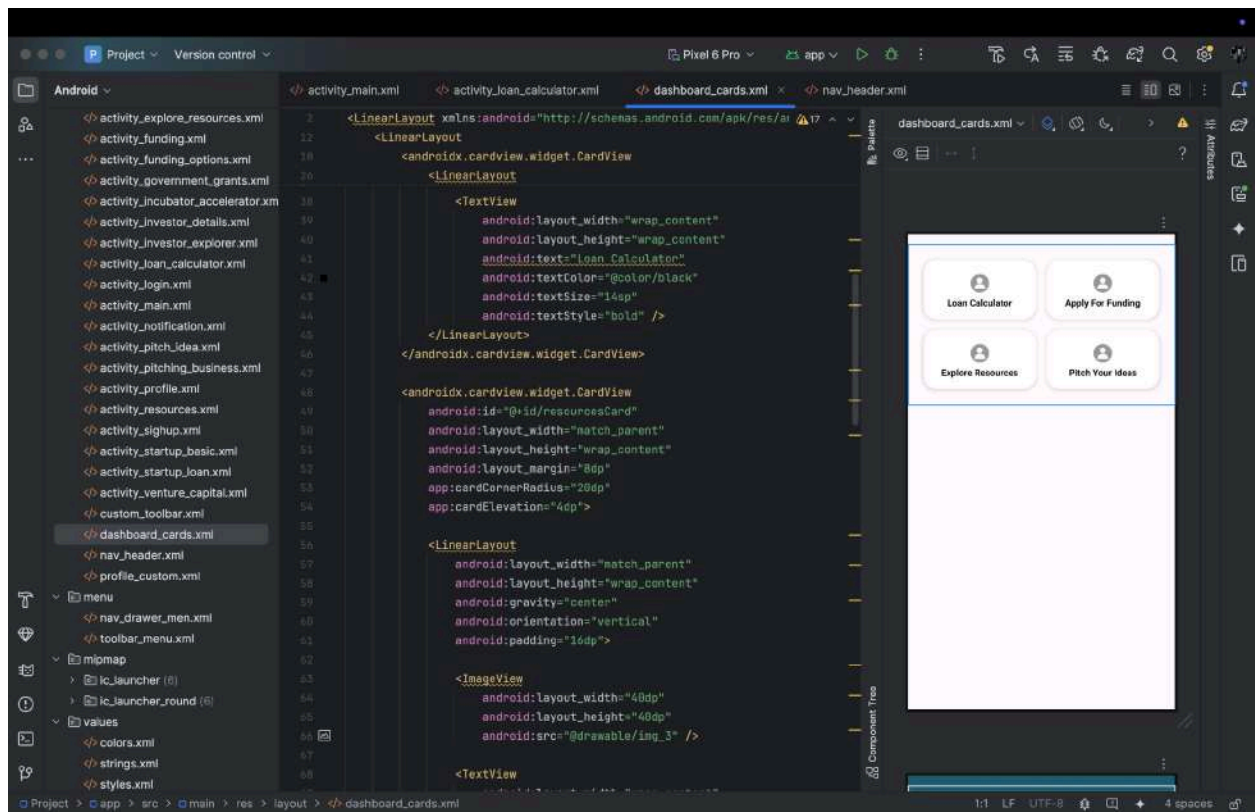
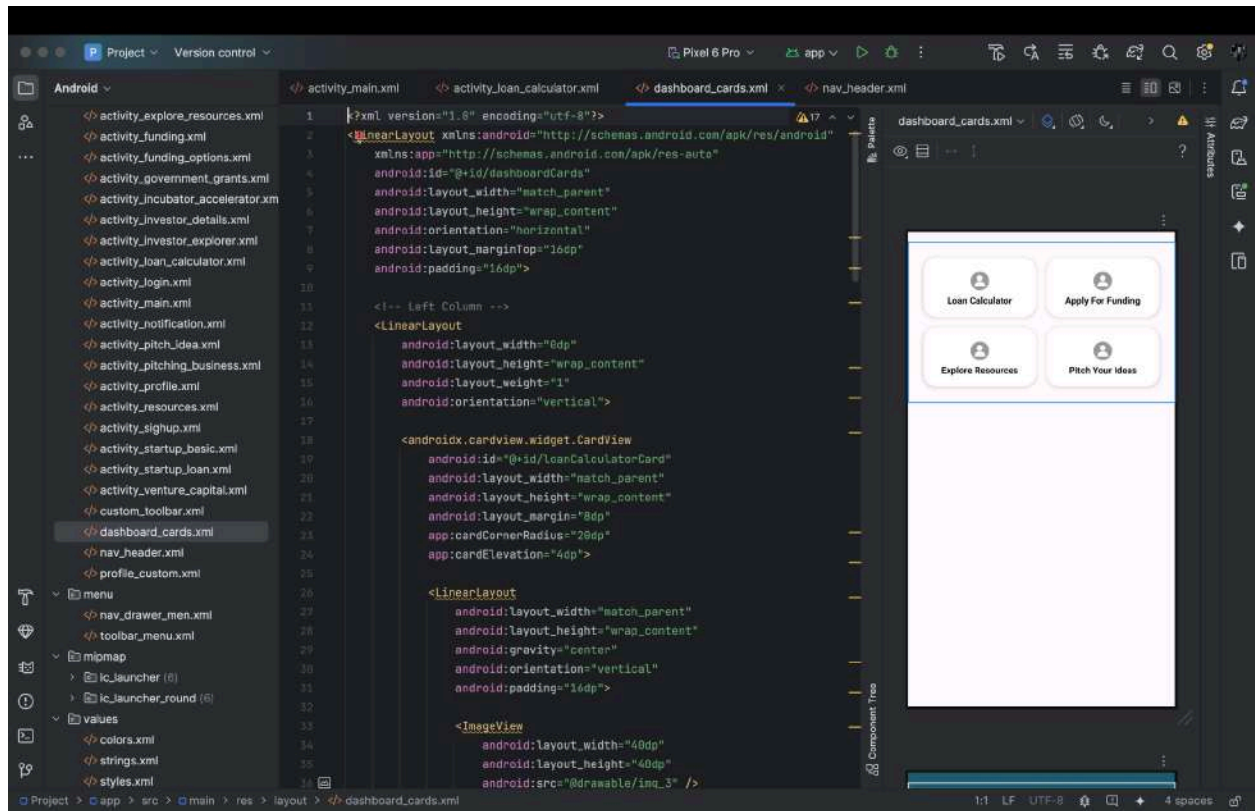
So here we can see three options in action box which are

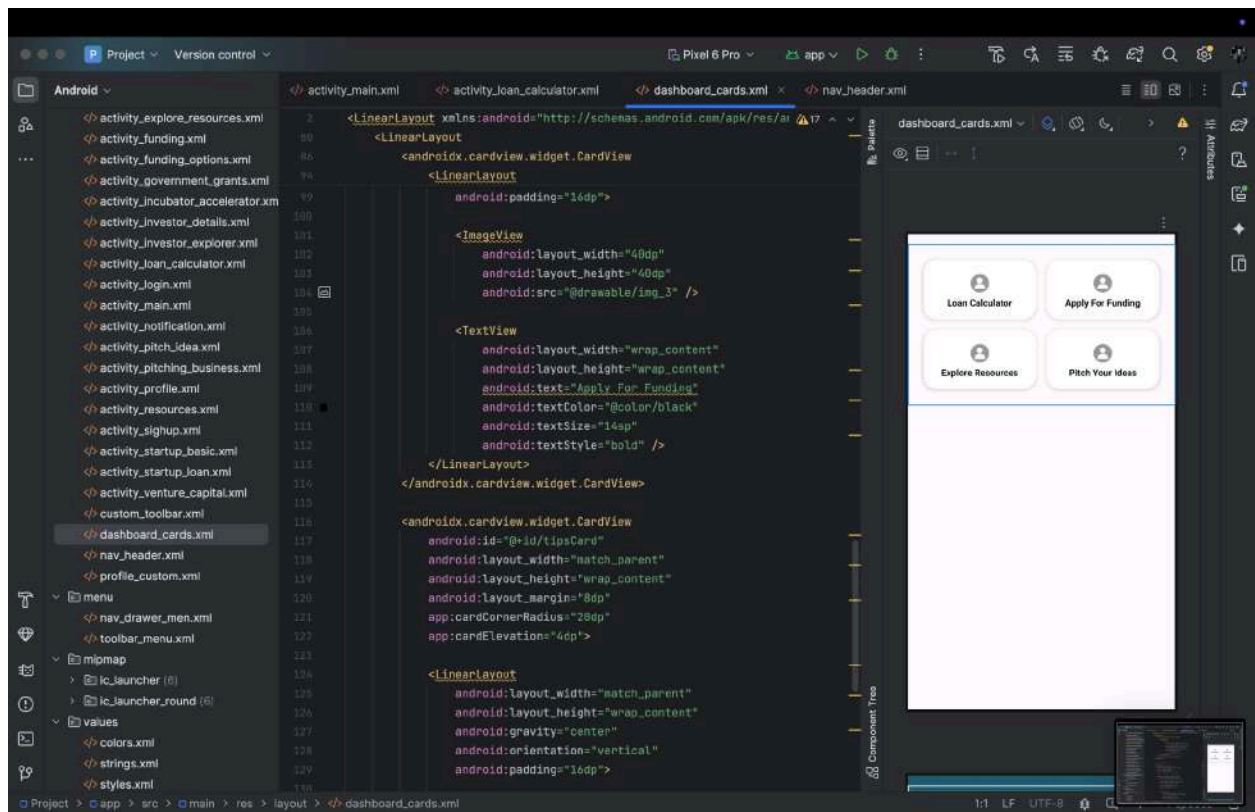
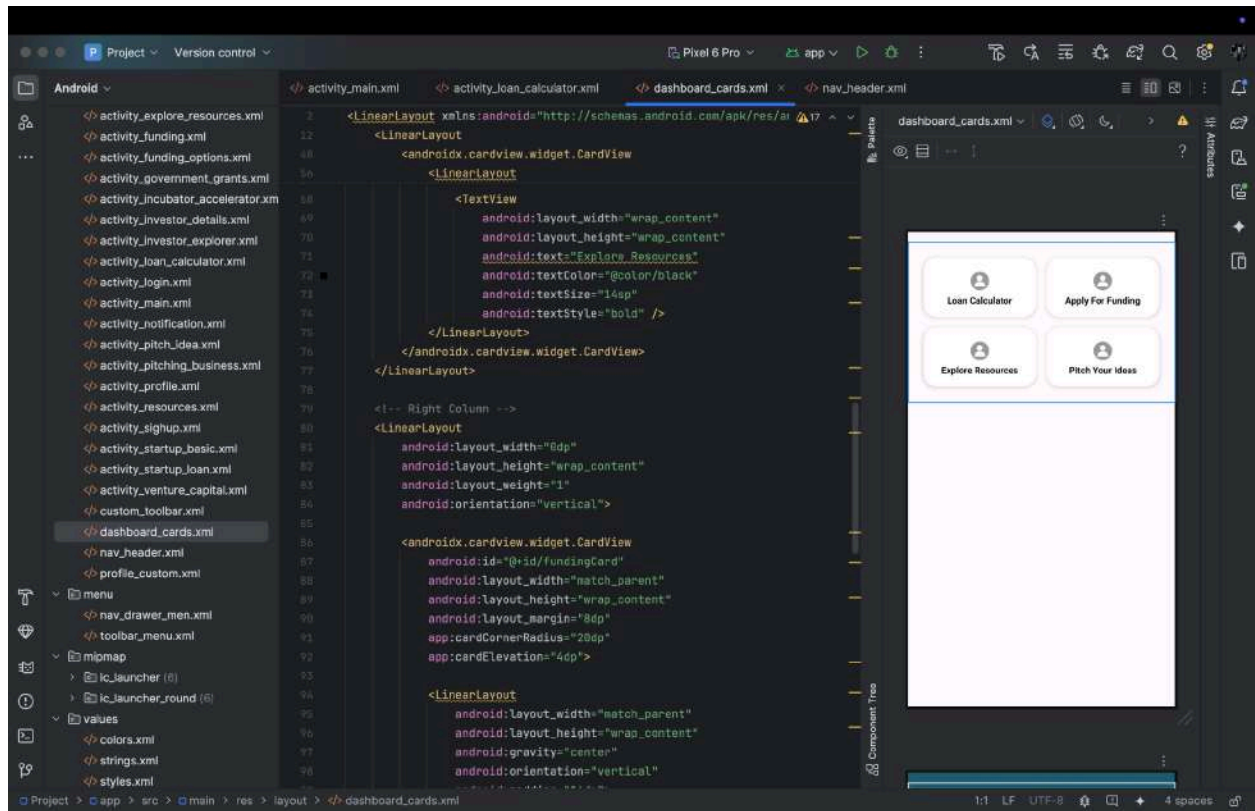
1. Profile
2. Notifications
3. Logout
-> logout takes you back to the login page.

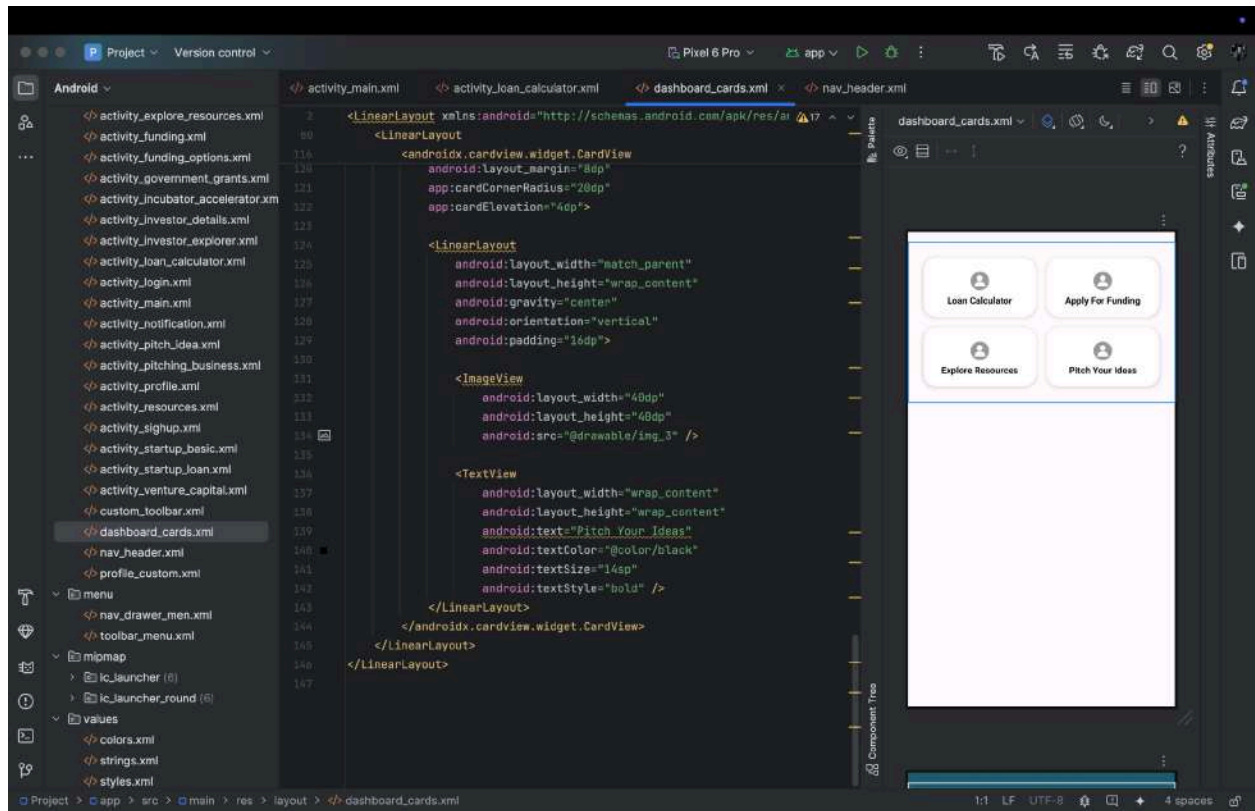
***Cancel makes the action box disappear**

-> **dashboard_cards.xml**

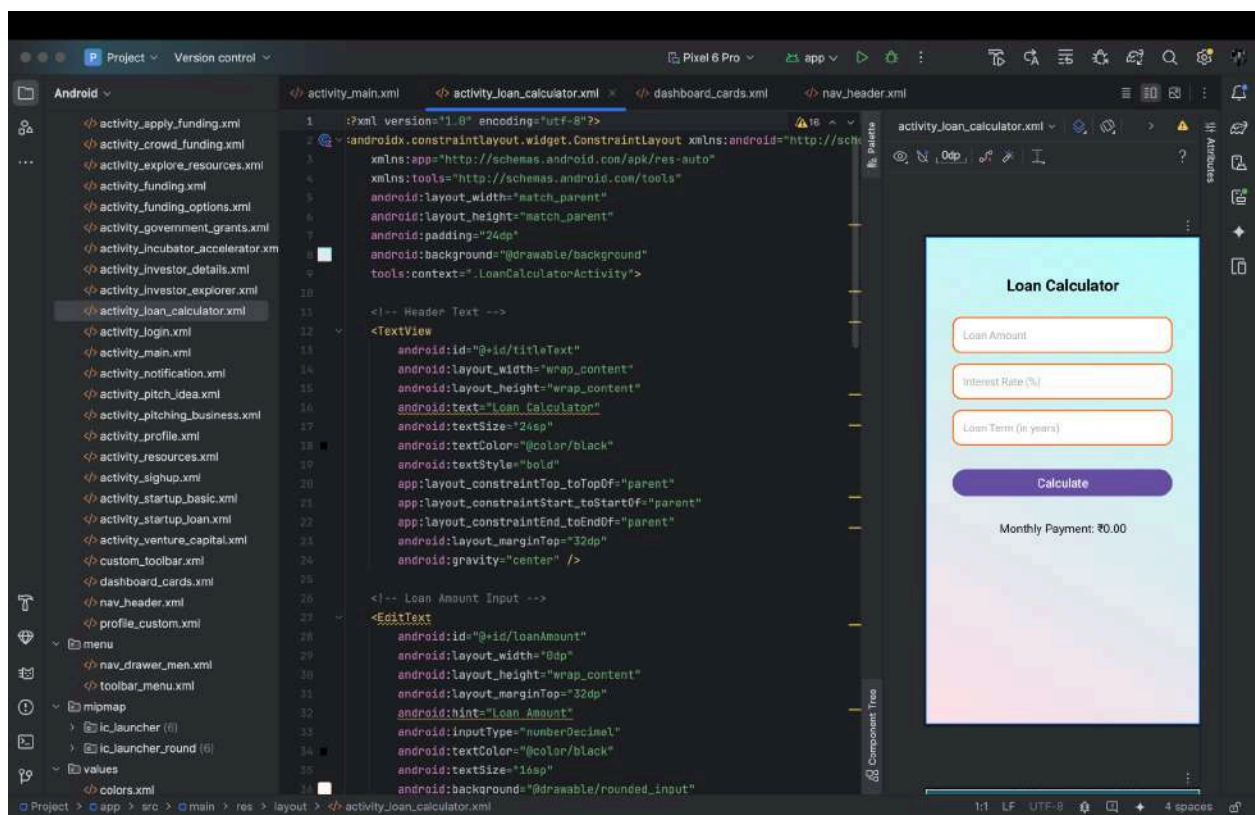
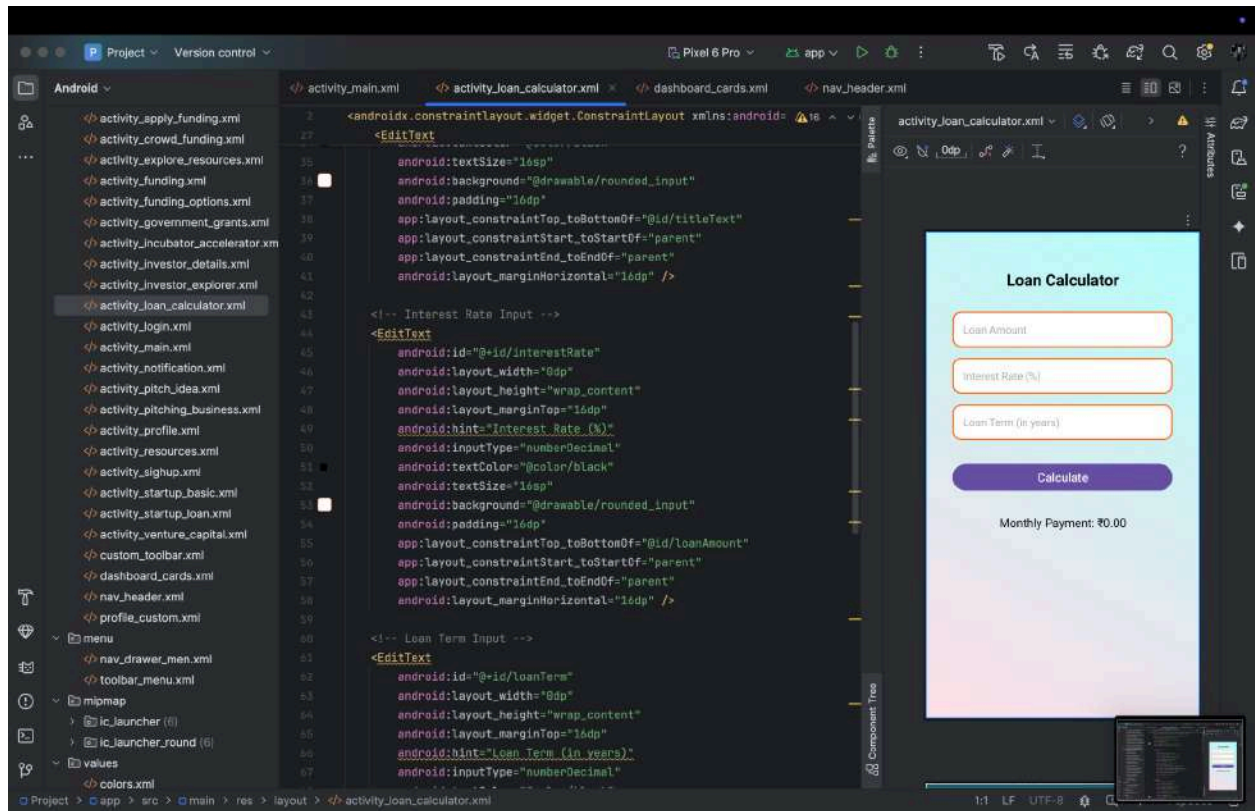
- The root is a Horizontal LinearLayout → Splits screen into two columns.
- Each column is a Vertical LinearLayout → Contains 2 CardViews stacked vertically.
- Each CardView includes:
 - An ImageView → Icon (using [img_3](#)).
 - A TextView → Title (like "Loan Calculator", "Apply For Funding", etc).

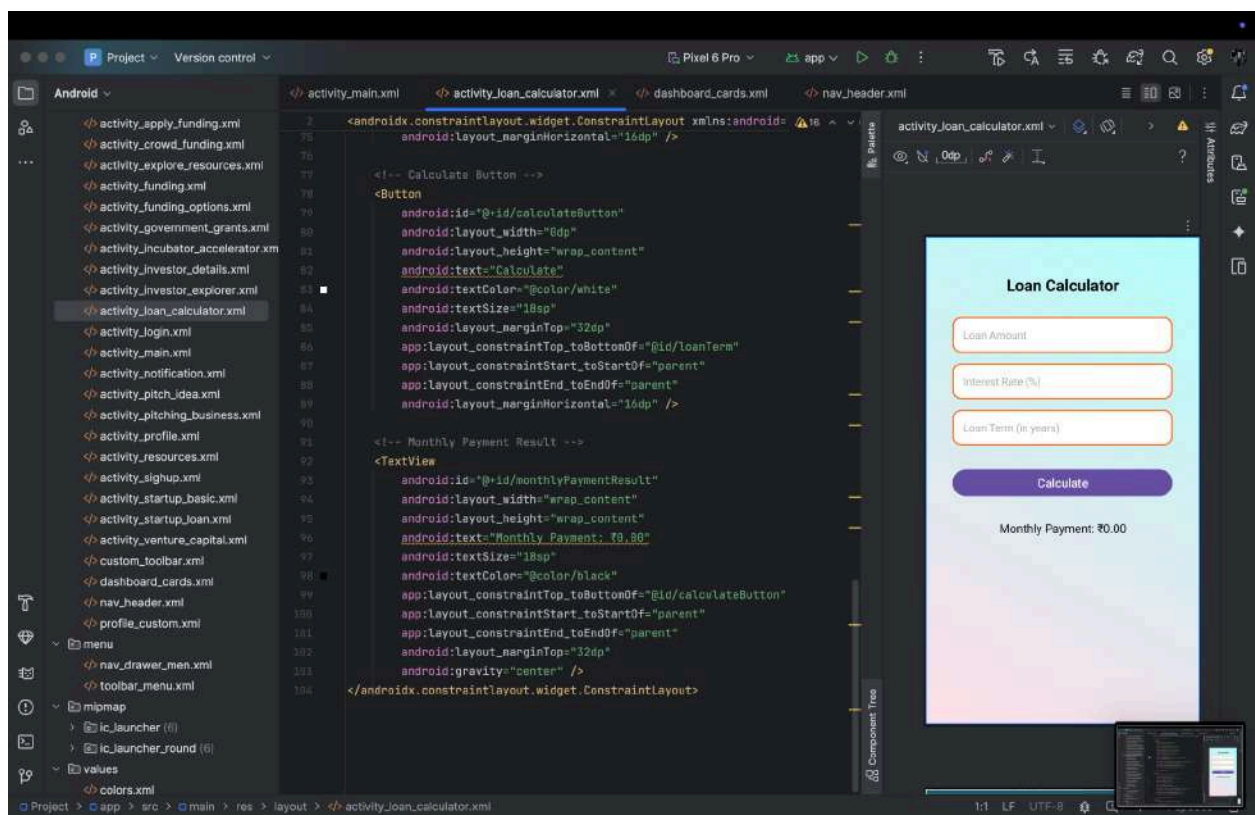
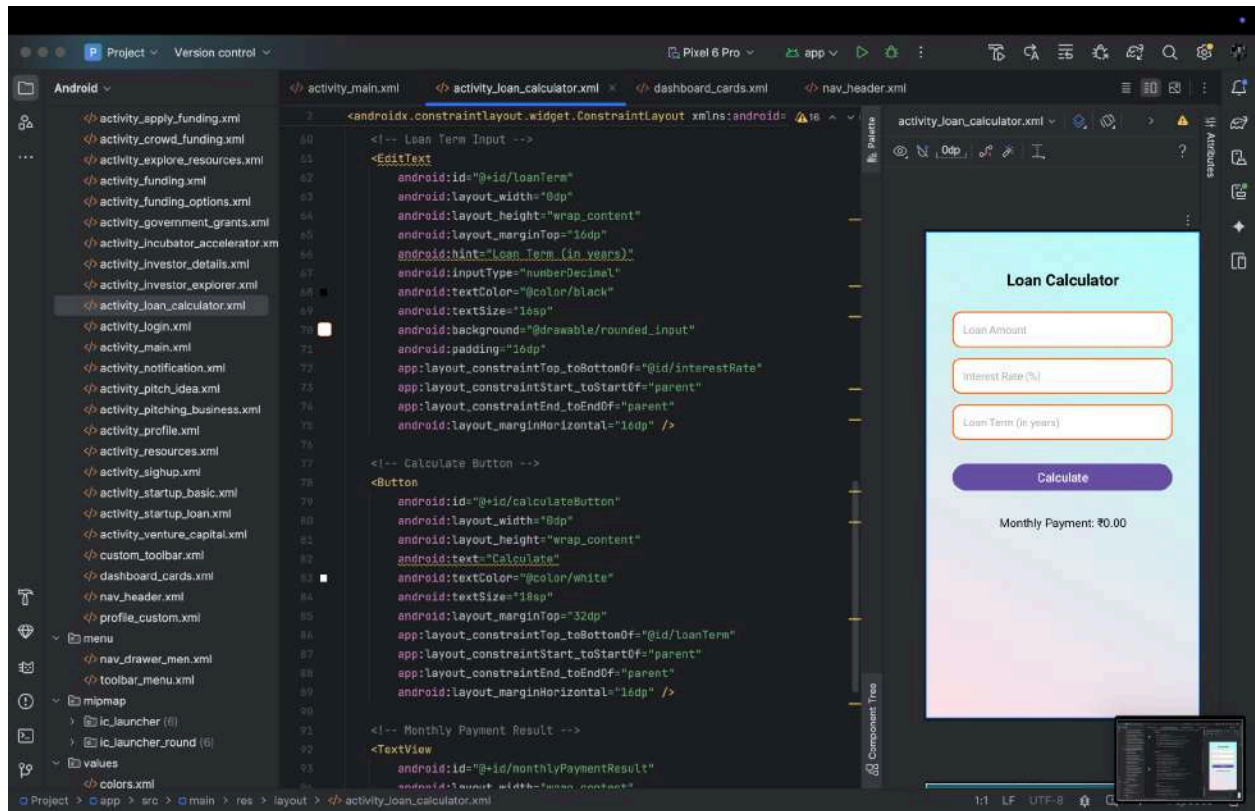






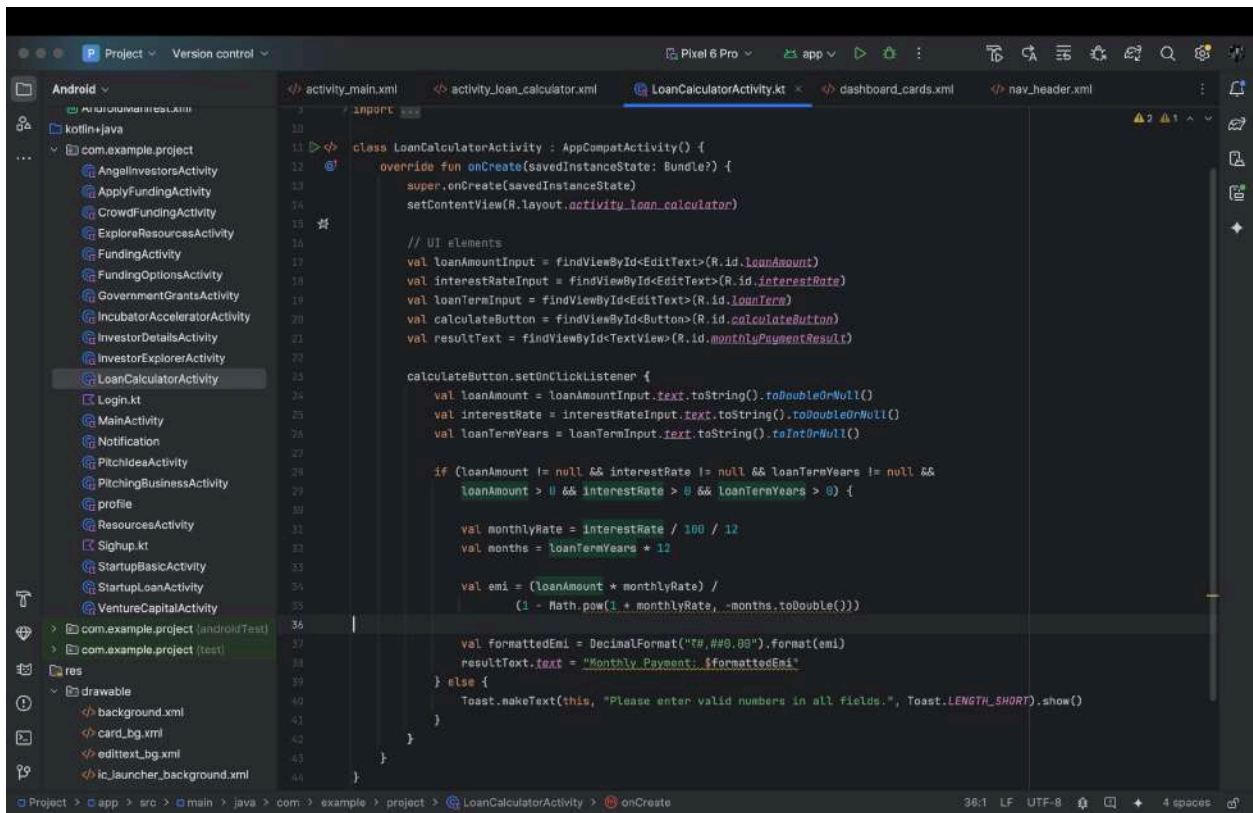
-> activity_loan_calculator.xml





1. Header Text: The "Loan Calculator" header is centered and bold, making it prominent.
2. Input Fields: **Loan Amount**, **Interest Rate**, and **Loan Term** fields have clear labels, rounded design, and appropriate padding.
3. Calculate Button: The button is easily visible and placed below the inputs, with white text on a contrasting background.
4. Result Display: The **Monthly Payment** text view shows the result clearly below the button with ample margin.

-> LoanCaluculatorActivity.kt



```
1 // import androidx.appcompat.app.AppCompatActivity
2
3 class LoanCalculatorActivity : AppCompatActivity() {
4     override fun onCreate(savedInstanceState: Bundle?) {
5         super.onCreate(savedInstanceState)
6         setContentView(R.layout.activity_loan_calculator)
7
8         // UI elements
9         val loanAmountInput = findViewById<EditText>(R.id.loanAmount)
10        val interestRateInput = findViewById<EditText>(R.id.interestRate)
11        val loanTermInput = findViewById<EditText>(R.id.loanTerm)
12        val calculateButton = findViewById<Button>(R.id.calculateButton)
13        val resultText = findViewById<TextView>(R.id.monthlyPaymentResult)
14
15        calculateButton.setOnClickListener {
16            val loanAmount = loanAmountInput.text.toString().toDoubleOrNull()
17            val interestRate = interestRateInput.text.toString().toDoubleOrNull()
18            val loanTermYears = loanTermInput.text.toString().toIntOrNull()
19
20            if (loanAmount != null && interestRate != null && loanTermYears != null &&
21                loanAmount > 0 && interestRate > 0 && loanTermYears > 0) {
22
23                val monthlyRate = interestRate / 100 / 12
24                val months = loanTermYears * 12
25
26                val emi = (loanAmount * monthlyRate) /
27                    (1 - Math.pow(1 + monthlyRate, -months.toDouble()))
28
29                val formattedEmi = DecimalFormat("#,##0.00").format(emi)
30                resultText.text = "Monthly Payment: $formattedEmi"
31            } else {
32                Toast.makeText(this, "Please enter valid numbers in all fields.", Toast.LENGTH_SHORT).show()
33            }
34        }
35    }
36}
```

UI Elements:

- EditText Fields: For loan amount, interest rate, and loan term.
- Button: A button to calculate the monthly EMI.

- TextView: To display the monthly payment result.

Calculation Logic:

- Retrieves values from the input fields and converts them to the appropriate data types.
- Performs validation to ensure all inputs are valid numbers greater than zero.
- Uses the standard formula for calculating EMI (Equated Monthly Installment):

$$EMI = \frac{P \times r \times (1 + r)^n}{(1 + r)^n - 1}$$
 where:
 - PPP = Loan Amount
 - rrr = Monthly Interest Rate
 - nnn = Number of Months

Formatted Result:

- The monthly EMI is calculated and formatted to Indian Rupees (₹) using `DecimalFormat` for proper display (e.g., ₹10,000.00).

Error Handling:

- Displays a `Toast` message if any input is invalid or missing.

-> **activity_apply_funding.xml**

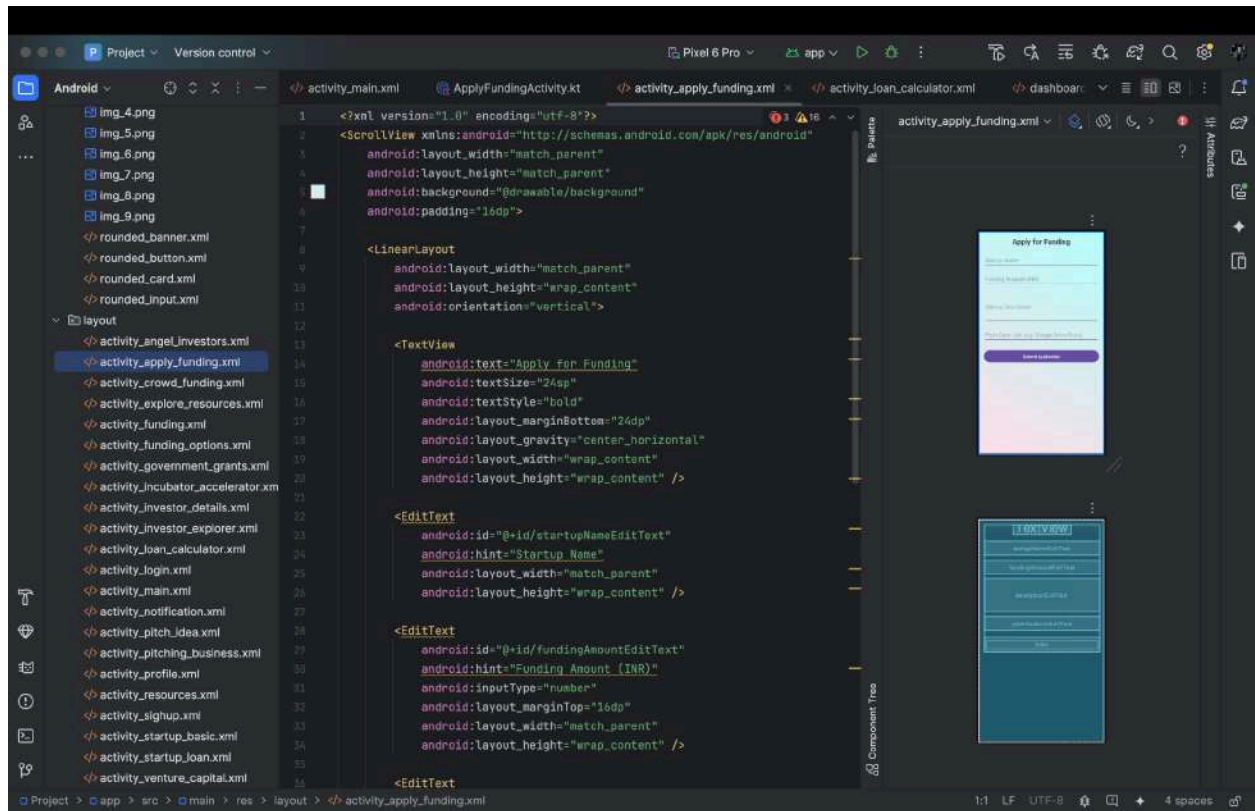
ScrollView: Makes the layout scrollable.

LinearLayout: Arranges elements vertically.

TextView: Displays "Apply for Funding" title.

EditText: Fields for Startup Name, Funding Amount, Description, and Pitch Deck Link.

Button: "Submit Application" button to submit the form.



-> ApplyFundingActivity.kt

Firestore Setup: Initializes Firestore and Firebase Authentication for data storage and user identification.

Views:

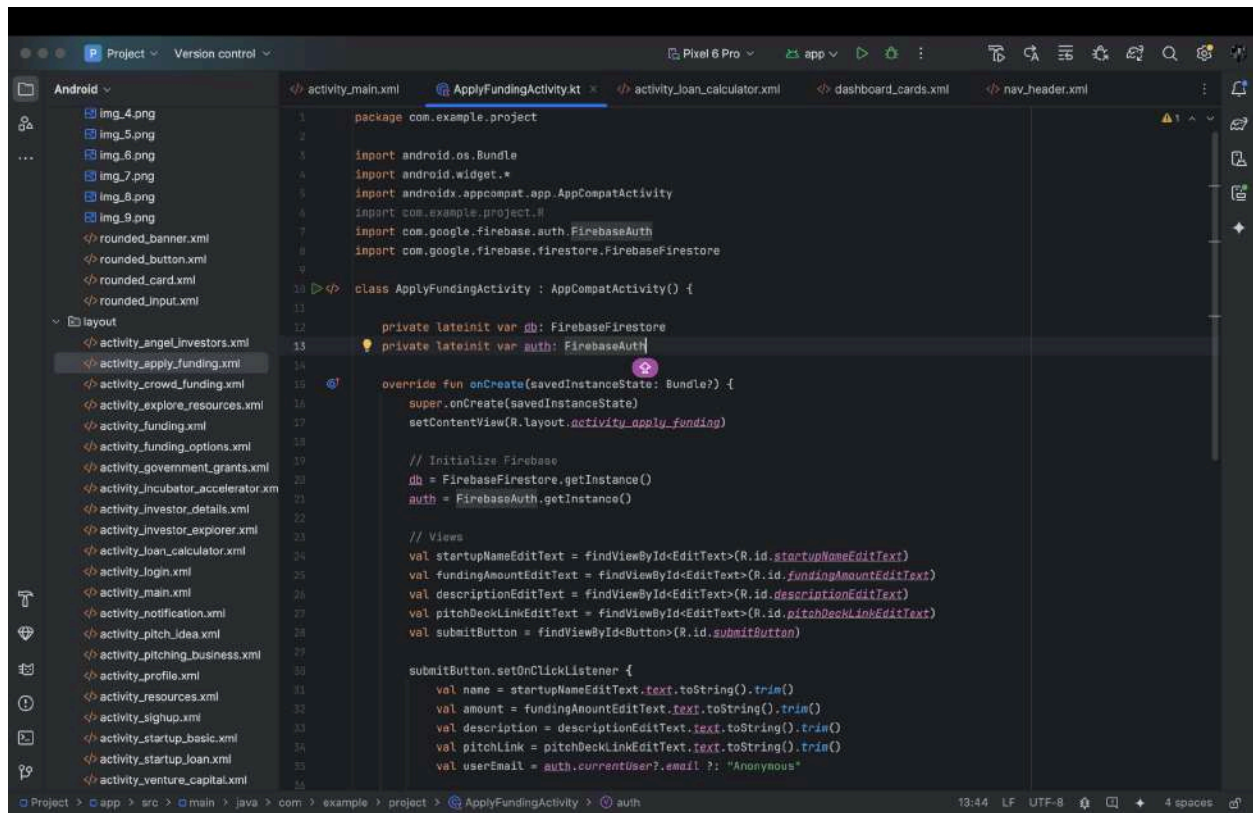
- EditText fields for Startup Name, Funding Amount, Description, and Pitch Deck Link.
- Button to submit the application.

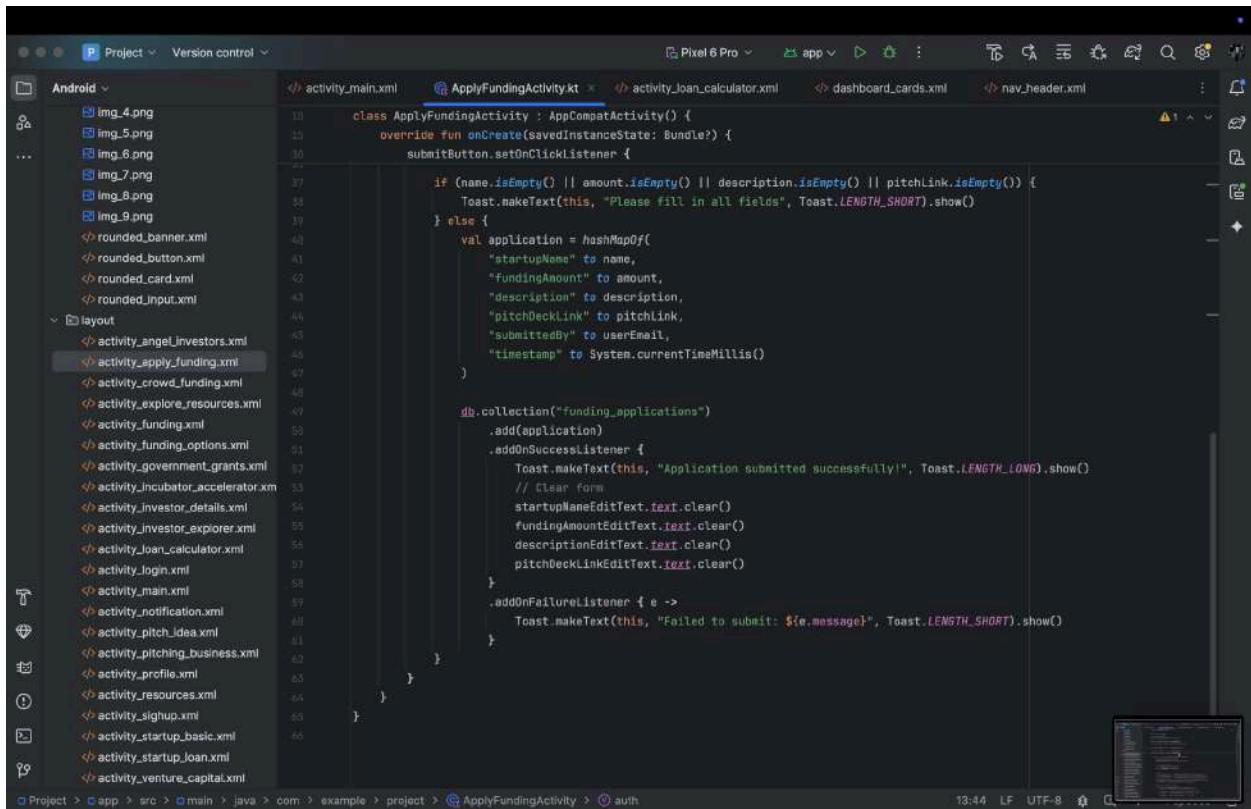
Logic:

- When the user clicks the submit button:
 - Fields are validated (ensures no empty fields).
 - If valid, data is submitted to Firestore under the `funding_applications` collection with a timestamp and user email.
 - On success, a success message is shown, and the form is cleared.

- On failure, an error message is displayed.

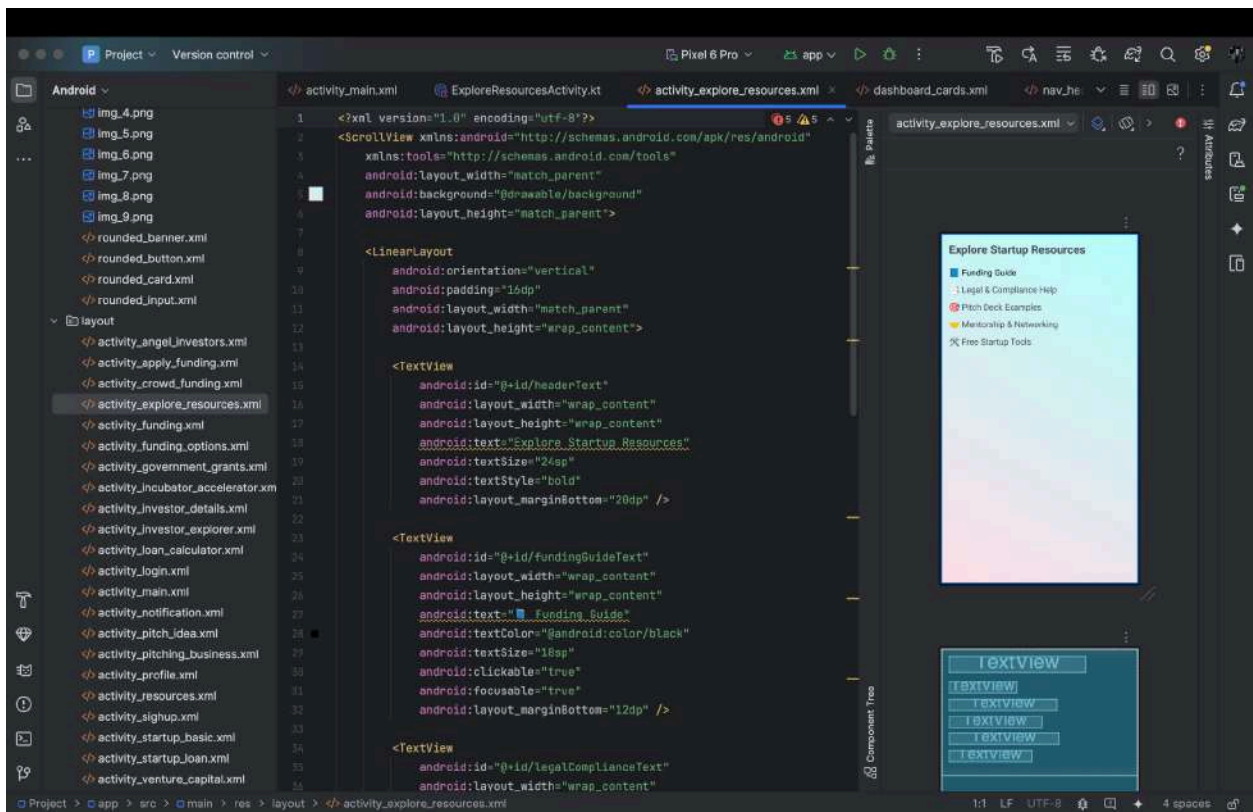
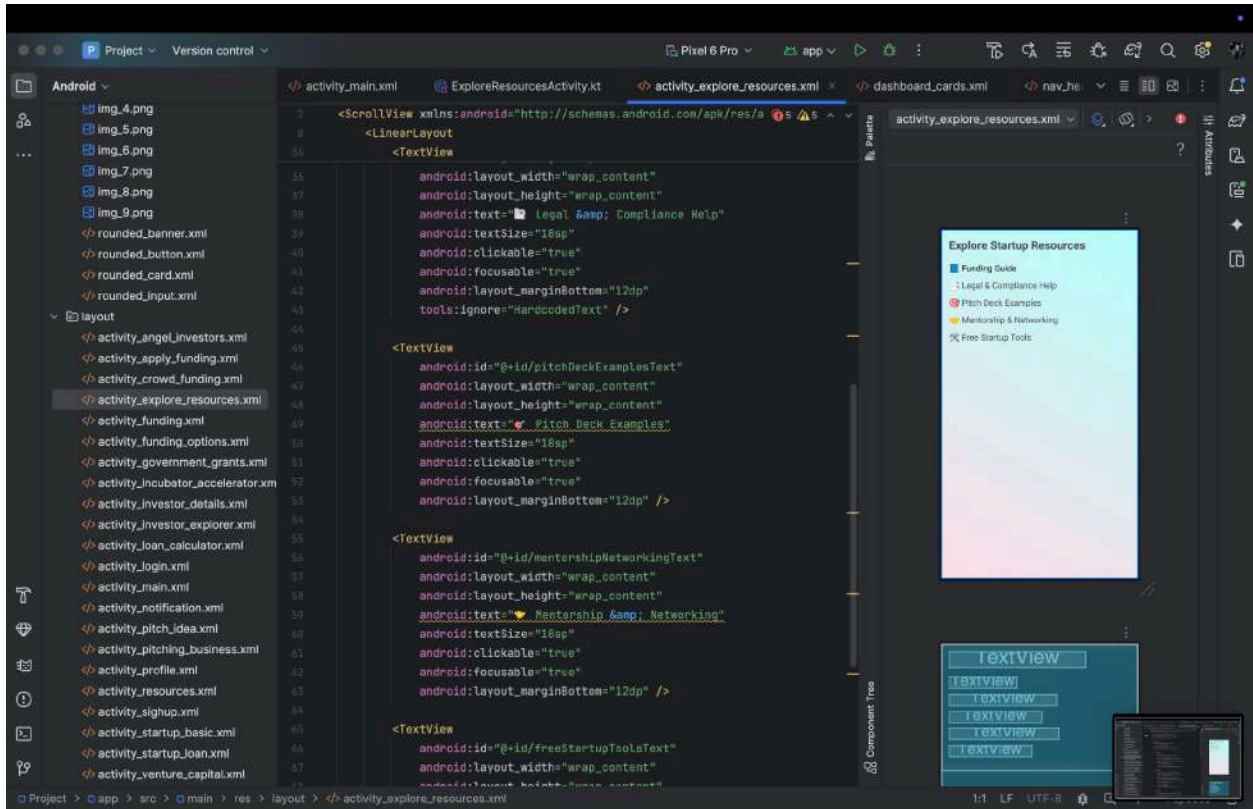
Error Handling: Provides Toast messages for both success and failure scenarios.

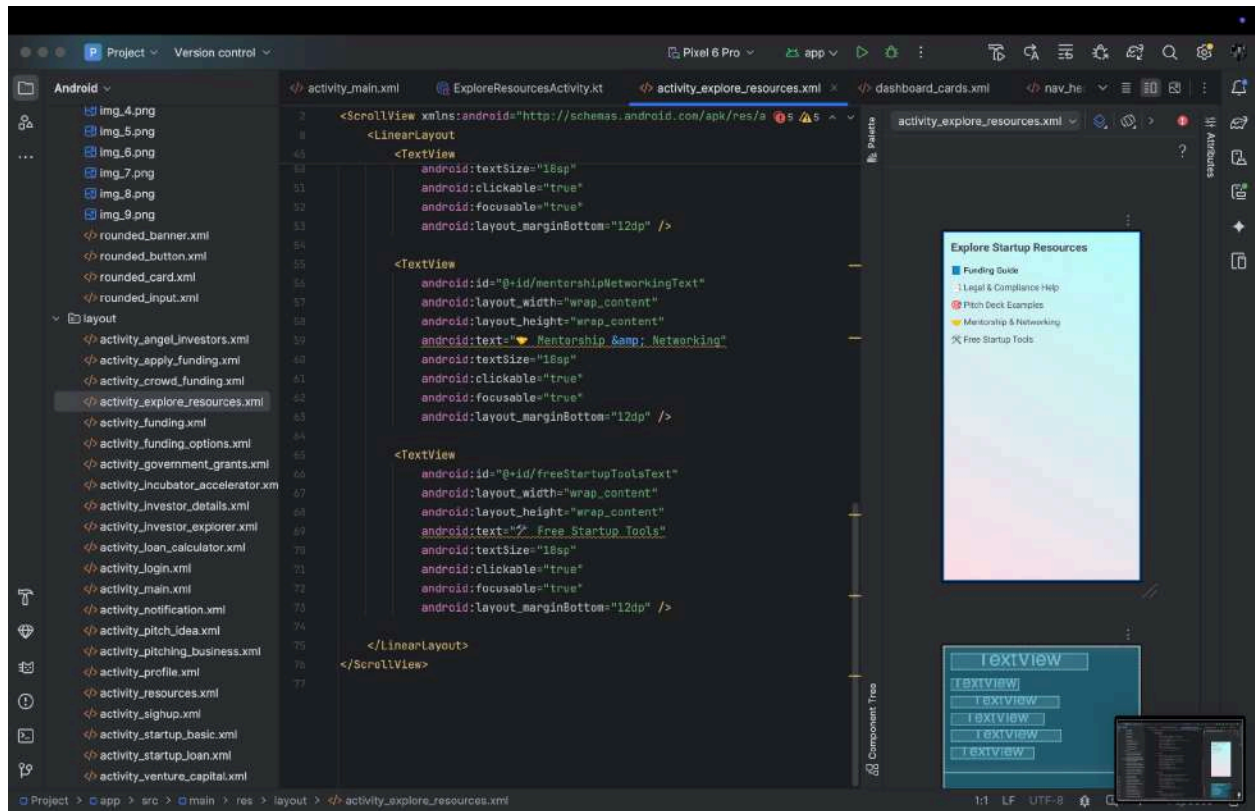




-> activity_explore_resources.xml

- Layout:
 - ScrollView is used to ensure the content is scrollable.
 - LinearLayout is used to arrange the elements vertically with padding.
- TextViews:
 - A header ([Explore Startup Resources](#)) is displayed at the top.
 - Five clickable TextViews follow the header, each representing a resource for startups:
 - Funding Guide: Guides related to funding.
 - Legal & Compliance Help: Information about legal and compliance.
 - Pitch Deck Examples: Examples for creating pitch decks.
 - Mentorship & Networking: Details on mentorship and networking.
 - Free Startup Tools: Resources for free startup tools.
- Clickable and Focusable: Each TextView is clickable and focusable, making them interactive to navigate to respective resources or actions (though navigation behavior isn't implemented in the XML layout).





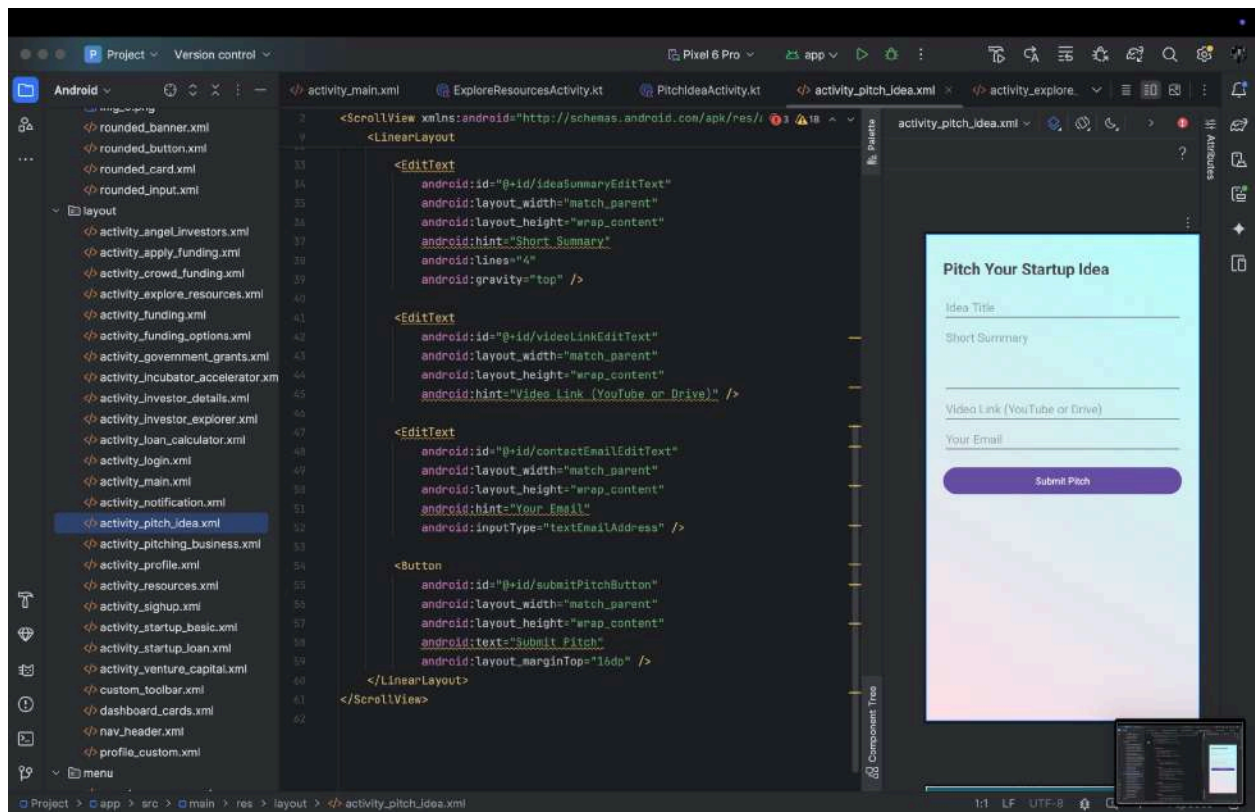
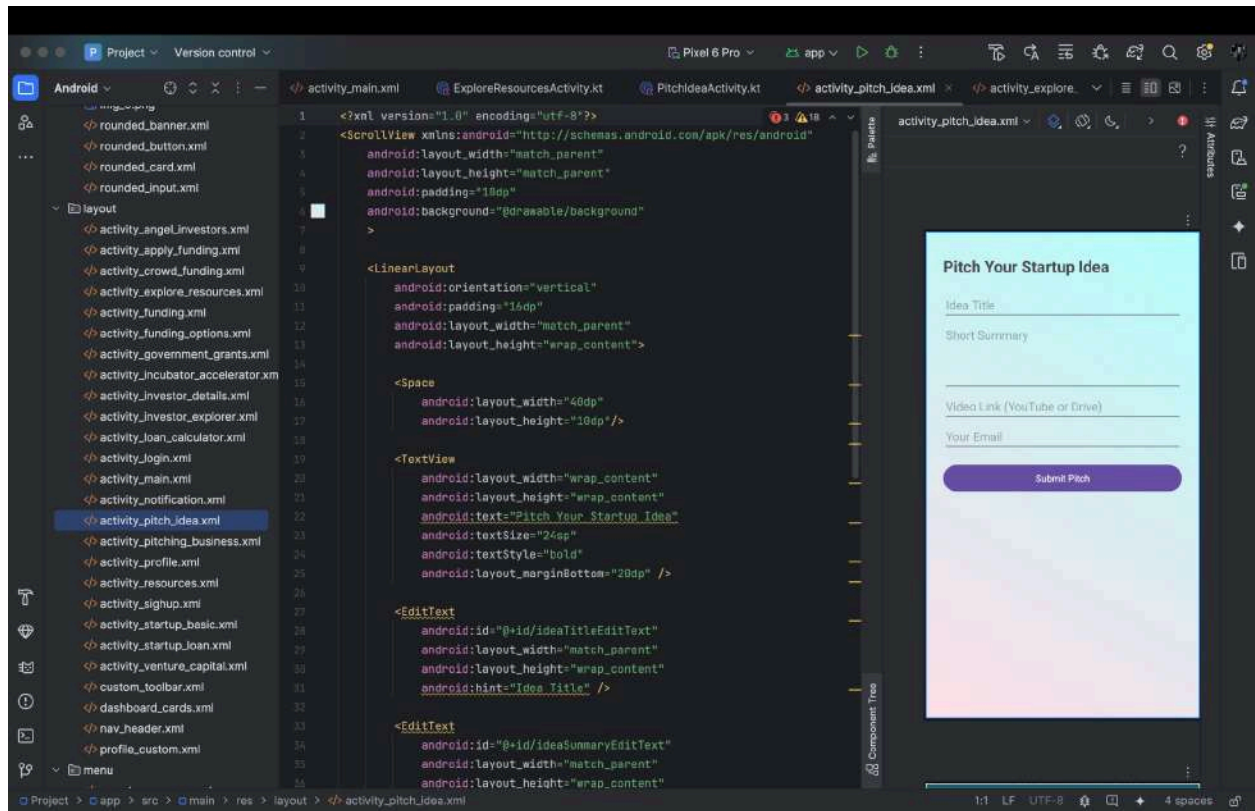
-> ExploreResourcesActivity.kt

1. The activity loads a layout where users can click on different resources.
2. Intent is used to open URLs using `Intent.ACTION_VIEW`.
3. Each resource corresponds to a different Google Drive file URL for the startup guide, legal help, pitch decks, mentorship, and free tools.

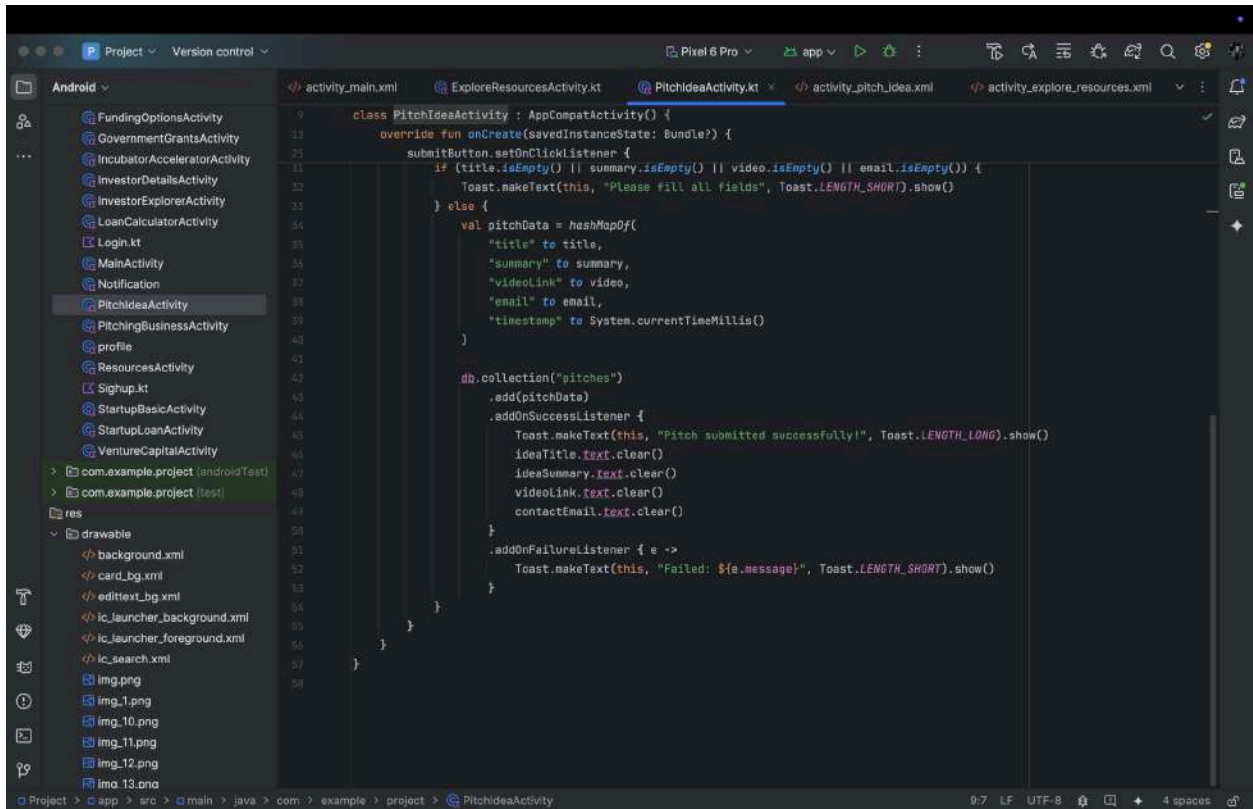
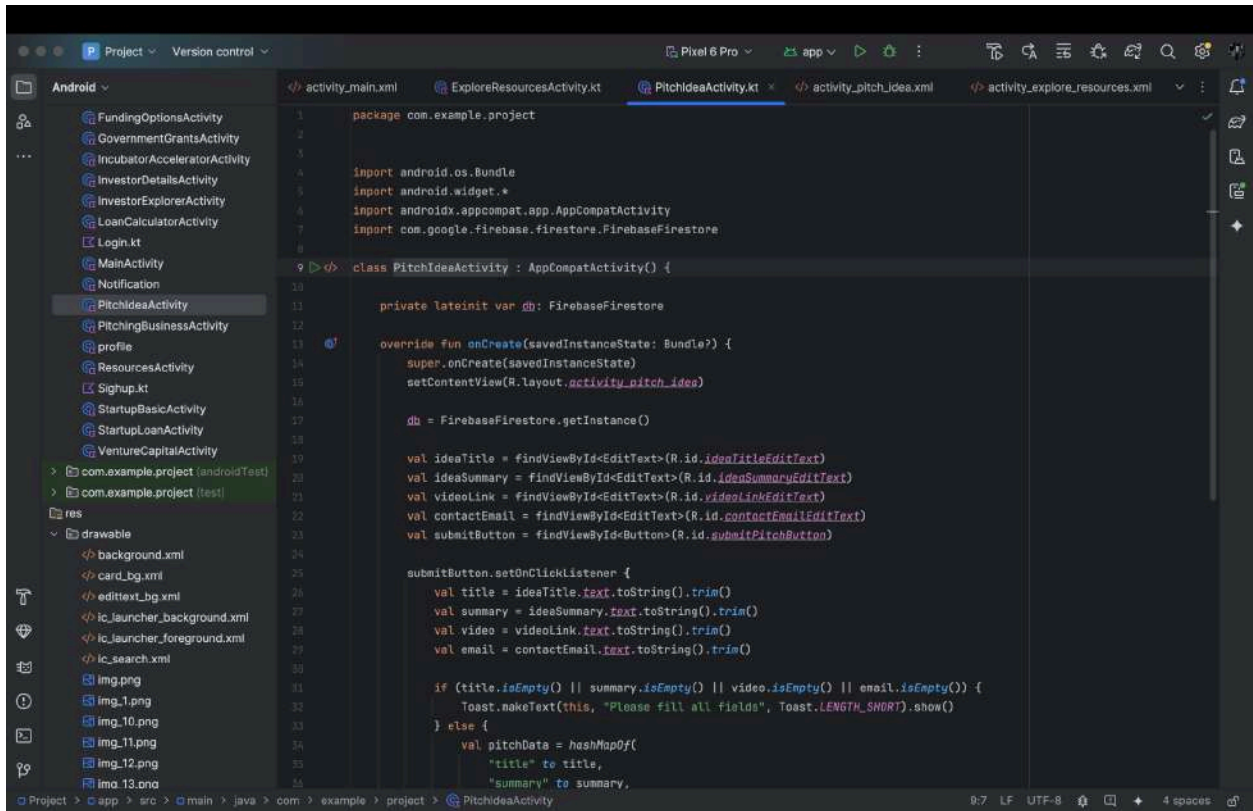
```
1 package com.example.project
2 import androidx.appcompat.app.AppCompatActivity
3 import android.os.Bundle
4 import android.view.View
5 import android.widget.TextView
6
7 class ExploreResourcesActivity : AppCompatActivity() {
8     @SuppressLint("MissingInflatedId")
9     override fun onCreate(savedInstanceState: Bundle?) {
10         super.onCreate(savedInstanceState)
11         setContentView(R.layout.activity_explore_resources)
12
13         val fundingGuide = findViewById<TextView>(R.id.fundingGuideText) // Update this ID if needed
14         val legalCompliance = findViewById<TextView>(R.id.legalComplianceText) // Update this ID if needed
15         val pitchDeckExamples = findViewById<TextView>(R.id.pitchDeckExamplesText) // Update this ID if needed
16         val mentorshipNetworking = findViewById<TextView>(R.id.mentorshipNetworkingText) // Update this ID if needed
17         val freeStartupTools = findViewById<TextView>(R.id.freeStartupToolsText) // Update this ID if needed
18
19         fundingGuide.setOnClickListener {
20             val intent = Intent(Intent.ACTION_VIEW)
21             intent.data = Uri.parse("https://drive.google.com/file/d/1ULdMIAvEF4vW3UzShR0-7WNUaKu1Nk/view?usp=sharing")
22             startActivity(intent)
23         }
24         legalCompliance.setOnClickListener {
25             val intent = Intent(Intent.ACTION_VIEW)
26             intent.data = Uri.parse("https://drive.google.com/file/d/105kxN6xZF3cyaH-WNngowz80kbWIP8q5/view?usp=sharing")
27             startActivity(intent)
28         }
29         pitchDeckExamples.setOnClickListener {
30             val intent = Intent(Intent.ACTION_VIEW)
31             intent.data = Uri.parse("https://drive.google.com/file/d/1aE-TT13tdJvU2d4A-3hr-Y5G0fe9480n/view?usp=sharing")
32             startActivity(intent)
33         }
34         mentorshipNetworking.setOnClickListener {
35             val intent = Intent(Intent.ACTION_VIEW)
36             intent.data = Uri.parse("https://drive.google.com/file/d/1TE3Ks23ycH8IFXWYfw5Luv4V0-7u8aon/view?usp=sharing")
37             startActivity(intent)
38         }
39         freeStartupTools.setOnClickListener {
40             val intent = Intent(Intent.ACTION_VIEW)
41             intent.data = Uri.parse("https://drive.google.com/file/d/1660F149Cv5Vda7MKmMY0bh5xaSJ_040Z/view?usp=sharing")
42             startActivity(intent)
43         }
44     }
45 }
```

```
9 class ExploreResourcesActivity : AppCompatActivity() {
10     override fun onCreate(savedInstanceState: Bundle?) {
11         legalCompliance.setOnClickListener {
12             val intent = Intent(Intent.ACTION_VIEW)
13             intent.data = Uri.parse("https://drive.google.com/file/d/105kxN6xZF3cyaH-WNngowz80kbWIP8q5/view?usp=sharing")
14             startActivity(intent)
15         }
16         pitchDeckExamples.setOnClickListener {
17             val intent = Intent(Intent.ACTION_VIEW)
18             intent.data = Uri.parse("https://drive.google.com/file/d/1aE-TT13tdJvU2d4A-3hr-Y5G0fe9480n/view?usp=sharing")
19             startActivity(intent)
20         }
21         mentorshipNetworking.setOnClickListener {
22             val intent = Intent(Intent.ACTION_VIEW)
23             intent.data = Uri.parse("https://drive.google.com/file/d/1TE3Ks23ycH8IFXWYfw5Luv4V0-7u8aon/view?usp=sharing")
24             startActivity(intent)
25         }
26         freeStartupTools.setOnClickListener {
27             val intent = Intent(Intent.ACTION_VIEW)
28             intent.data = Uri.parse("https://drive.google.com/file/d/1660F149Cv5Vda7MKmMY0bh5xaSJ_040Z/view?usp=sharing")
29             startActivity(intent)
30         }
31     }
32 }
```

-> activity_pitch_ideas.xml



-> PitchIdeaActivity.kt



UI Elements:

- `EditText` fields for Idea Title, Idea Summary, Video Link, and Contact Email.
- A Submit Button to submit the pitch idea.

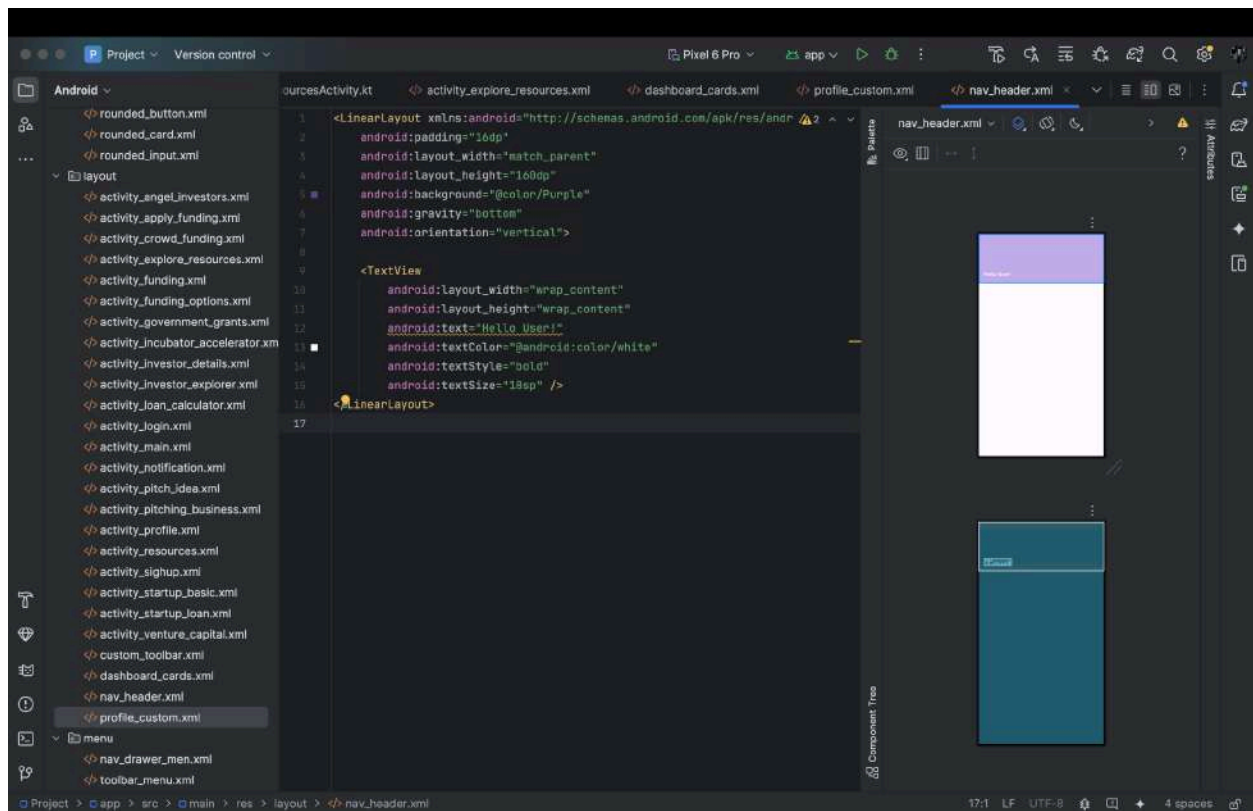
Firestore Integration:

- Firestore is used to save the pitch data in the "pitches" collection.
- On successful submission, a confirmation message is shown, and the input fields are cleared.
- If the submission fails, an error message is displayed.

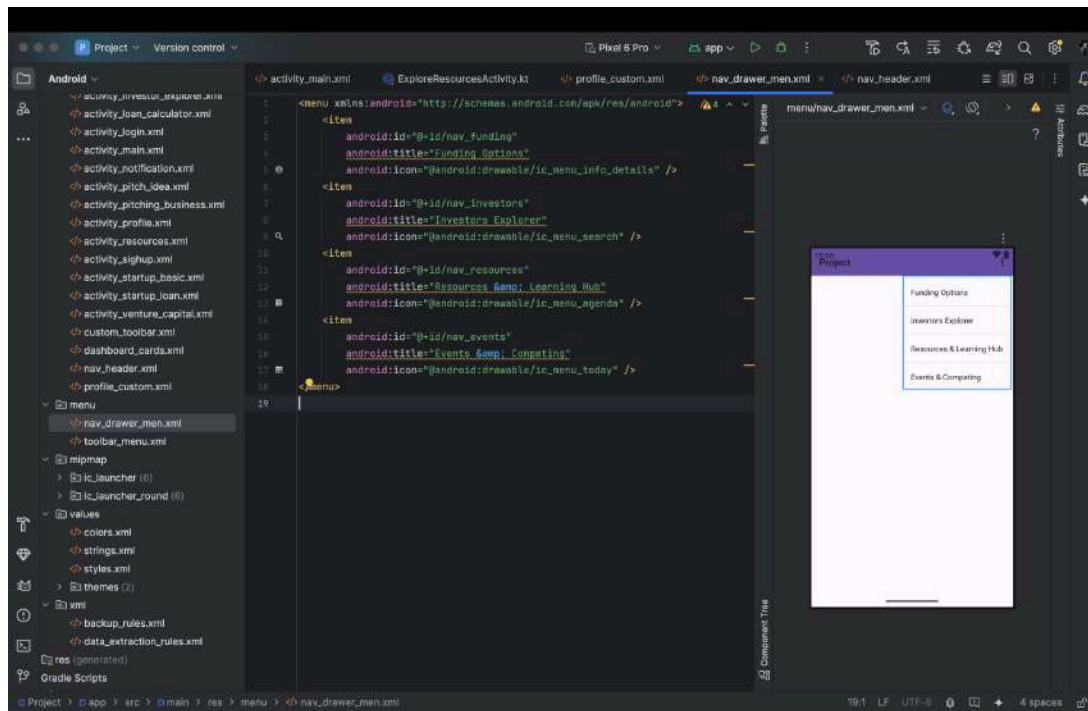
Validation:

- Ensures that all fields are filled before submission. If any field is empty, a toast is shown to prompt the user to fill in all fields.

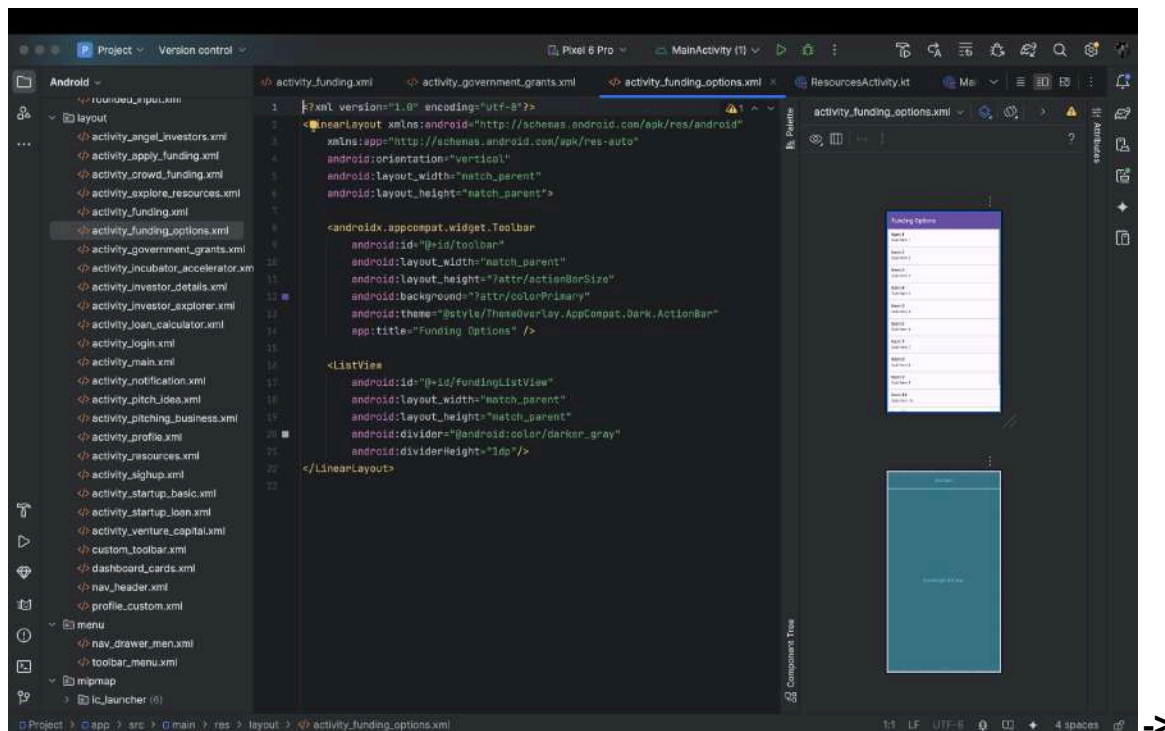
-> nav_header.xml



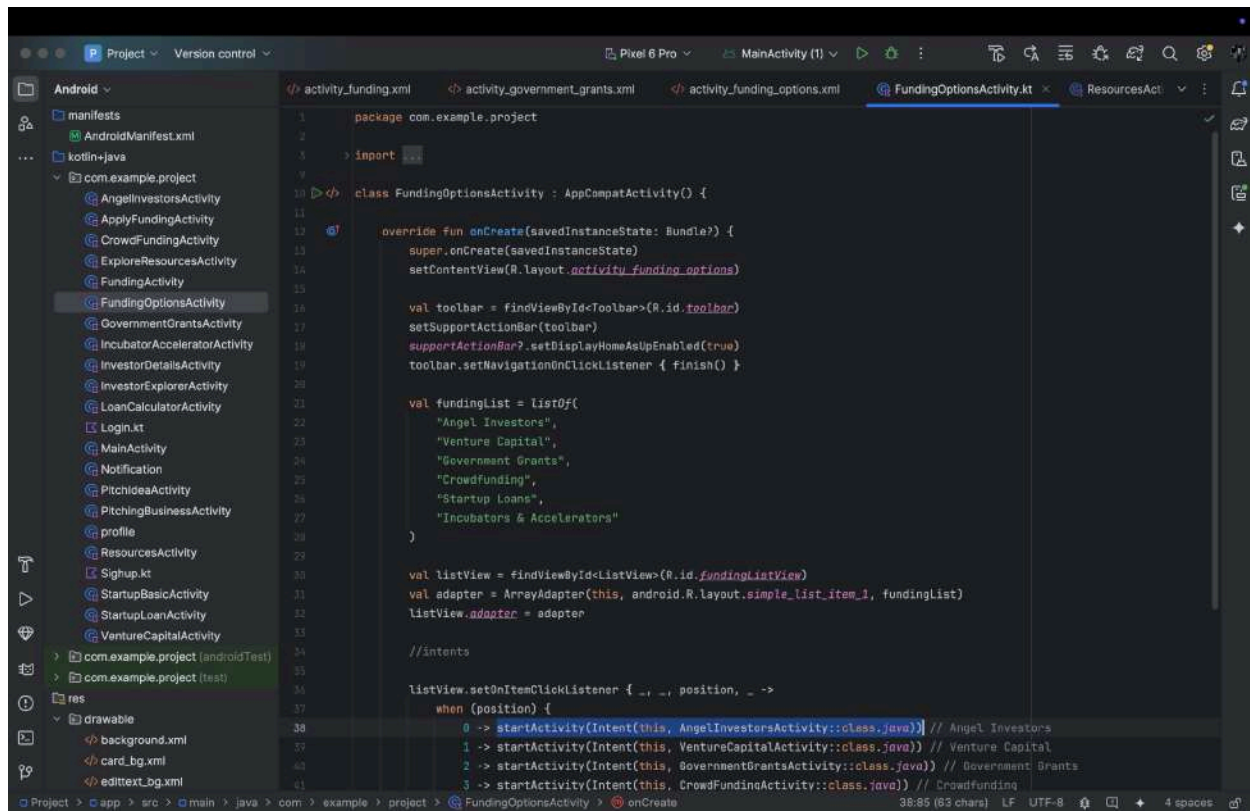
Nav_drawer_menu.xml



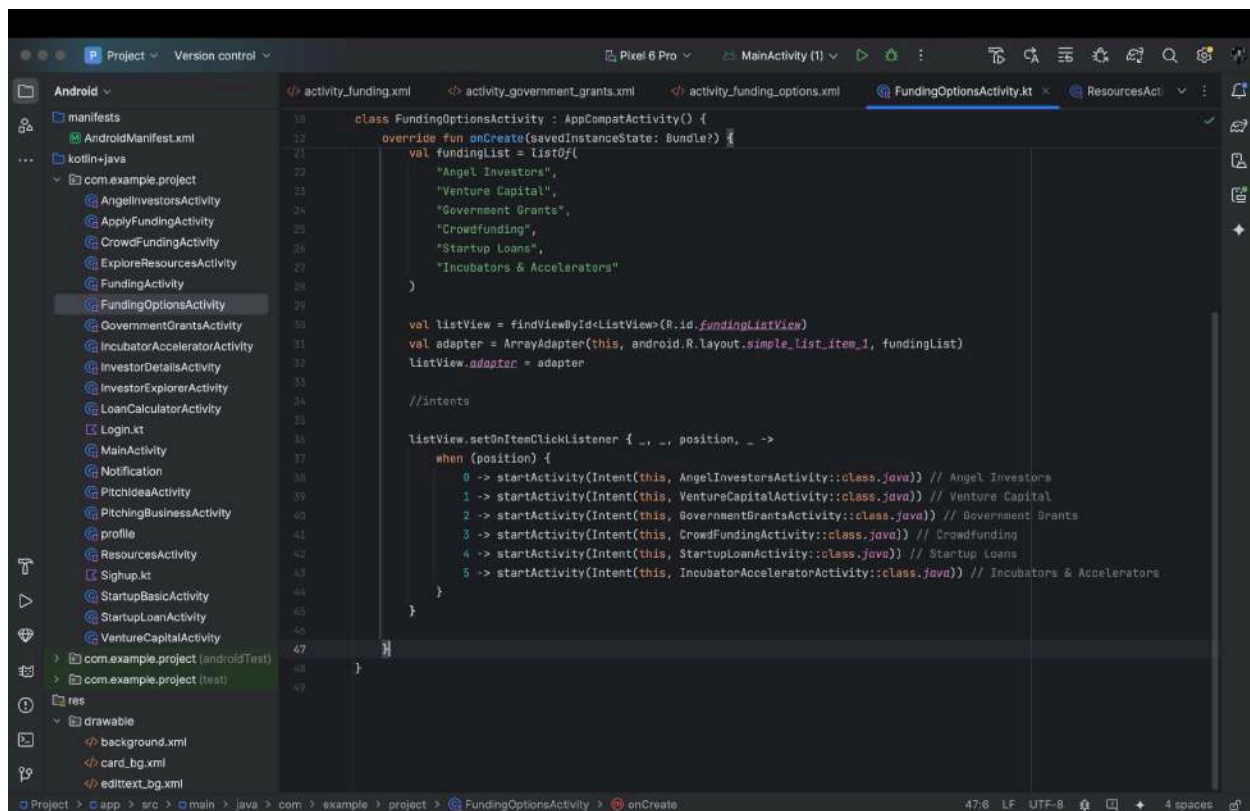
-> activity_funding_options.xml



-> FundingOptionsActivity.kt



```
1 package com.example.project
2
3 import androidx.appcompat.app.AppCompatActivity
4
5 class FundingOptionsActivity : AppCompatActivity() {
6
7     override fun onCreate(savedInstanceState: Bundle?) {
8         super.onCreate(savedInstanceState)
9         setContentView(R.layout.activity_funding_options)
10
11         val toolbar = findViewById<Toolbar>(R.id.toolbar)
12         setSupportActionBar(toolbar)
13         supportActionBar?.setDisplayHomeAsUpEnabled(true)
14         toolbar.setNavigationOnClickListener { finish() }
15
16         val fundingList = listOf(
17             "Angel Investors",
18             "Venture Capital",
19             "Government Grants",
20             "Crowdfunding",
21             "Startup Loans",
22             "Incubators & Accelerators"
23         )
24
25         val listView = findViewById<ListView>(R.id.fundingListView)
26         val adapter = ArrayAdapter(this, android.R.layout.simple_list_item_1, fundingList)
27         listView.adapter = adapter
28
29         //Intents
30
31         listView.setOnItemClickListener { _, _, position, _ ->
32             when (position) {
33                 0 -> startActivity(Intent(this, AngelInvestorsActivity::class.java)) // Angel Investors
34                 1 -> startActivity(Intent(this, VentureCapitalActivity::class.java)) // Venture Capital
35                 2 -> startActivity(Intent(this, GovernmentGrantsActivity::class.java)) // Government Grants
36                 3 -> startActivity(Intent(this, CrowdfundingActivity::class.java)) // Crowdfunding
37             }
38         }
39     }
40 }
```



```
10 class FundingOptionsActivity : AppCompatActivity() {
11     override fun onCreate(savedInstanceState: Bundle?) {
12         val fundingList = listOf(
13             "Angel Investors",
14             "Venture Capital",
15             "Government Grants",
16             "Crowdfunding",
17             "Startup Loans",
18             "Incubators & Accelerators"
19         )
20
21         val listView = findViewById<ListView>(R.id.fundingListView)
22         val adapter = ArrayAdapter(this, android.R.layout.simple_list_item_1, fundingList)
23         listView.adapter = adapter
24
25         //Intents
26
27         listView.setOnItemClickListener { _, _, position, _ ->
28             when (position) {
29                 0 -> startActivity(Intent(this, AngelInvestorsActivity::class.java)) // Angel Investors
30                 1 -> startActivity(Intent(this, VentureCapitalActivity::class.java)) // Venture Capital
31                 2 -> startActivity(Intent(this, GovernmentGrantsActivity::class.java)) // Government Grants
32                 3 -> startActivity(Intent(this, CrowdfundingActivity::class.java)) // Crowdfunding
33                 4 -> startActivity(Intent(this, StartupLoanActivity::class.java)) // Startup Loans
34                 5 -> startActivity(Intent(this, IncubatorAcceleratorActivity::class.java)) // Incubators & Accelerators
35             }
36         }
37     }
38 }
```

Toolbar Setup:

- A toolbar is set up as the action bar (`setSupportActionBar(toolbar)`), and the back button (`setDisplayHomeAsUpEnabled(true)`) is enabled to allow navigation back to the previous activity.

List of Funding Options:

- A list of funding options is created (`fundingList`), including options like "Angel Investors", "Venture Capital", etc.

ListView:

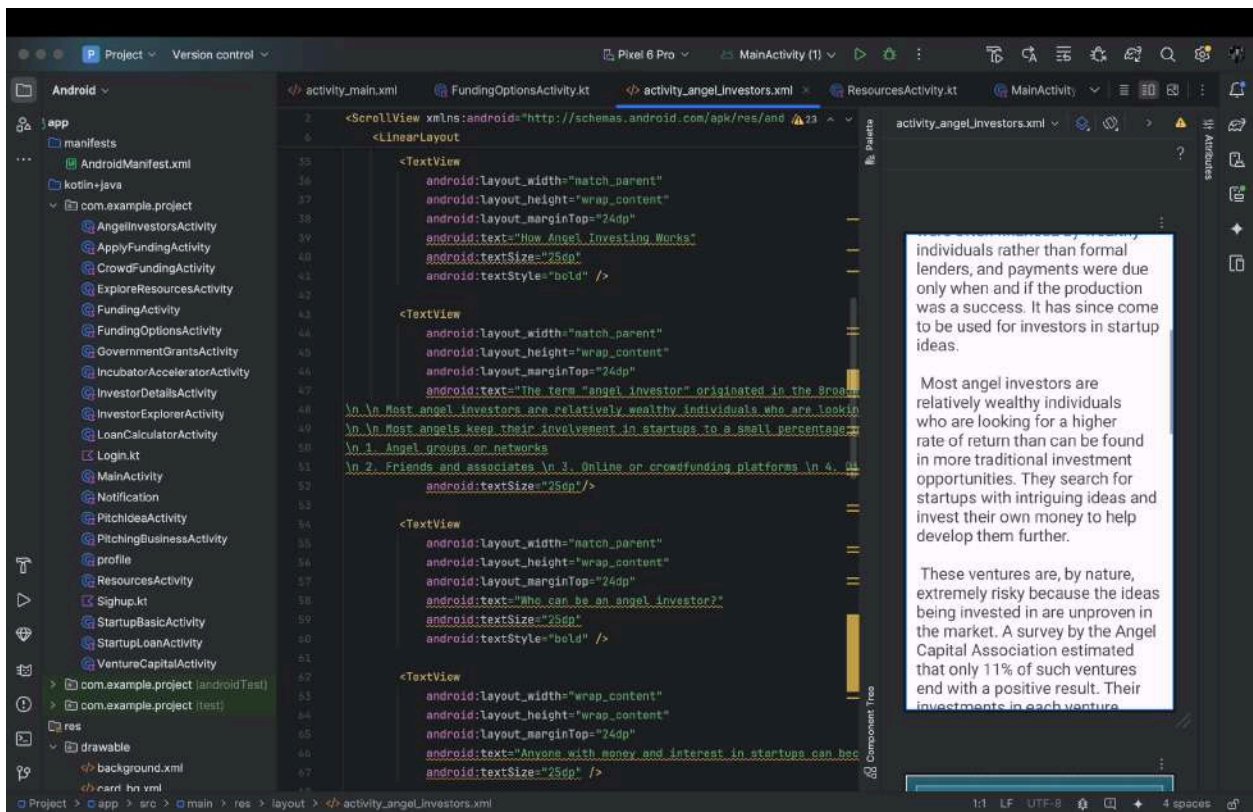
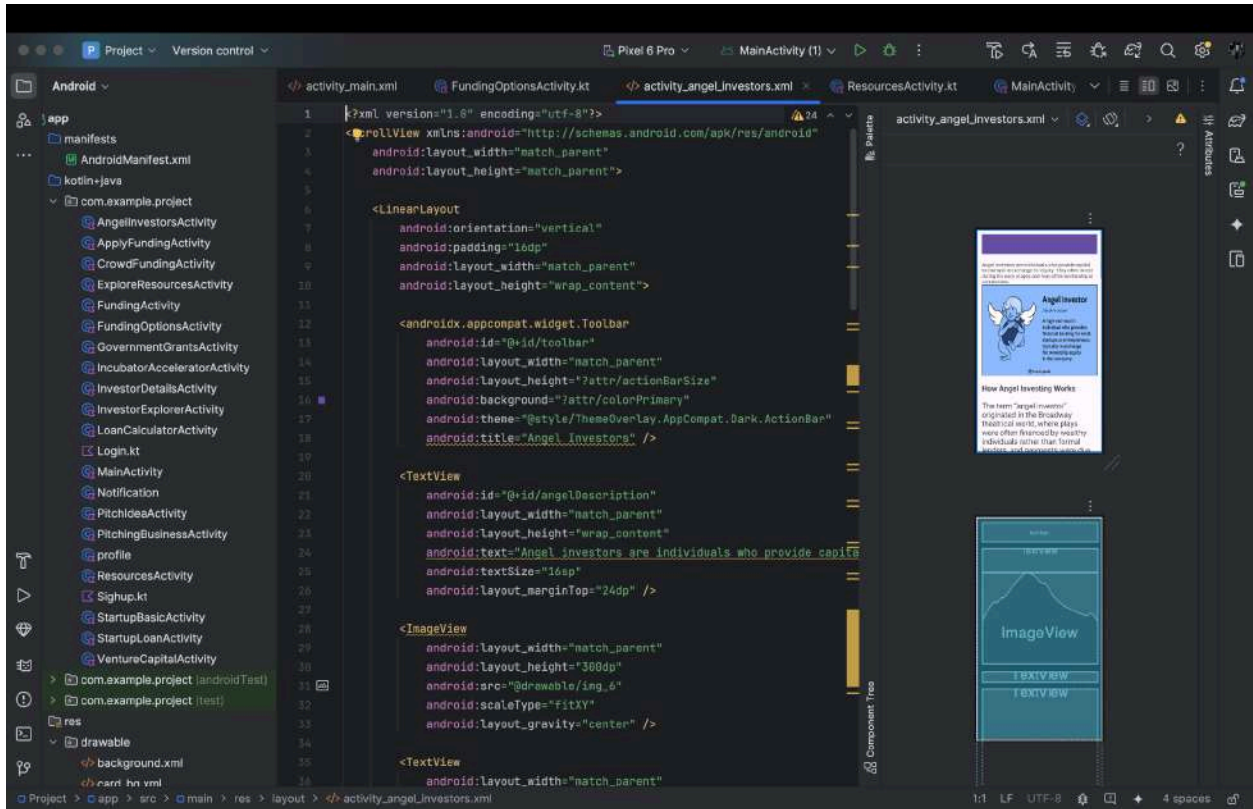
- A `ListView` is used to display the list of funding options. The data is passed to the `ListView` through an `ArrayAdapter`, which maps each item in the `fundingList` to a simple list item layout (`android.R.layout.simple_list_item_1`).

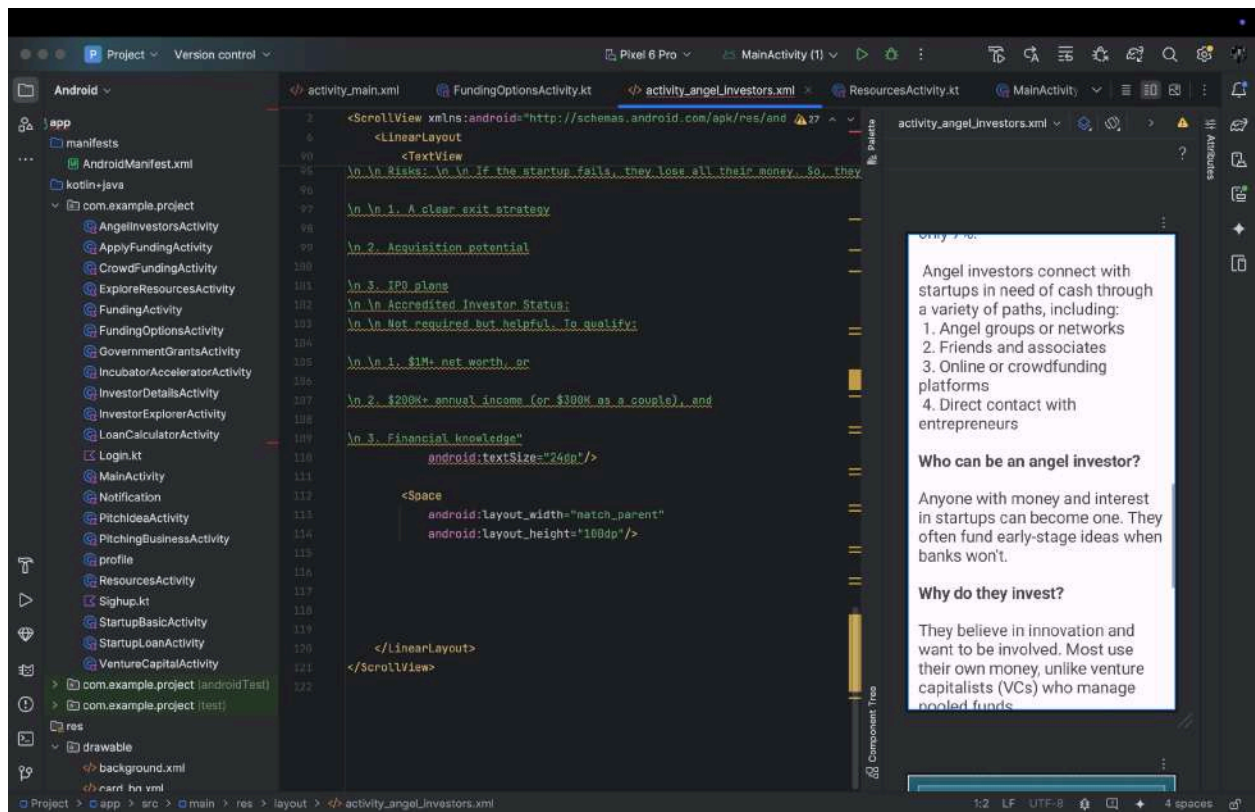
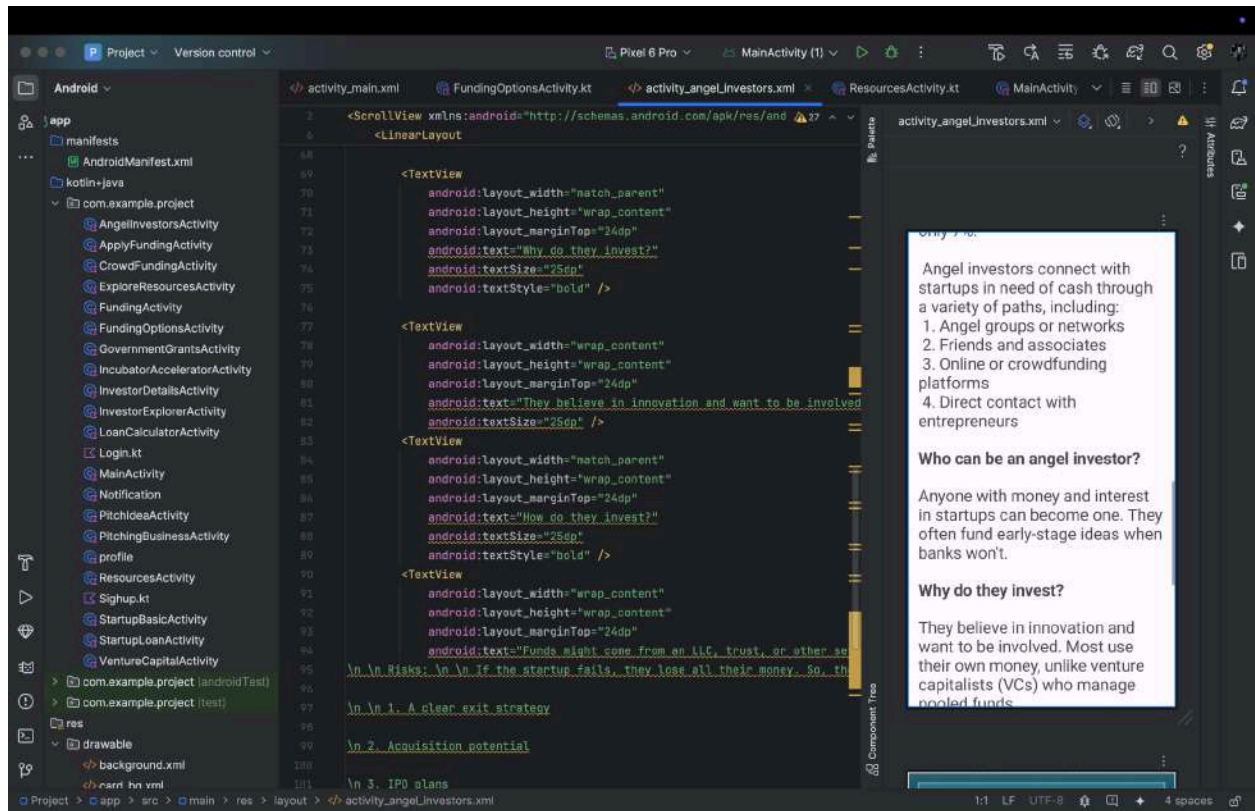
Item Click Listener:

- When a user clicks on an item in the list, the `setOnItemClickListener` is triggered. Based on the position of the clicked item, it starts a new activity corresponding to the selected funding option.
 - For example, selecting "Angel Investors" will open the `AngelInvestorsActivity`.

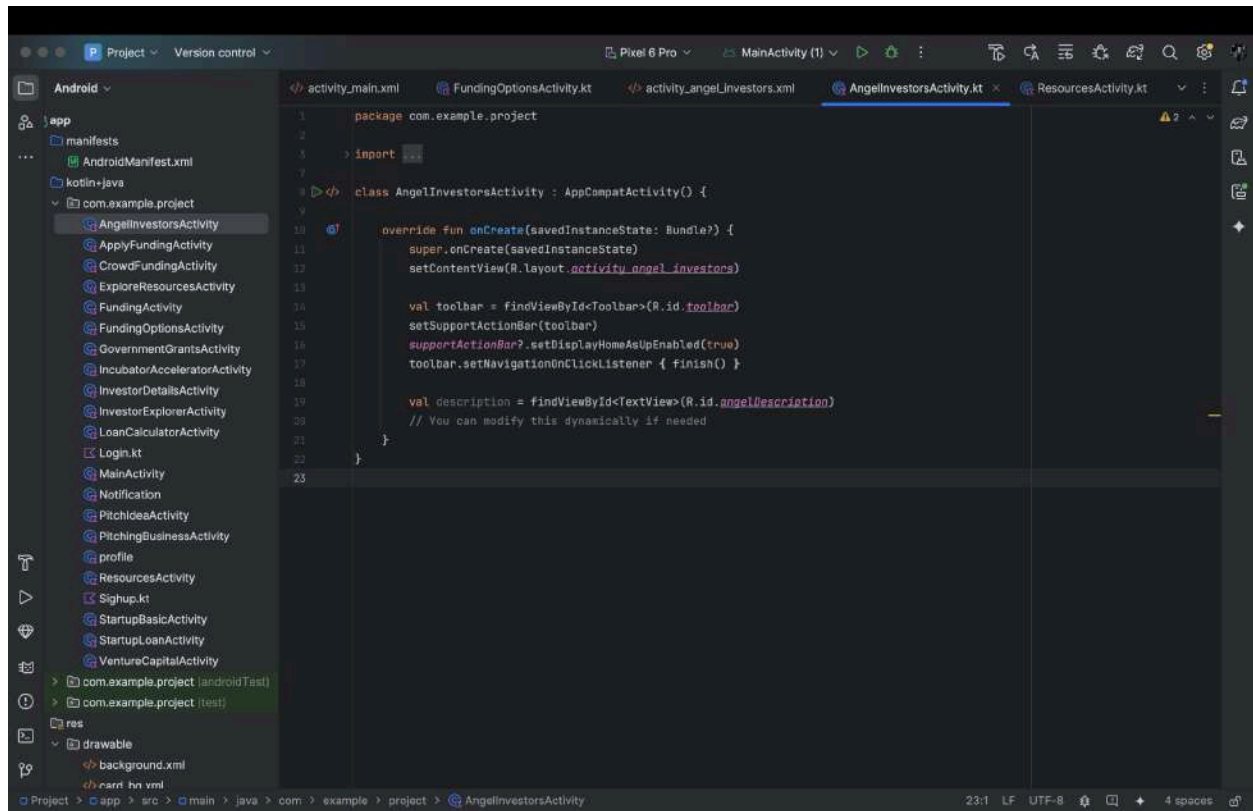
-> activity_angel_investors.xml

1. `ScrollView`: Makes the screen scrollable.
2. `LinearLayout`: Arranges items vertically.
3. `Toolbar`: Shows the title "Angel Investors" with a back button.
4. `TextViews`: Display information in sections (what is angel investing, who can be an angel, etc.).
5. `ImageView`: Shows an image.
6. `Space`: Adds empty space at the bottom.





-> AngelInvestorActivity.kt

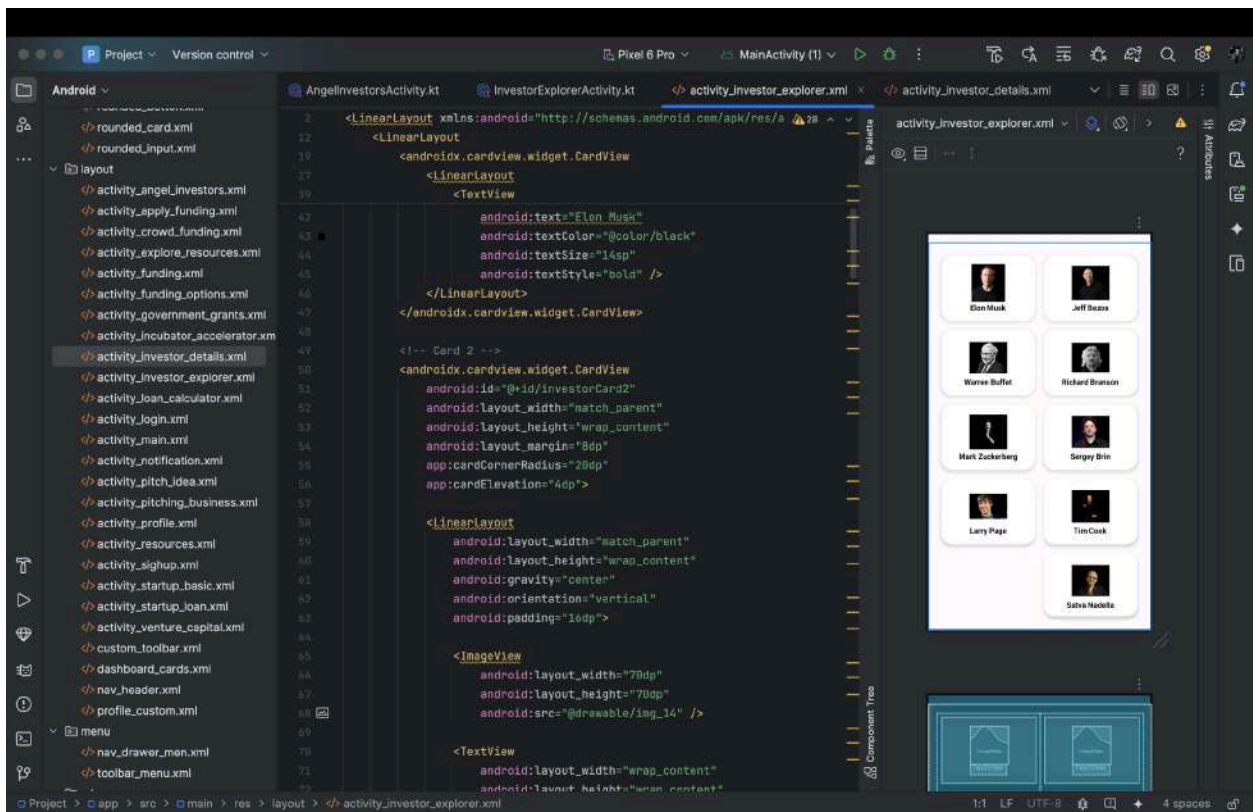
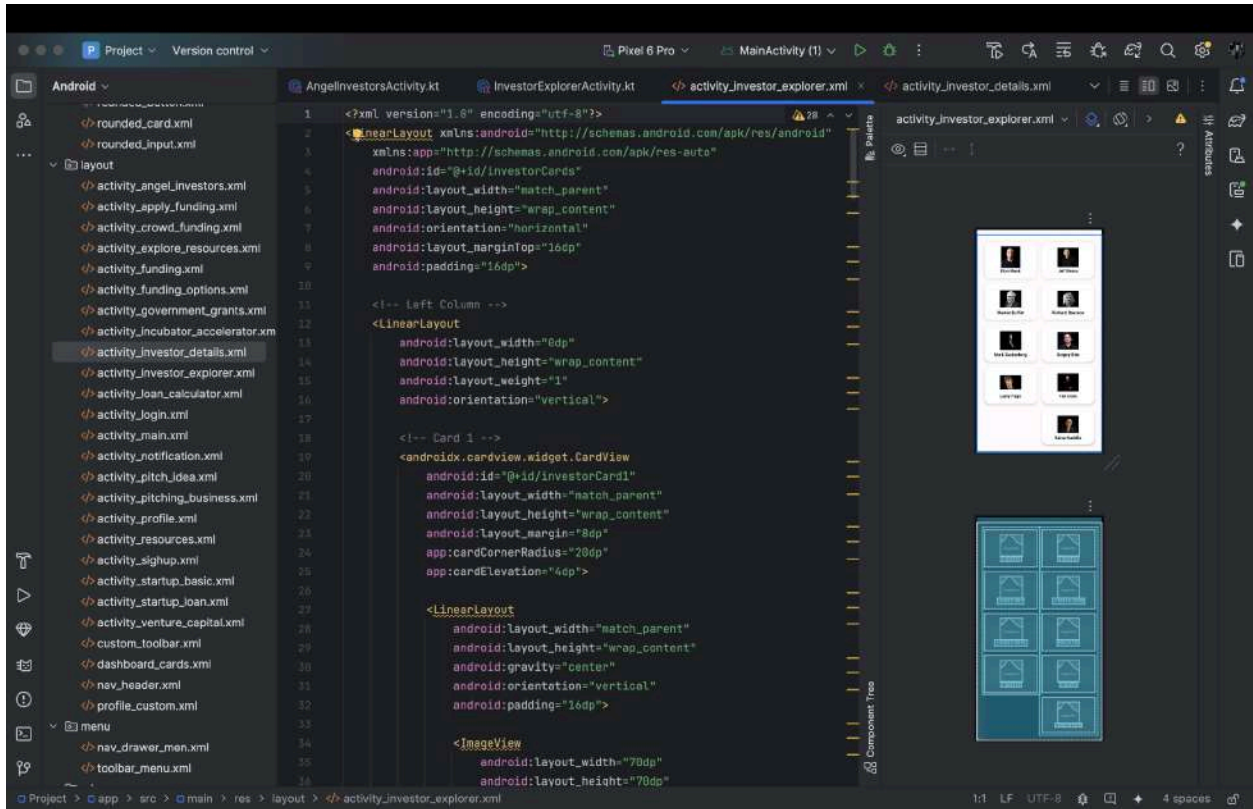


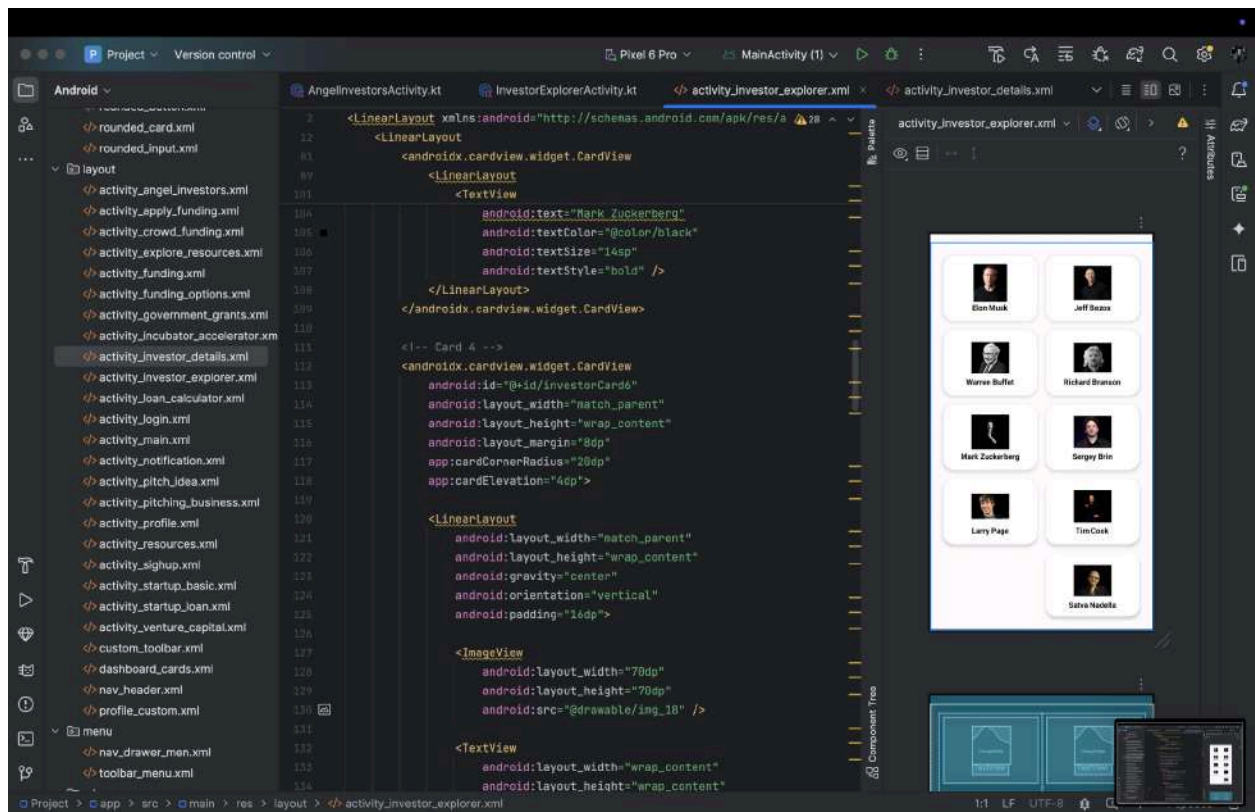
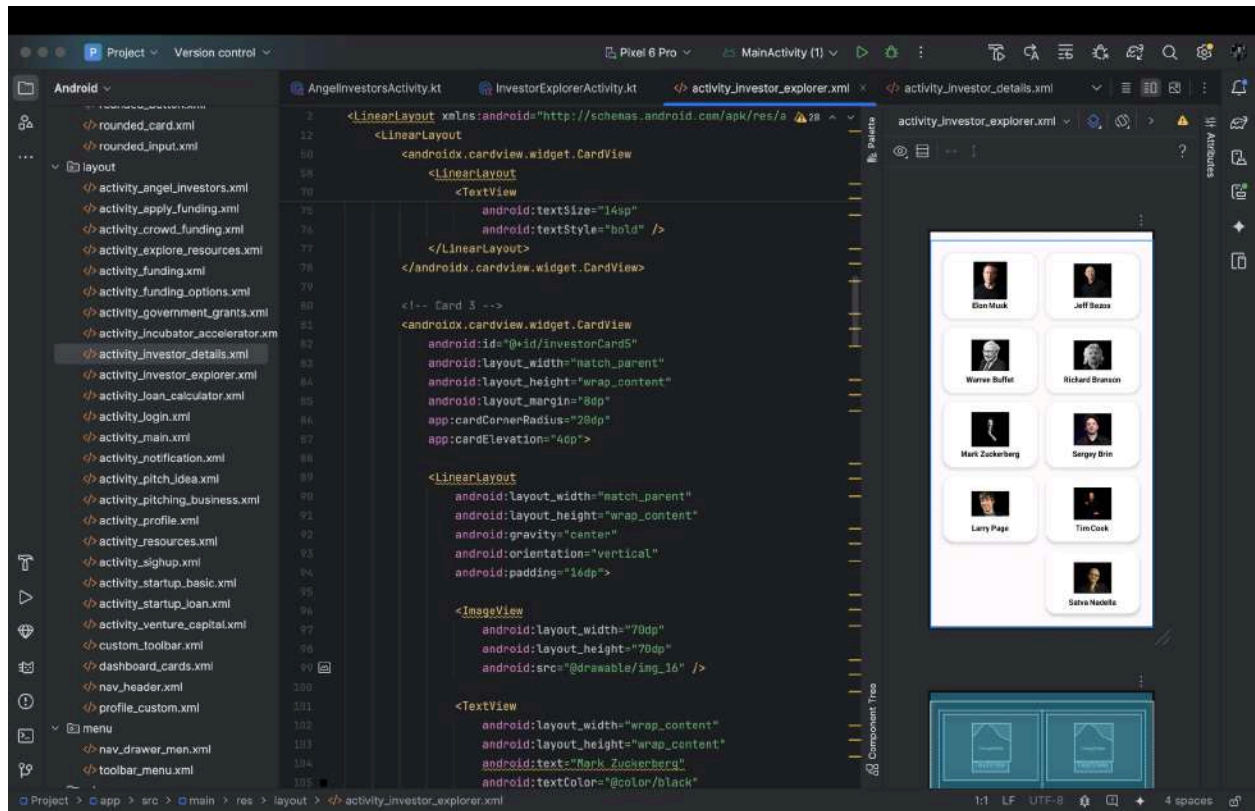
-> it is same for other funding options too

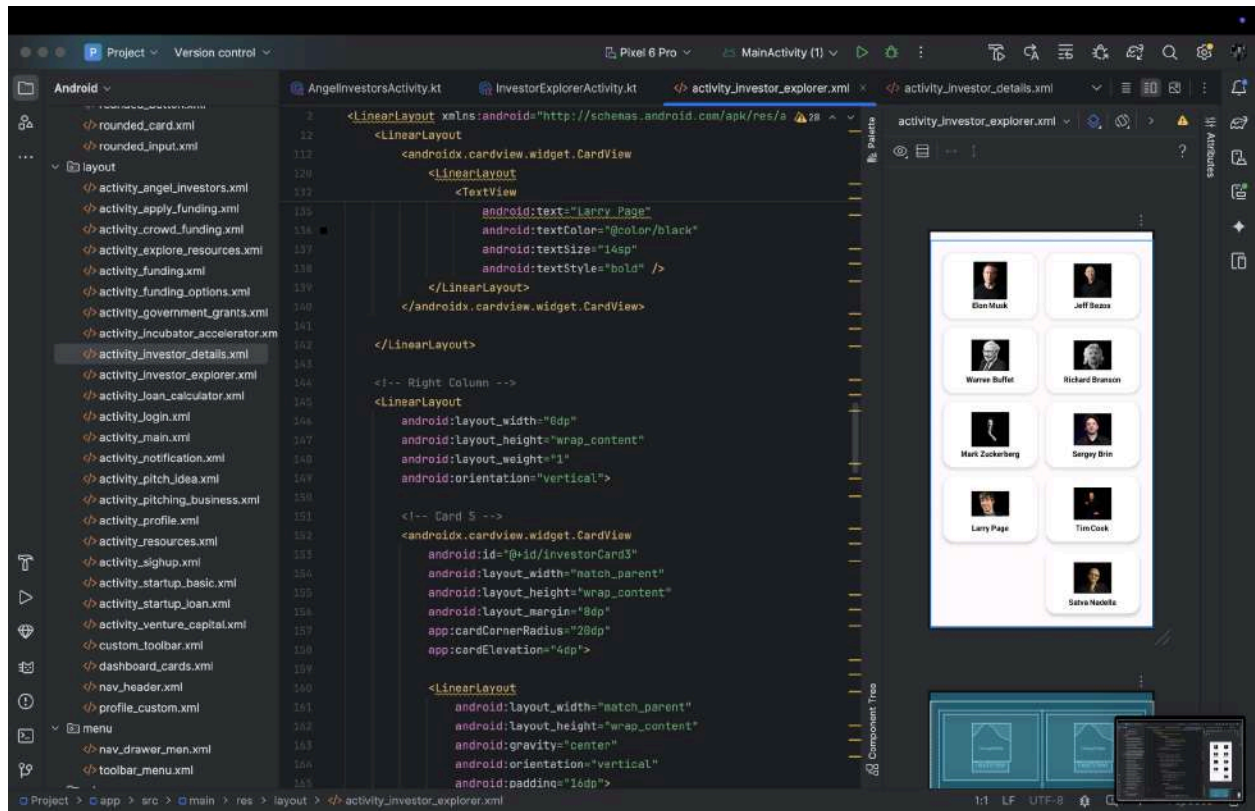
-> activity_investor_explorer.xml

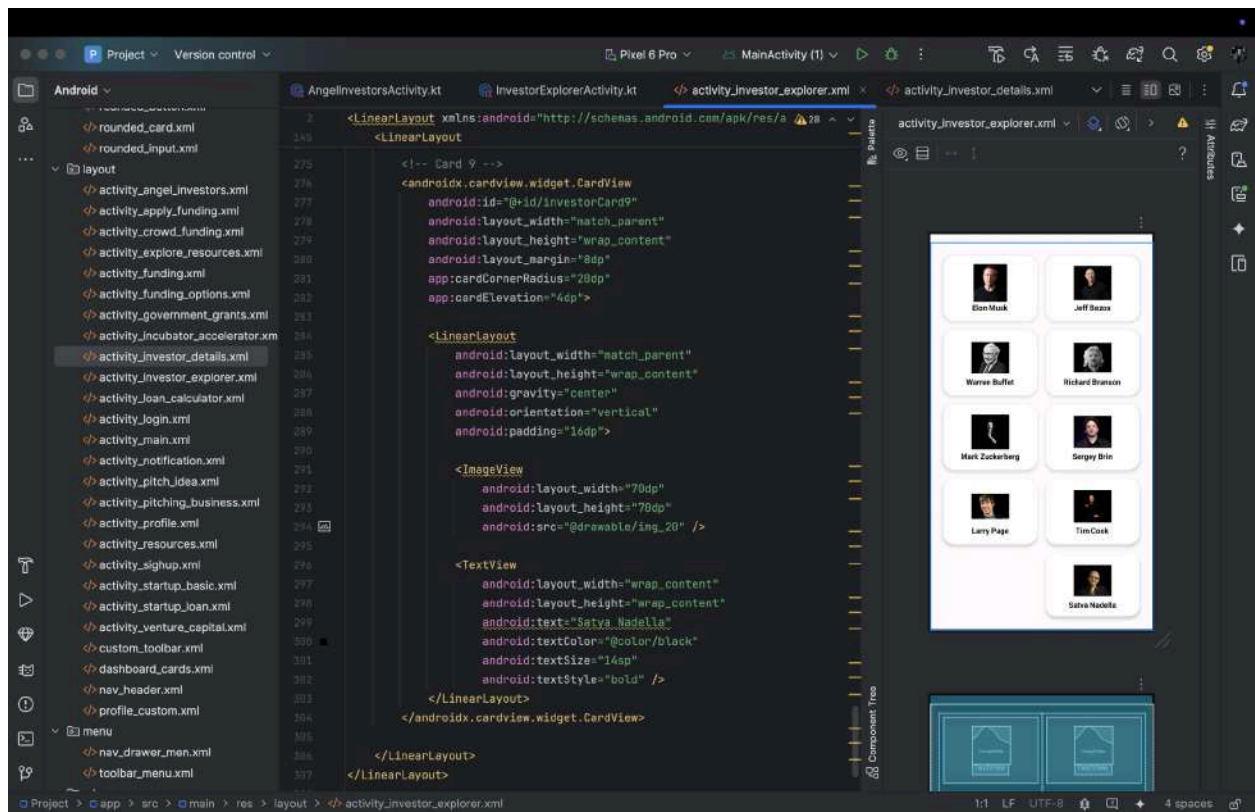
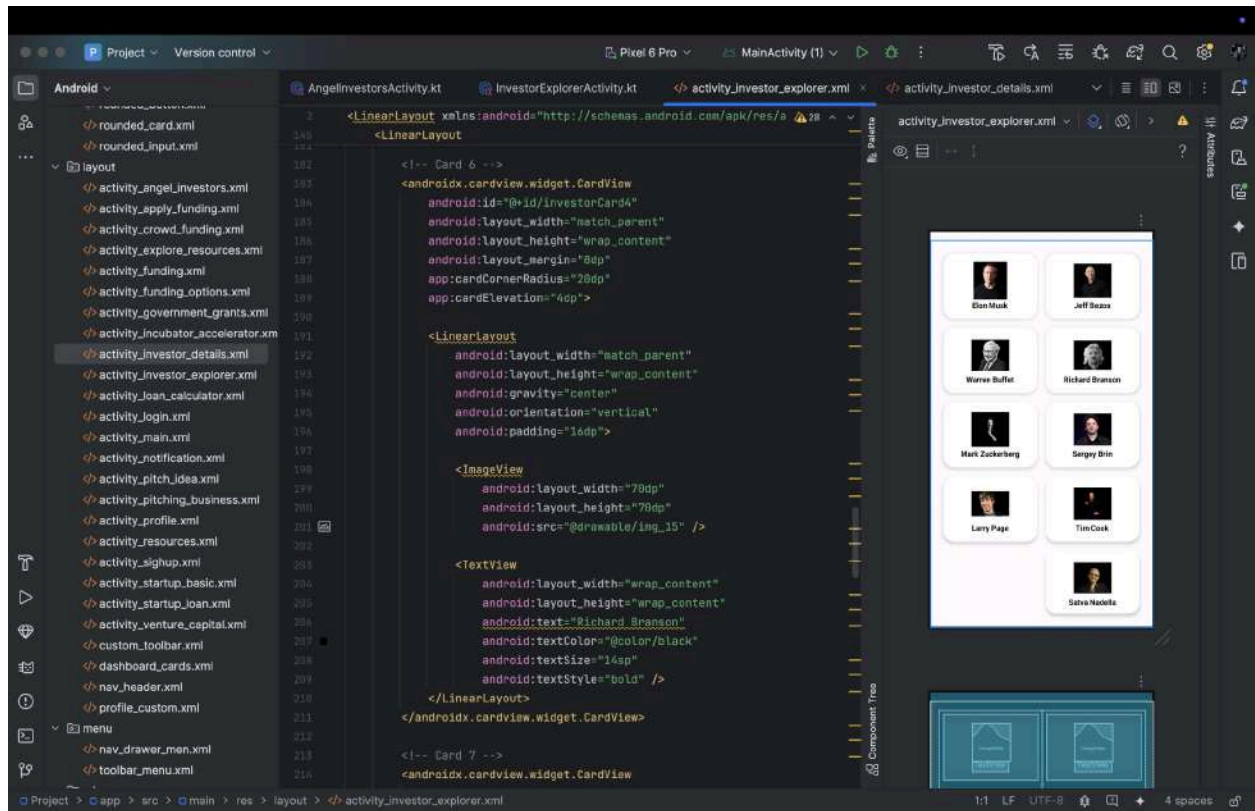
- Parent LinearLayout: Horizontal, with two columns (left and right).
- Each column: Vertical LinearLayout with multiple CardViews inside.
- Each CardView:
 - Shows an ImageView (investor photo)
 - Below that, a TextView (investor name)
- 9 Cards total: Famous investors like Elon Musk, Warren Buffet, Jeff Bezos, etc.
- Cards: Rounded corners (20dp), slight shadow (elevation 4dp).

👉 Final look: Two-column grid of investor cards with images and names.

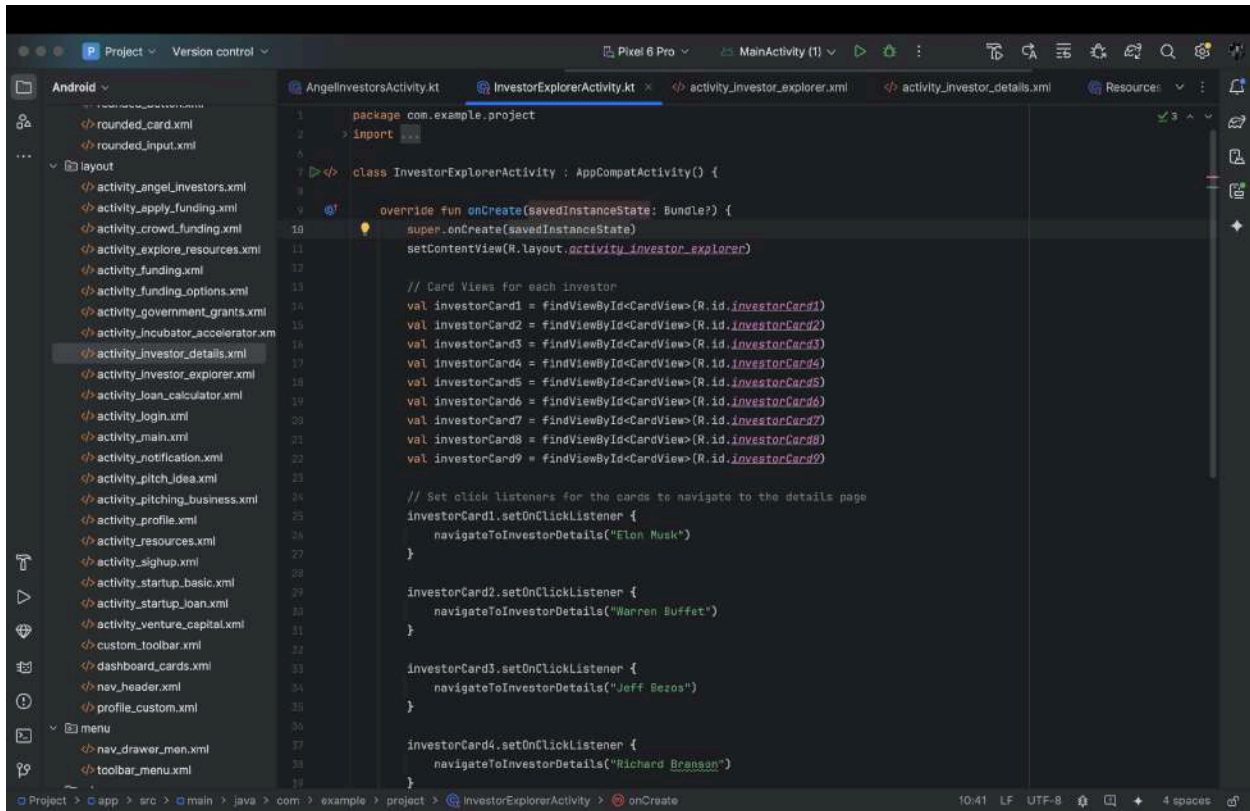




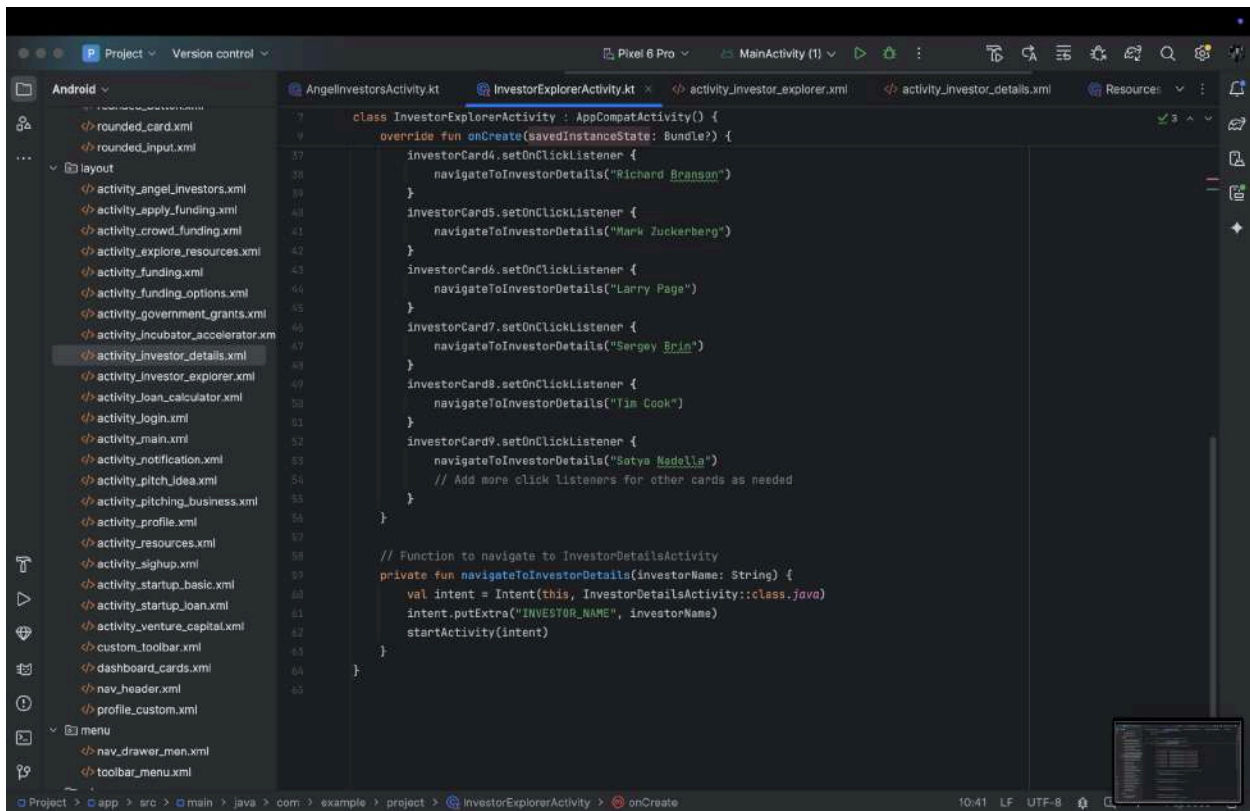




-> InvestorExplorerActivity.kt



```
1 package com.example.project
2 import androidx.appcompat.app.AppCompatActivity
3
4 class InvestorExplorerActivity : AppCompatActivity() {
5
6     override fun onCreate(savedInstanceState: Bundle?) {
7         super.onCreate(savedInstanceState)
8         setContentView(R.layout.activity_investor_explorer)
9
10        // Card Views for each investor
11        val investorCard1 = findViewById<CardView>(R.id.investorCard1)
12        val investorCard2 = findViewById<CardView>(R.id.investorCard2)
13        val investorCard3 = findViewById<CardView>(R.id.investorCard3)
14        val investorCard4 = findViewById<CardView>(R.id.investorCard4)
15        val investorCard5 = findViewById<CardView>(R.id.investorCard5)
16        val investorCard6 = findViewById<CardView>(R.id.investorCard6)
17        val investorCard7 = findViewById<CardView>(R.id.investorCard7)
18        val investorCard8 = findViewById<CardView>(R.id.investorCard8)
19        val investorCard9 = findViewById<CardView>(R.id.investorCard9)
20
21        // Set click listeners for the cards to navigate to the details page
22        investorCard1.setOnClickListener {
23            navigateToInvestorDetails("Eton Musk")
24        }
25
26        investorCard2.setOnClickListener {
27            navigateToInvestorDetails("Warren Buffet")
28        }
29
30        investorCard3.setOnClickListener {
31            navigateToInvestorDetails("Jeff Bezos")
32        }
33
34        investorCard4.setOnClickListener {
35            navigateToInvestorDetails("Richard Branson")
36        }
37    }
38}
```

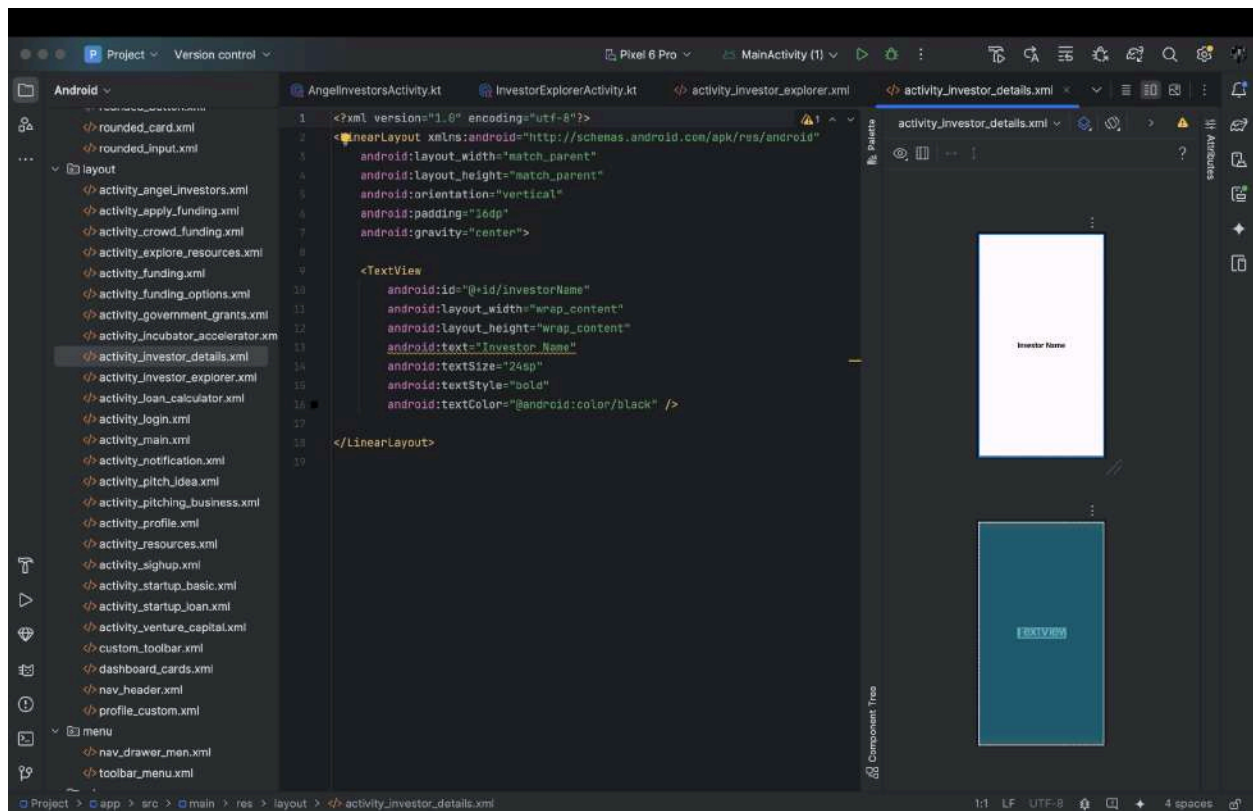


```
37        investorCard4.setOnClickListener {
38            navigateToInvestorDetails("Richard Branson")
39        }
40        investorCard5.setOnClickListener {
41            navigateToInvestorDetails("Mark Zuckerberg")
42        }
43        investorCard6.setOnClickListener {
44            navigateToInvestorDetails("Larry Page")
45        }
46        investorCard7.setOnClickListener {
47            navigateToInvestorDetails("Sergey Brin")
48        }
49        investorCard8.setOnClickListener {
50            navigateToInvestorDetails("Tim Cook")
51        }
52        investorCard9.setOnClickListener {
53            navigateToInvestorDetails("Satya Nadella")
54        }
55        // Add more click listeners for other cards as needed
56    }
57
58    // Function to navigate to InvestorDetailsActivity
59    private fun navigateToInvestorDetails(investorName: String) {
60        val intent = Intent(this, InvestorDetailsActivity::class.java)
61        intent.putExtra("INVESTOR_NAME", investorName)
62        startActivity(intent)
63    }
64}
```


- A is an `AppCompatActivity` showing investor cards.
- Each card (`CardView`) is linked by `findViewById`.
- When a card is clicked → `navigateToInvestorDetails()` is called with the investor's name.
- `navigateToInvestorDetails()` → Starts `InvestorDetailsActivity` and passes the name using Intent extra.

👉 Simply:
Card click → Opens new page with investor's name.

-> activity_investor_details.xml



~ This is like if you click the investor name it will take u to this page through intent and here you can find details of potential investors.

-> InvestorDetailsActivity.kt

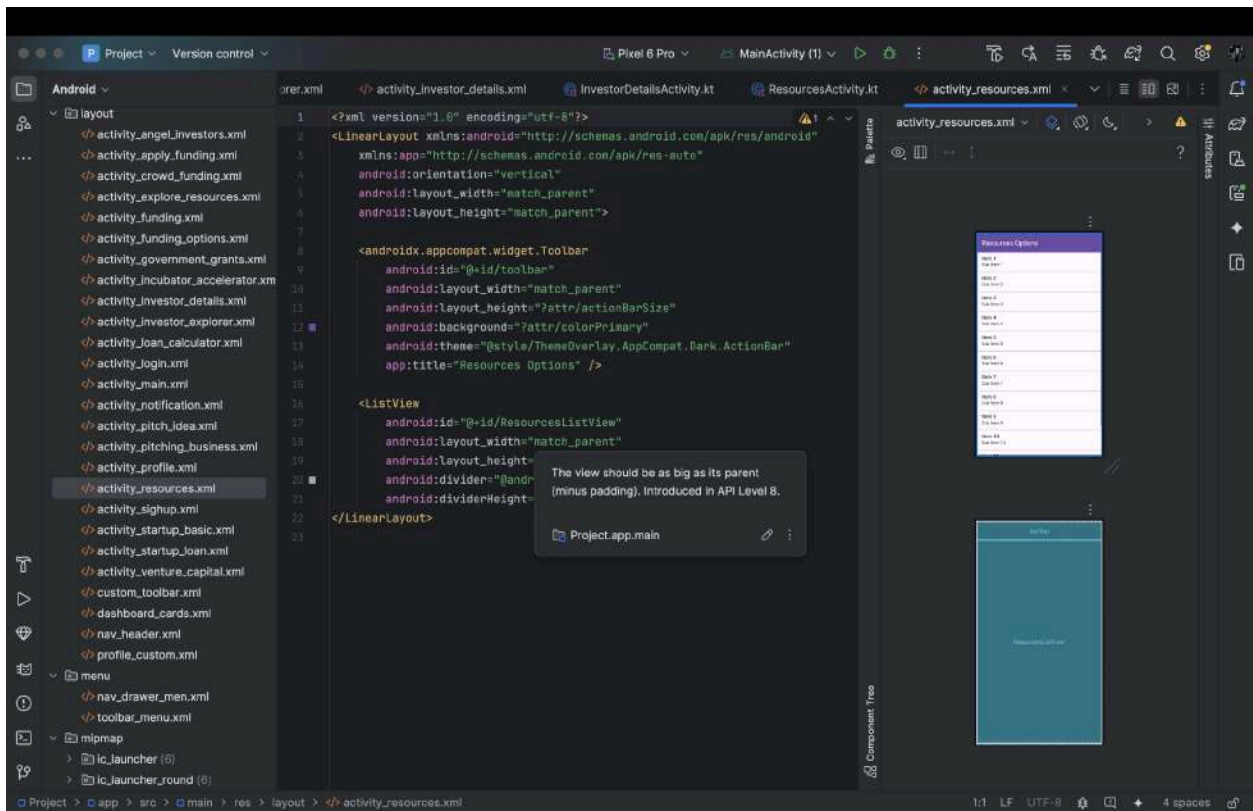
~ In the below code you can give whatever details you wanna add about investor there and it will show in the page.

```

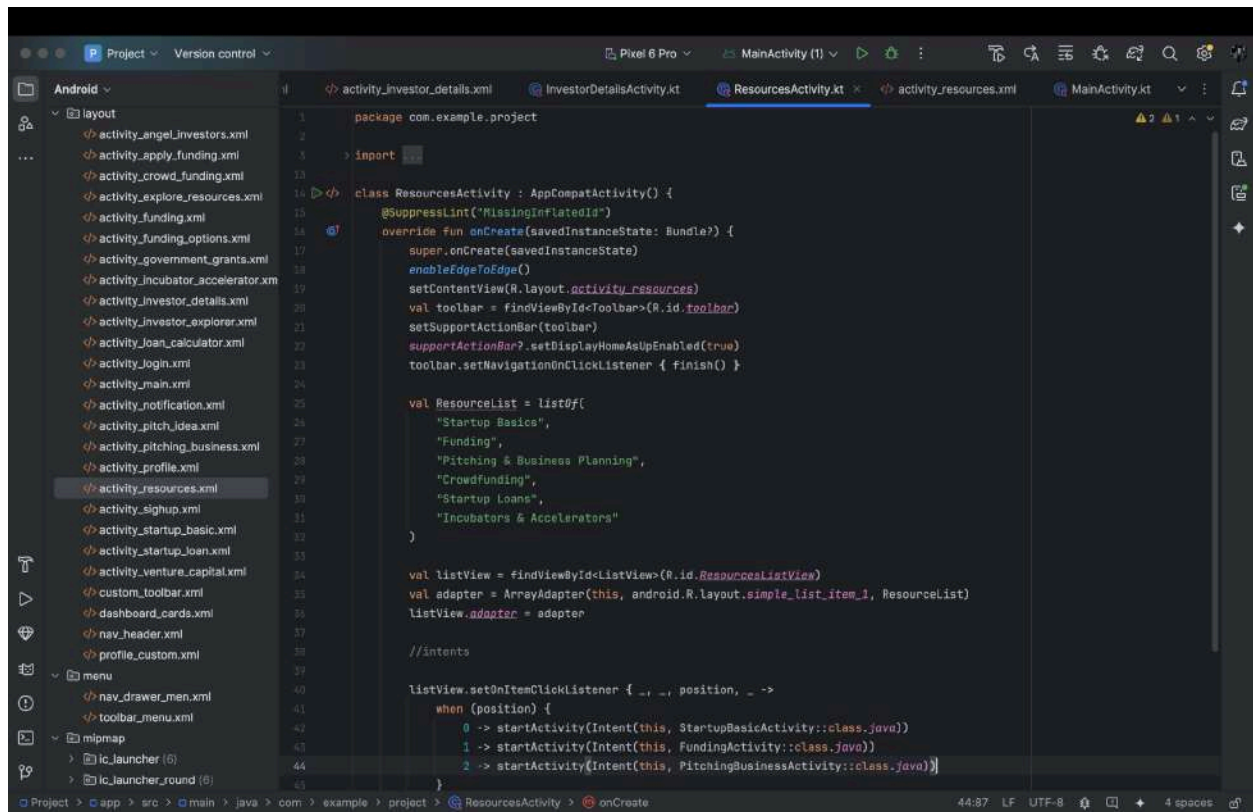
1 package com.example.project
2
3 > import ...
4
5
6
7 </> class InvestorDetailsActivity : AppCompatActivity() {
8
9     override fun onCreate(savedInstanceState: Bundle?) {
10         super.onCreate(savedInstanceState)
11         setContentView(R.layout.activity_investor_details)
12
13         // Get investor name from the intent
14         val investorName = intent.getStringExtra("INVESTOR_NAME")
15
16         // Set investor name in the TextView
17         val investorNameTextView = findViewById<TextView>(R.id.investorName)
18         investorNameTextView.text = investorName
19     }
20 }
21

```

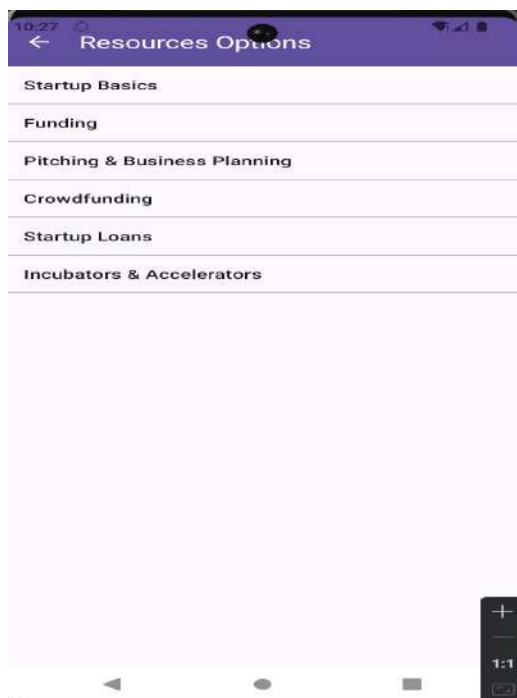
//Resource&LearningHub
->activity_resources.xml



-> ResourcesActivity.kt

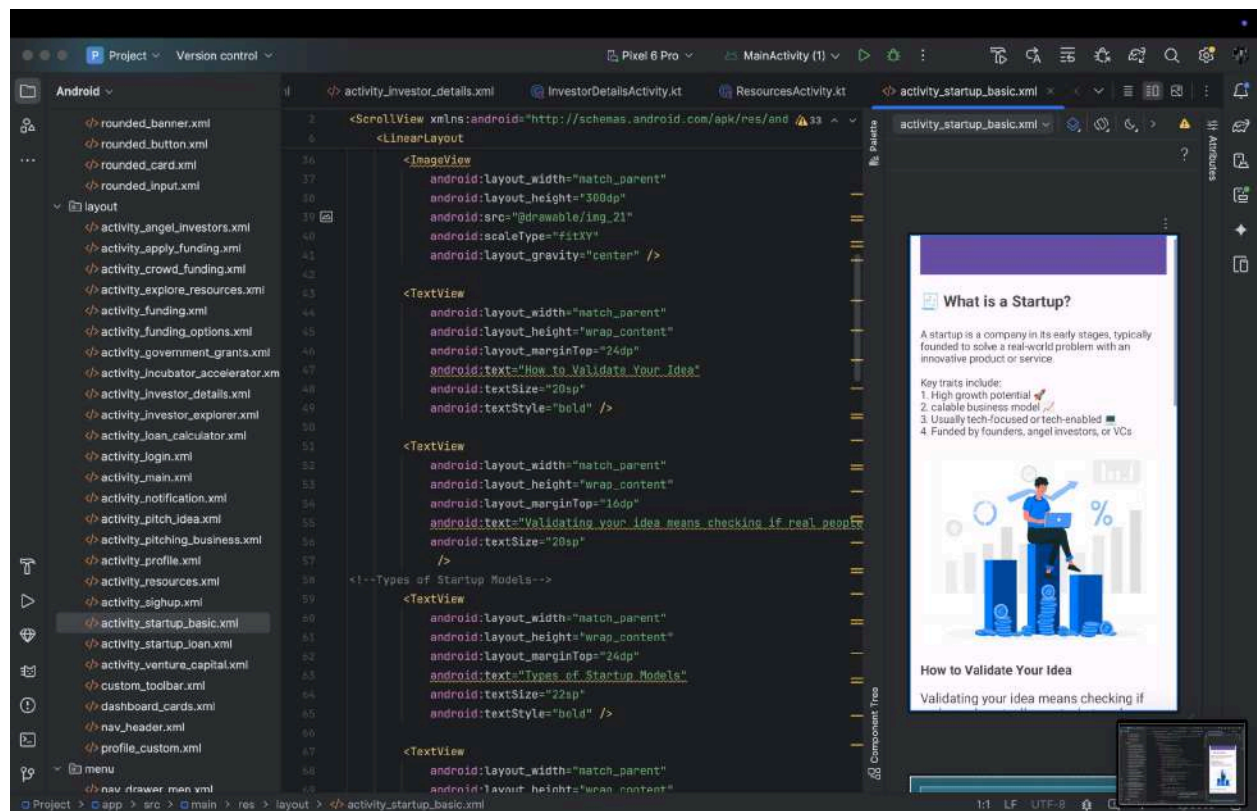
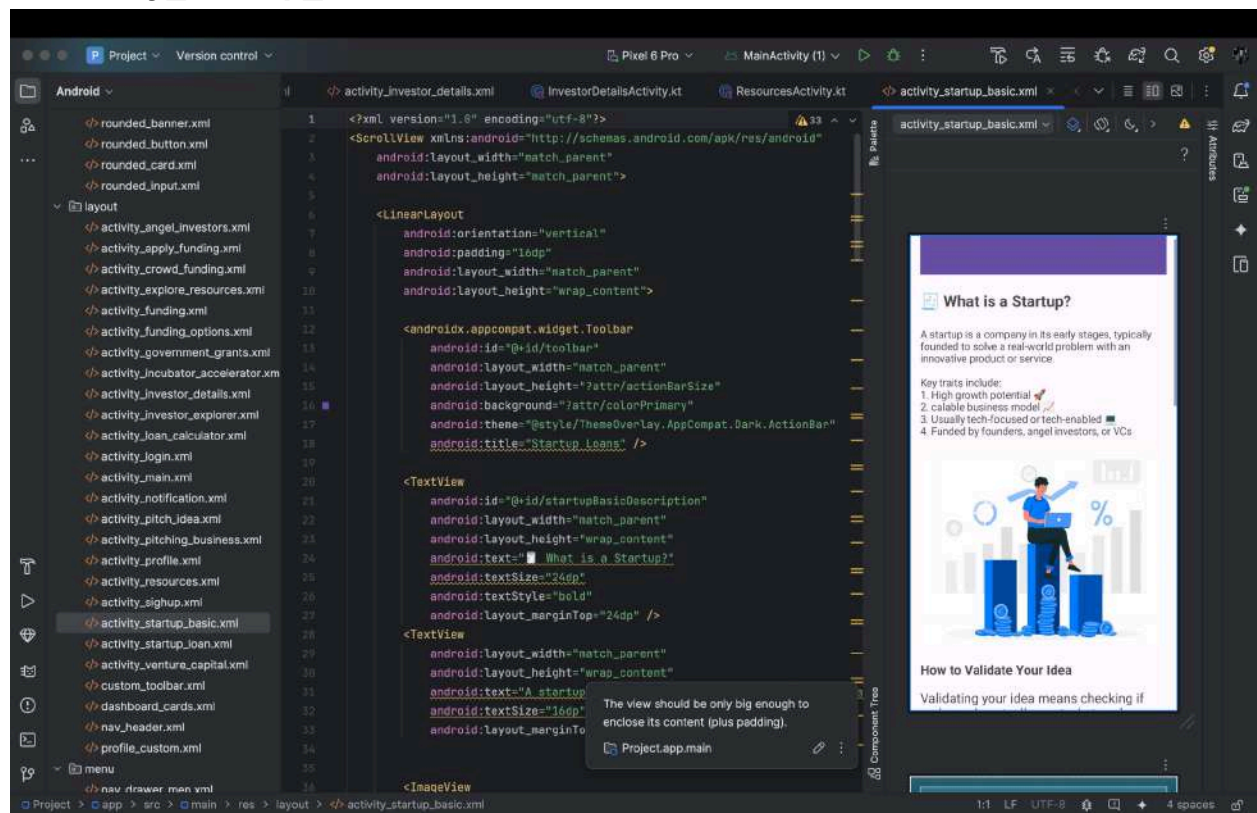


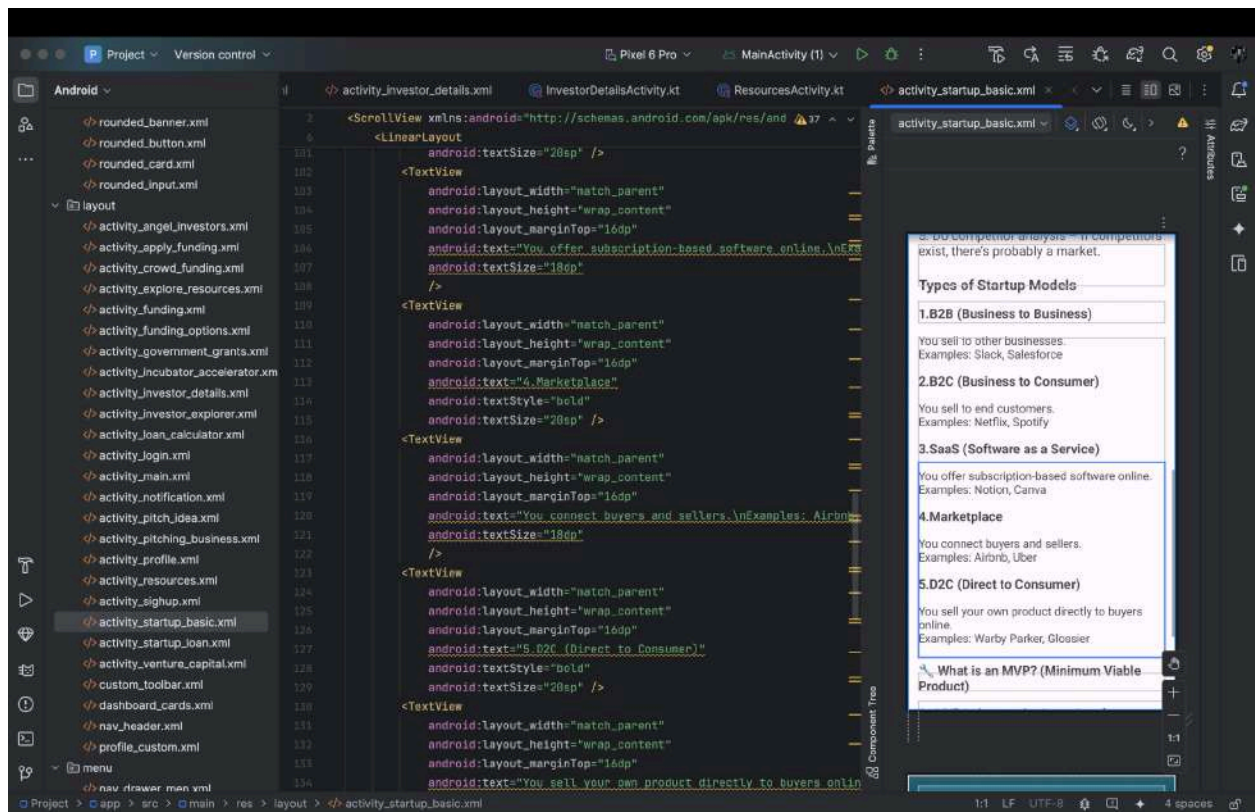
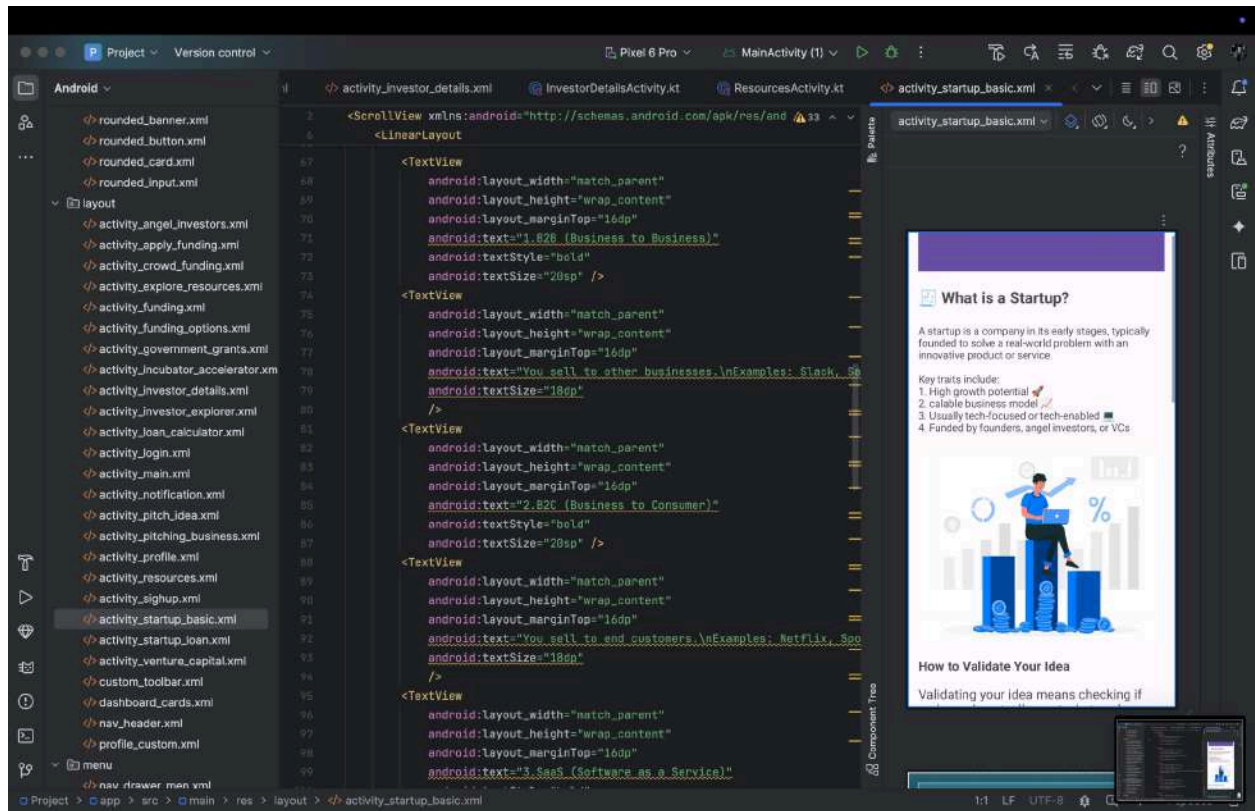
```
1 package com.example.project
2
3 import androidx.appcompat.app.AppCompatActivity
4 import androidx.appcompat.widget.Toolbar
5 import androidx.recyclerview.widget.LinearLayoutManager
6 import androidx.recyclerview.widget.RecyclerView
7 import androidx.appcompat.widget.Toolbar
8
9 class ResourcesActivity : AppCompatActivity() {
10     @SuppressLint("MissingInflatedId")
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         enableEdgeToEdge()
14         setContentView(R.layout.activity_resources)
15         val toolbar = findViewById<Toolbar>(R.id.toolbar)
16         setSupportActionBar(toolbar)
17         supportActionBar?.setDisplayHomeAsUpEnabled(true)
18         toolbar.setNavigationOnClickListener { finish() }
19
20         val ResourceList = listOf(
21             "Startup Basics",
22             "Funding",
23             "Pitching & Business Planning",
24             "Crowdfunding",
25             "Startup Loans",
26             "Incubators & Accelerators"
27         )
28
29         val listView = findViewById<ListView>(R.id.ResourceListView)
30         val adapter = ArrayAdapter(this, android.R.layout.simple_list_item_1, ResourceList)
31         listView.adapter = adapter
32
33         //Intents
34
35         listView.setOnItemClickListener { _, _, position, _ ->
36             when (position) {
37                 0 -> startActivity(Intent(this, StartupBasicActivity::class.java))
38                 1 -> startActivity(Intent(this, FundingActivity::class.java))
39                 2 -> startActivity(Intent(this, PitchingBusinessActivity::class.java))
40             }
41         }
42     }
43 }
```



This is the app version of the above code and now if we click on any of the option it will take you to the information related to it which will help people with less knowledge to gain and study

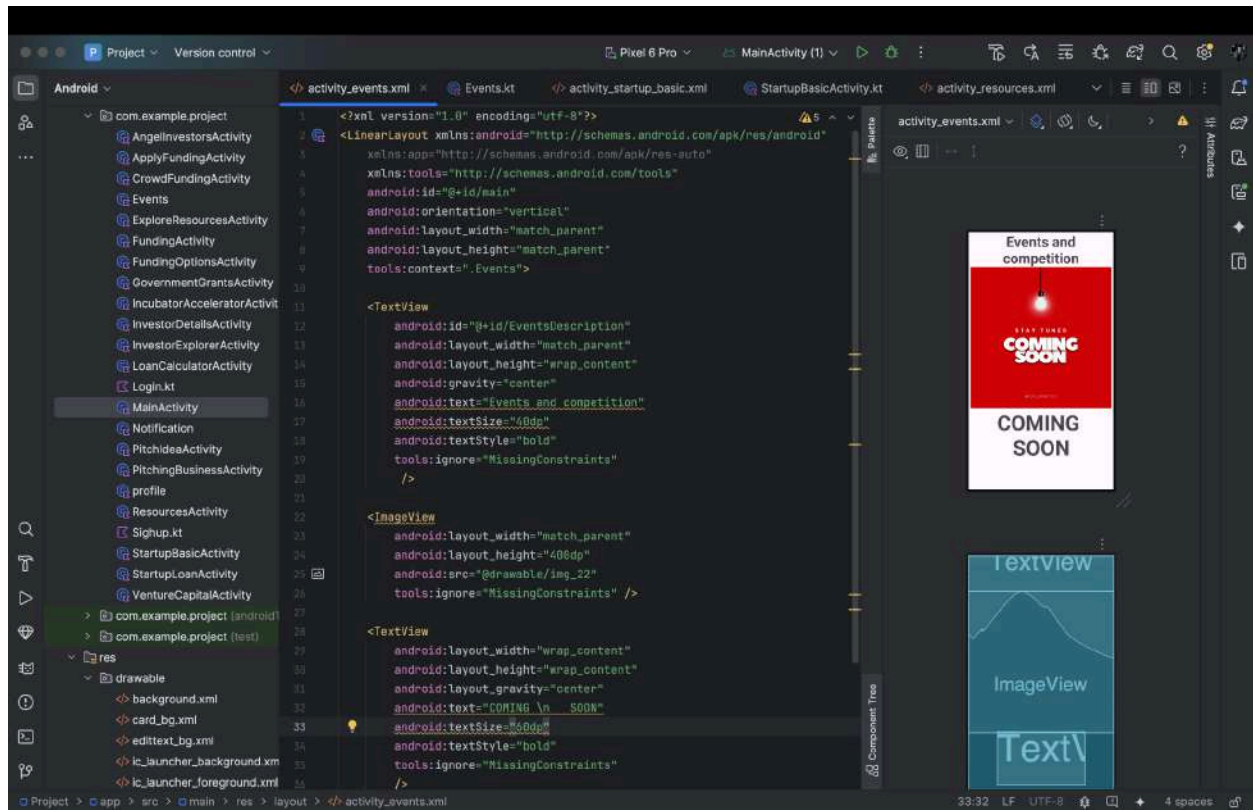
-> activity_startup_basics.xml





!! It is same for all other options i used same layout and kotlin code but changed the information related to the respective topic

->activity_events.xml



-> Events.kt

